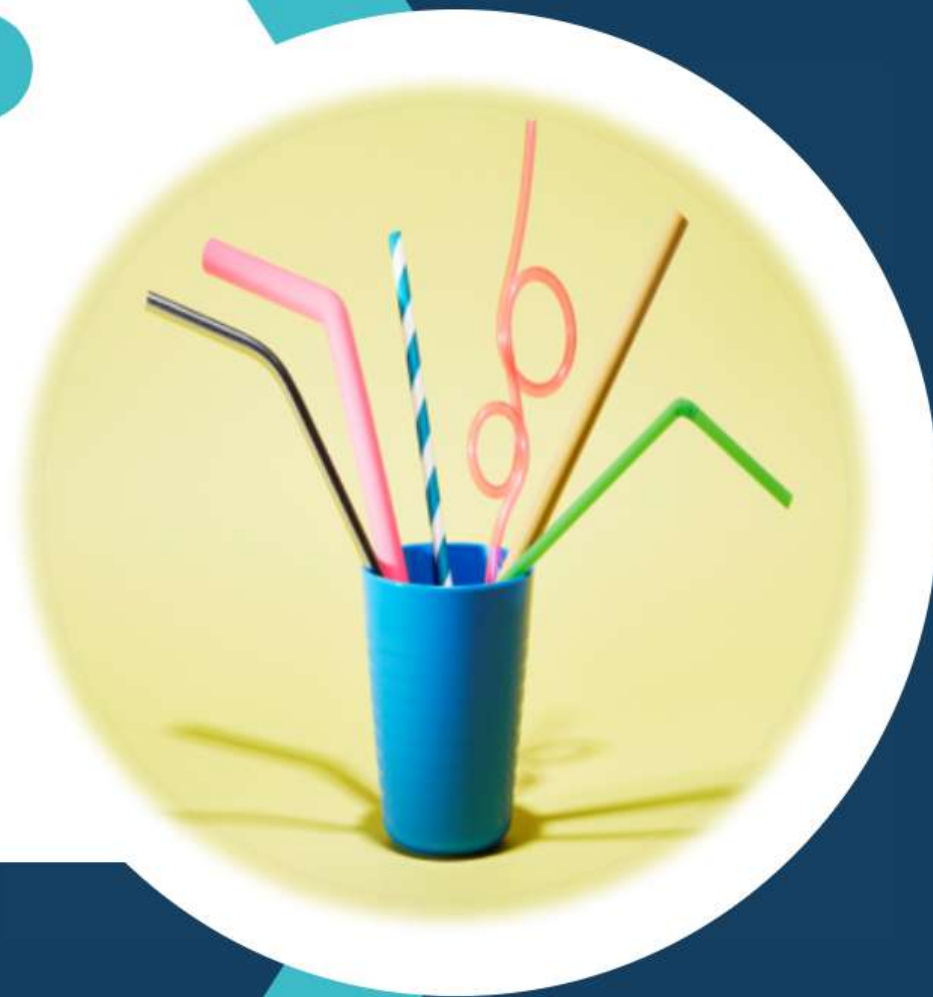


תכנות מונחה עצמים

הכלת עצמים



ביחידה זו נלמד:

○ אובייקטים כתכונות של מחלקה / הכלת אובייקטים:

- אובייקט כתכונה
- מערך פשוט כתכונה
- מערך של אובייקטים כתכונה
- מערך דו-ממדי כתכונה

○ השמת אובייקטים

○ ניהול נתונים פנימי

ביחידה זו נלמד:

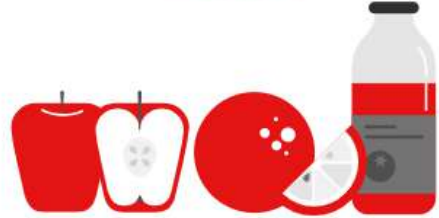
- אובייקטים כתכונות של מחלקה / הכלת אובייקטים:
 - אובייקט כתכונה
 - מערך פשוט כתכונה
 - מערך של אובייקטים כתכונה
 - מערך דו-ממדי כתכונה
- השמת אובייקטים
- ניהול נתונים פנימי
- שמירת הפניה של אובייקטים לעומת העתקה

הכלת עצמים

מחלקה המכילה עצם פשוט



תכנות מונחה עצמים - תזכורת



○ הבסיס התפיסתי של תכנות מונחה עצמים הוא שהעולם מאורגן בעצמים (אובייקטים)

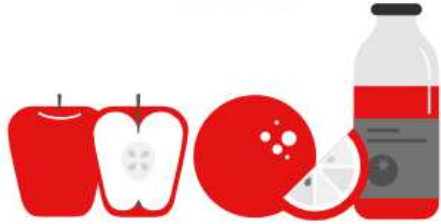
○ כל דבר הוא אובייקט

○ לכל אובייקט יש מאפיינים המייצגים אותו – לאובייקטים מאותו טיפוס, יש אותם מאפיינים

○ לכל אובייקט יש פעולות שהוא יודע לעשות - כל האובייקטים מאותו טיפוס יכולים לבצע אותן פעולות



תכנות מונחה עצמים - תזכורת



○ הבסיס התפיסתי של תכנות מונחה עצמים הוא שהעולם מאורגן בעצמים (אובייקטים)

○ כל דבר הוא אובייקט

את המאפיינים תרגמנו לתכונות -
משתנים מטיפוס המאפיין.

○ לכל אובייקט יש מאפיינים המייצגים אותו – לאובייקטים מאותו טיפוס, יש אותם מאפיינים

○ לכל אובייקט יש פעולות שהוא יודע לעשות - כל האובייקטים מאותו טיפוס יכולים לבצע אותן פעולות



מאפייני הטיפוס – תכונות המחלקה

○ תכונות המחלקה יכולות להיות מכל טיפוס חוקי בשפה, פרימיטיבי או משתנה שהוא הפניה למחלקה אחרת:

```
class Book
{
    private string title;
    private int year;
    private Author author;
}
```

למחלקה ספר שלושה מאפיינים – כותרת (מחרוזת), שנת הוצאה (מספר שלם) וסופר – עצם ממחלקת סופר

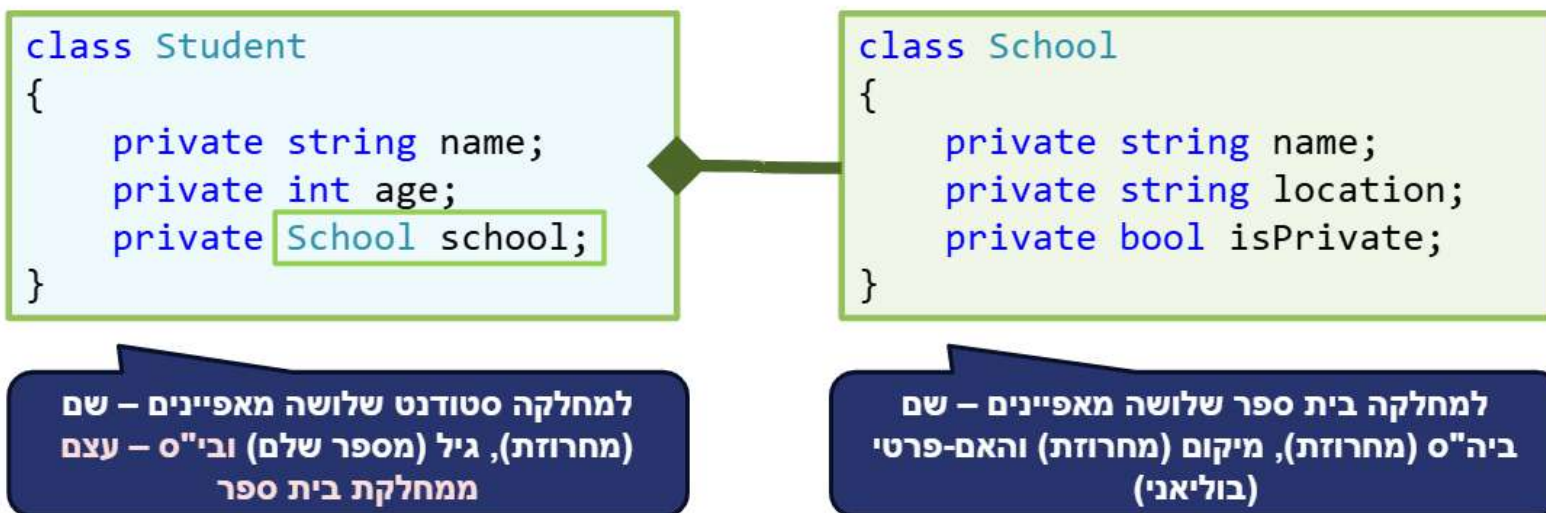
```
class Author
{
    private string name;
    private string nationality;
}
```

למחלקה סופר שני מאפיינים – שם הסופר (מחרוזת) ולאום (מחרוזת)

בין המחלקה Book למחלקה Author קיים קשר של הכלה
Book מכילה עצם מטיפוס Author

מאפייני הטיפוס – תכונות המחלקה

○ תכונות המחלקה יכולות להיות מכל טיפוס חוקי בשפה, פרימיטיבי או משתנה שהוא הפניה למחלקה אחרת:



בין המחלקה Student למחלקה School קיים קשר של הכלה
Student מכילה עצם מטיפוס School

מאפייני הטיפוס – תכונות המחלקה

○ תכונות המחלקה יכולות להיות מכל טיפוס חוקי בשפה, פרימיטיבי או משתנה שהוא הפניה למחלקה אחרת:

```
class Employee
{
    private string name;
    private int employeeId;
    private double salary;
    private Department depart;
}
```

למחלקה עובד ארבעה מאפיינים – שם (מחוזת), מס' עובד (מספר שלם), שכר (מס' ממשי) ומחלקה – עצם ממחלקת מחלקה

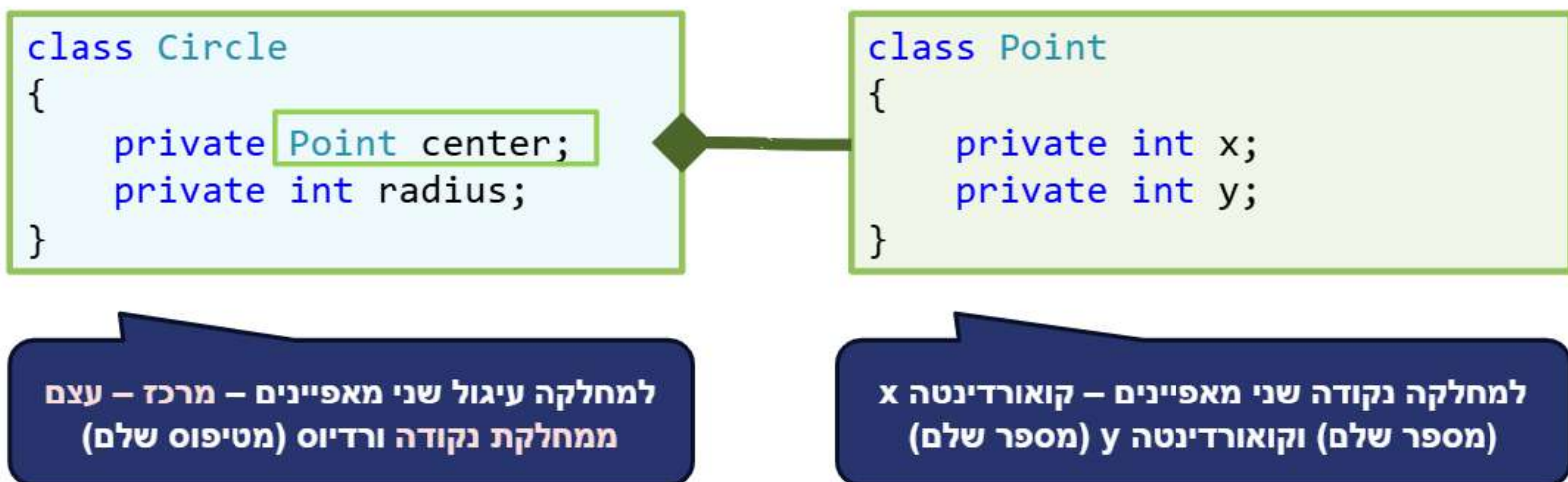
```
class Department
{
    private string name;
    private string location;
}
```

למחלקה מחלקה שני מאפיינים – שם המחלקה (מחוזת) ומיקום (מחוזת)

בין המחלקה Employee למחלקה Department קיים קשר של הכלה
Employee מכילה עצם מטיפוס Department

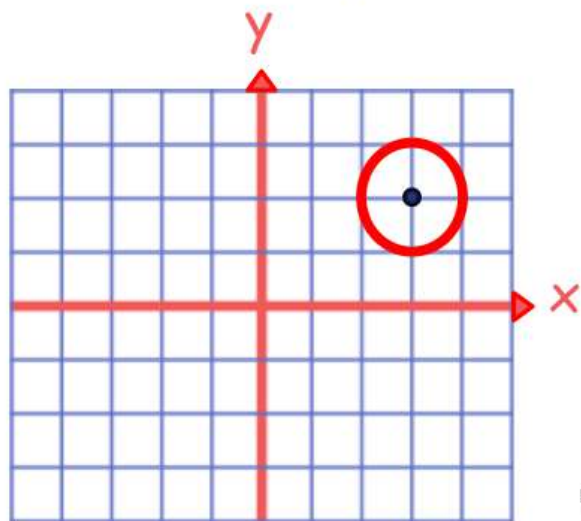
מאפייני הטיפוס – תכונות המחלקה

○ תכונות המחלקה יכולות להיות מכל טיפוס חוקי בשפה, פרימיטיבי או משתנה שהוא הפניה למחלקה אחרת:



בין המחלקה **Circle** למחלקה **Point** קיים קשר של הכלה
Circle מכילה עצם מטיפוס **Point**

מחלקה עם תכונה שהיא עצם – תכונות המחלקה



○ מחלקה שאחת התכונות שלה היא עצם ממחלקה אחרת

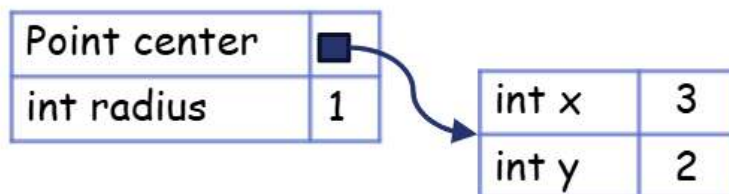
• למשל, במחלקת **עיגול** (**Circle**) התכונות יהיו:

מרכז העיגול - מטיפוס **נקודה** (**Point**)

אורך הרדיוס - מטיפוס מספר שלם

• תכונות המחלקה **נקודה** (**Point**) – קואורדינטות x, y

○ כאשר אובייקט מכיל אובייקט אחר, הוא למעשה מכיל הפניה לאובייקט המוכל



`class Point`

```
{
    private int x, y;

    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
    public override string ToString()
    {
        return "(" + x + ", " + y + ")";
    }
    public void SetX(int x) {this.x = x;}
    public int GetX() {return x;}
    public void SetY(int y) {this.y = y;}
    public int GetY() {return y;}
} // class Point
```

`class Circle`

```
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = center;
        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter() {return center;}
} // class Circle
```

הבנאי מקבל הפניה לנקודה,
ומעתיק את ההפניה לתכונה
center

הפעולה מחזירה את ההפניה
לתכונה center

```

class Point
{
    private int x, y;

    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
    public override string ToString()
    {
        return "(" + x + ", " + y + ")";
    }
    public void SetX(int x) {this.x = x;}
    public int GetX()      {return x;}
    public void SetY(int y) {this.y = y;}
    public int GetY()      {return y;}
} // class Point

```

```

class Circle
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = center;
        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter() {return center;}
} // class Circle

```

x=1
y=1

Point: p	
Circle: c	

```

static void Main(string[] args)
{
    Point p = new Point(1, 1);
    Circle c = new Circle(p, 5);
    p.SetX(0);
    c.GetCenter()
}

```

```

class Point
{
    private int x, y;

    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
    public override string ToString()
    {
        return "(" + x + ", " + y + ")";
    }
    public void SetX(int x) {this.x = x;}
    public int GetX() {return x;}
    public void SetY(int y) {this.y = y;}
    public int GetY() {return y;}
} // class Point

```

```

class Circle
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = center;
        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter() {return center;}
} // class Circle

```

x=1
y=1

center =
radius = 0

Point: p	
Circle: c	

```

static void Main(string[] args)
{
    Point p = new Point(1, 1);
    Circle c = new Circle(p, 5);
    p.SetX(0);
    c.GetCenter()
}

```

```

class Point
{
    private int x, y;

    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
    public override string ToString()
    {
        return "(" + x + ", " + y + ")";
    }
    public void SetX(int x) {this.x = x;}
    public int GetX() {return x;}
    public void SetY(int y) {this.y = y;}
    public int GetY() {return y;}
} // class Point

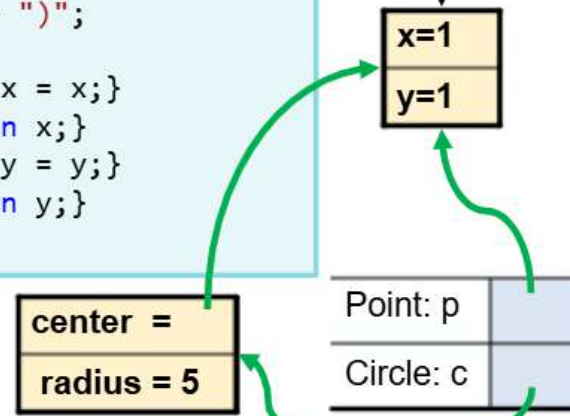
```

```

class Circle
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = center;
        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter() {return center;}
} // class Circle

```



```

static void Main(string[] args)
{
    Point p = new Point(1, 1);
    Circle c = new Circle(p, 5);
    p.SetX(0);
    c.GetCenter()
}

```

```

class Point
{
    private int x, y;

    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
    public override string ToString()
    {
        return "(" + x + ", " + y + ")";
    }
    public void SetX(int x) {this.x = x;}
    public int GetX() {return x;}
    public void SetY(int y) {this.y = y;}
    public int GetY() {return y;}
} // class Point

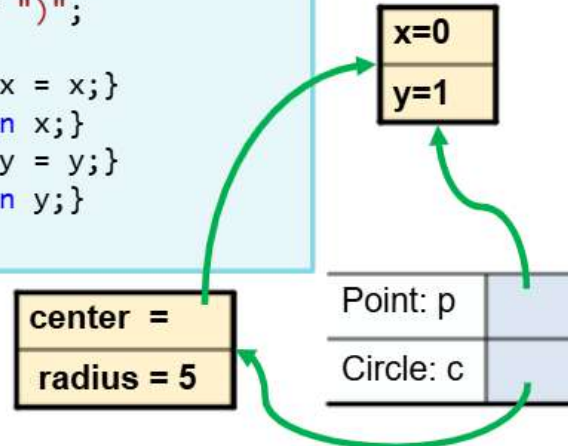
```

```

class Circle
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = center;
        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter() {return center;}
} // class Circle

```



```

static void Main(string[] args)
{
    Point p = new Point(1, 1);
    Circle c = new Circle(p, 5);
    p.SetX(0);
    c.GetCenter()
}

```

```

class Point
{
    private int x, y;

    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
    public override string ToString()
    {
        return "(" + x + ", " + y + ")";
    }
    public void SetX(int x) {this.x = x;}
    public int GetX() {return x;}
    public void SetY(int y) {this.y = y;}
    public int GetY() {return y;}
} // class Point

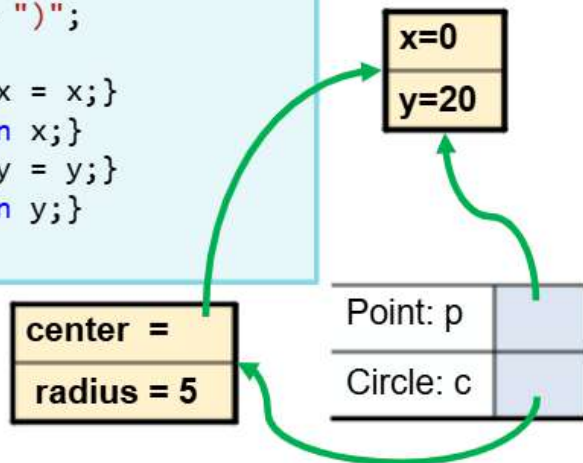
```

```

class Circle
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = center;
        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter() {return center;}
} // class Circle

```

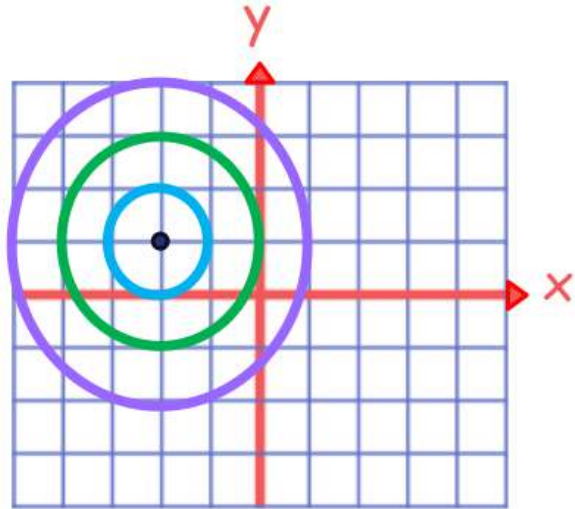


```

static void Main(string[] args)
{
    Point p = new Point(1, 1);
    Circle c = new Circle(p, 5);
    p.SetX(0);
    c.GetCenter().SetY(20);
}

```

הכלת מחלקות – הבעיה



- העצם Circle מכיל הפניה center לעצם מוכל מטיפוס Point, אליו קיימת הפניה p גם במקום אחר (ב Main)
- שינוי בעצם דרך אחת ההפניות, משפיע על העצם
- אם נשתמש באותה נקודה כנקודת המרכז של כמה עיגולים, ונרצה להזיז את אחד העיגולים, כל העיגולים יזוזו ימינה!
- ניתן למנוע מצב זה, ע"י יצירת עצם חדש (נקודה חדשה) בערכי הנקודה המועברת כפרמטר

```
class Point
{
    private int x, y;

    public Point(int x, int y)
    {
        this.x = x
        this.y = y
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
    public override string ToString()
    {
        return "(" + x + ", " + y + ")";
    }
    public void SetX(int x) {this.x = x;}
    public int GetX()      {return x;}
    public void SetY(int y) {this.y = y;}
    public int GetY()      {return y;}
} // class Point
```

```
class Circle
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = new Point(center);
        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter()
    {
        return new Point(center);
    }
} // class Circle
```

```

class Point
{
    private int x, y;

    public Point(int1 x, int1 y)
    {
        this.x = x;
        this.y = y;
    }

    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
}
...
} // class Point

```

```

static void Main(string[] args)
{
    Point p = new Point(1, 1);
    Circle c = new Circle(p, 5);
    p.SetX(0);
    c.GetCenter()
}

```

```

class Circle
{
    private Point center;
    private int radius;

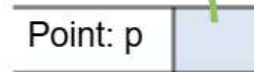
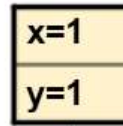
    public Circle(Point center, int radius)
    {
        this.center = new Point(center);

        this.radius = radius;
    }

    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }

    public int GetRadius() {return radius;}
    public Point GetCenter()
    {
        return new Point(center);
    }
} // class Circle

```



```
class Point
{
    private int x, y;

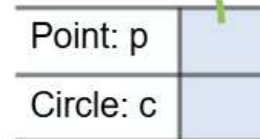
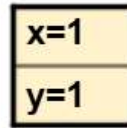
    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
}
...
} // class Point
```

```
static void Main(string[] args)
{
    Point p = new Point(1, 1);
    Circle c = new Circle(p, 5);
    p.SetX(0);
    c.GetCenter()
}
```

```
class Circle
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = new Point(center);

        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter()
    {
        return new Point(center);
    }
} // class Circle
```



```

class Point
{
    private int x, y;

    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
}
...
} // class Point

```

```

static void Main(string[] args)
{
    Point p = new Point(1, 1);
    Circle c = new Circle(p, 5);
    p.SetX(0);
    c.GetCenter()
}

```

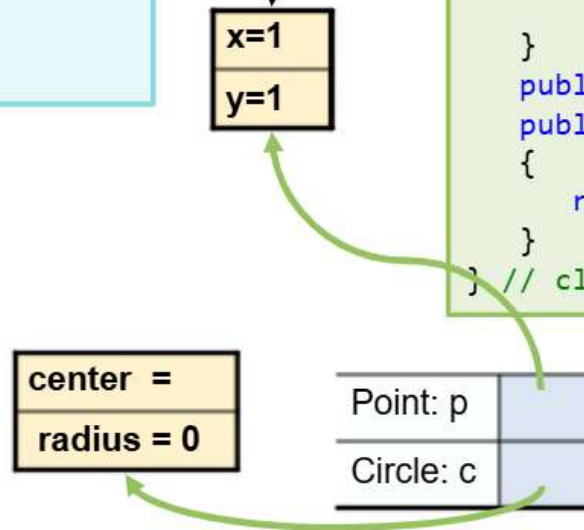
```

class Circle
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = new Point(center);

        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter()
    {
        return new Point(center);
    }
}
// class Circle

```



```

class Point
{
    private int x, y;

    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
}
...
} // class Point

```

```

static void Main(string[] args)
{
    Point p = new Point(1, 1);
    Circle c = new Circle(p, 5);
    p.SetX(0);
    c.GetCenter()
}

```

```

class Circle
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = new Point(center);

        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter()
    {
        return new Point(center);
    }
} // class Circle

```

x=0
y=0

x=1
y=1

center =
radius = 0

Point: p	
Circle: c	

```

class Point
{
    private int x, y;

    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
    ...
} // class Point

```

```

static void Main(string[] args)
{
    Point p = new Point(1, 1);
    Circle c = new Circle(p, 5);
    p.SetX(0);
    c.GetCenter()
}

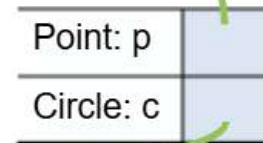
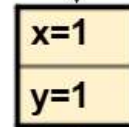
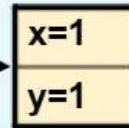
```

```

class Circle
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = new Point(center);
        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter()
    {
        return new Point(center);
    }
} // class Circle

```



```

class Point
{
    private int x, y;

    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
}
...
} // class Point

```

```

static void Main(string[] args)
{
    Point p = new Point(1, 1);
    Circle c = new Circle(p, 5);
    p.SetX(0);
    c.GetCenter()
}

```

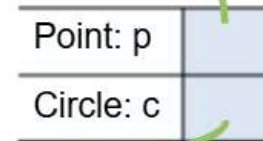
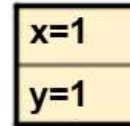
```

class Circle
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = new Point(center);

        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter()
    {
        return new Point(center);
    }
} // class Circle

```



```

class Point
{
    private int x, y;

    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
}
...
} // class Point

```

```

static void Main(string[] args)
{
    Point p = new Point(1, 1);
    Circle c = new Circle(p, 5);
    p.SetX(0);
    c.GetCenter()
}

```

```

class Circle
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = new Point(center);

        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter()
    {
        return new Point(center);
    }
} // class Circle

```

x=1
y=1

x=0
y=1

center =
radius = 5

Point: p	
Circle: c	

```

class Point
{
    private int x, y;

    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
    public Point(Point other)
    {
        this.x = other.x;
        this.y = other.y;
    }
}
// class Point

```

```

static void Main(string[] args)
{
    Point p = new Point(1, 1);
    Circle c = new Circle(p, 5);
    p.SetX(0);
    c.GetCenter().SetY(20);
}

```

```

class Circle
{
    private Point center;
    private int radius;

    public Circle(Point center, int radius)
    {
        this.center = new Point(center);

        this.radius = radius;
    }
    public override string ToString()
    {
        return "center: " + center.ToString()
            + ", radius: " + radius;
    }
    public int GetRadius() {return radius;}
    public Point GetCenter()
    {
        return new Point(center);
    }
}
// class Circle

```

x=0
y=20

x=0
y=1

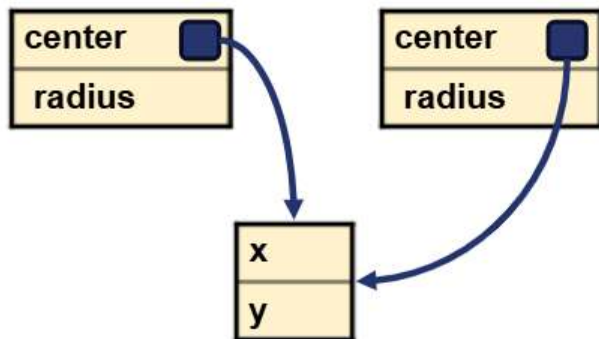
center =
radius = 5

Point: p	
Circle: c	

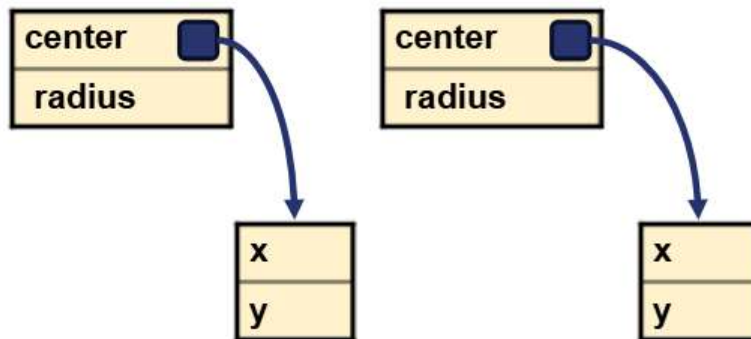
הכלת מחלקות – פעולה בונה מעתיקה

פעולה בונה מעתיקה – בונה עצם חדש שתכונותיו זהות לעצם המועבר כפרמטר

```
public Circle(Circle other)
{
    this.center = other.center;
    this.radius = other.radius;
}
```



```
public Circle(Circle other)
{
    this.center = new Point(other.center);
    this.radius = other.radius;
}
```

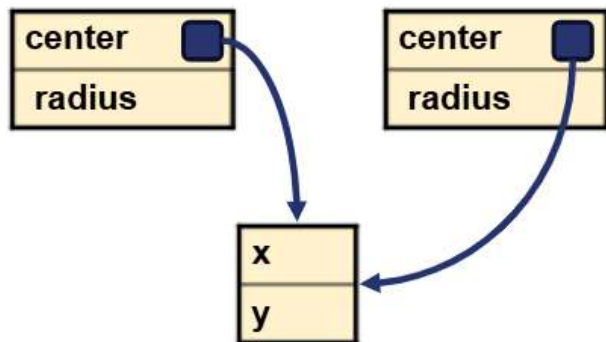


הכלת מחלקות – פעולה בונה מעתיקה

○ פעולה בונה מעתיקה – בונה עצם חדש שתכונותיו זהות לעצם המועבר כפרמטר

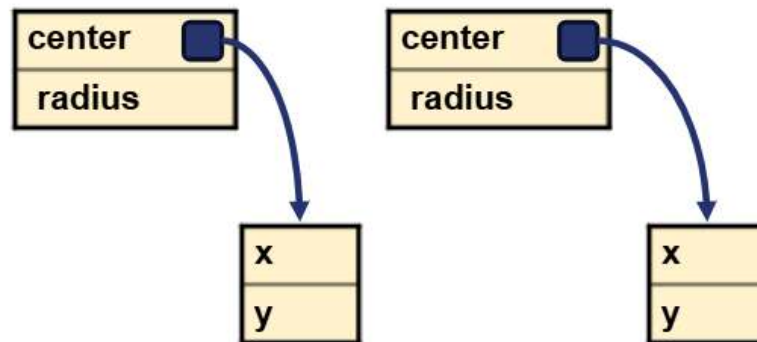
העתקה שטוחה

```
public Circle(Circle other)
{
    this.center = other.center;
    this.radius = other.radius;
}
```

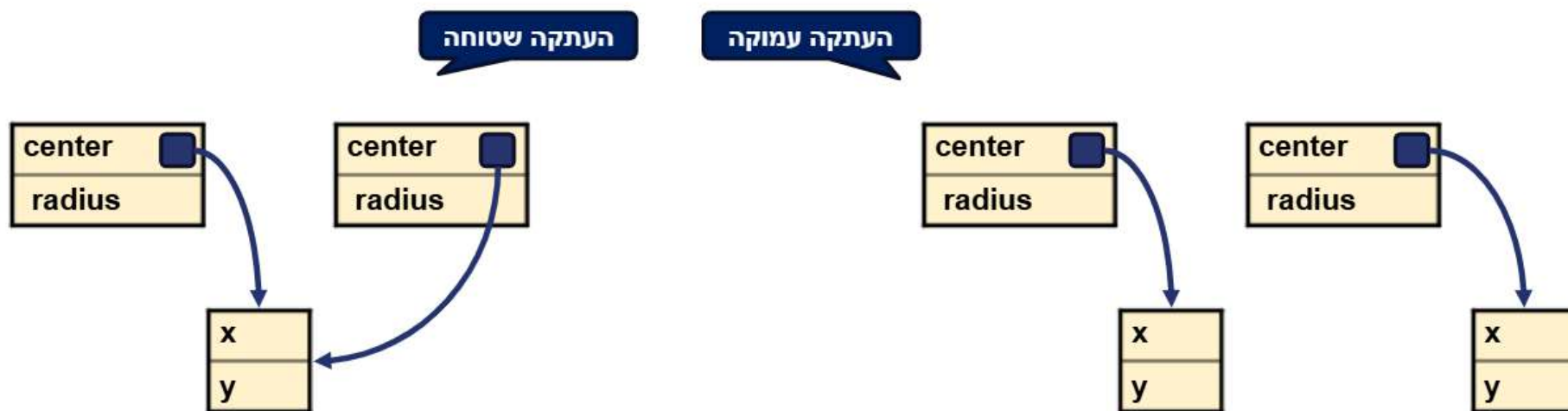


העתקה עמוקה

```
public Circle(Circle other)
{
    this.center = new Point(other.center);
    this.radius = other.radius;
}
```

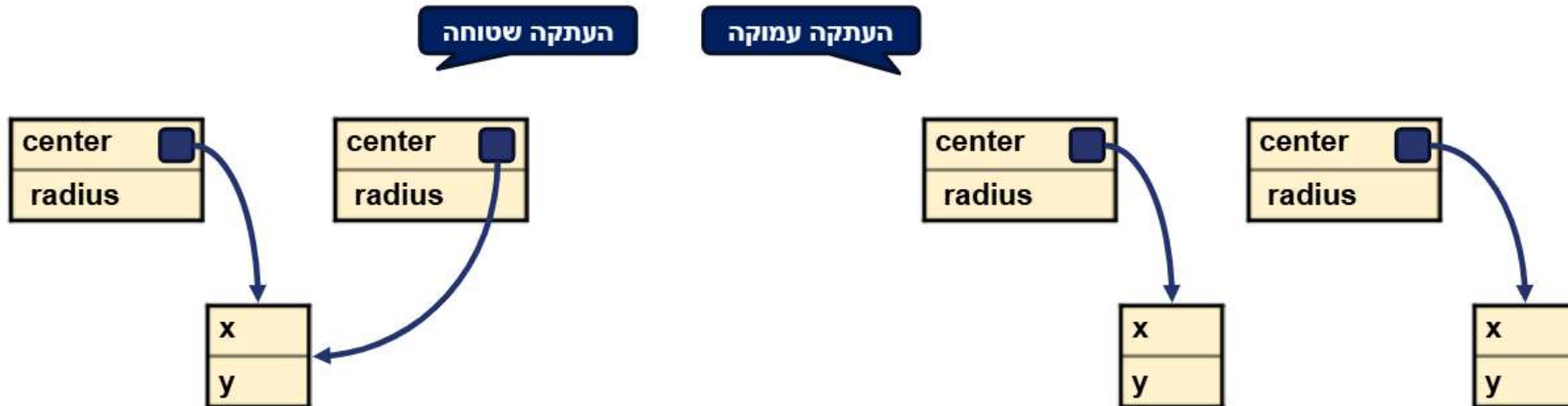


הכלת מחלקות – פעולה בונה מעתיקה



הכלת מחלקות – פעולה בונה מעתיקה

- מה נכון? תלוי בנסיבות
- לעיתים נרצה העתקה עמוקה ולעיתים שטוחה – האם מדובר באותו עצם ממש או שיש צורך בשניים שונים?
- לדוגמה - המחלקה **שיעור** מכילה עצם **זמן** המצביע על מועד תחילת השיעור – אם שיעור ראשון מתחיל ב 8:15, נרצה שכל השיעורים המתחילים ב 8:15 יכילו אותו עצם, כך שאם יהיה שינוי בשעת ההתחלה בבית הספר כולו (למשל, יתחיל ב 8:30), תהיה השפעה על כל השיעורים





דוגמה – מחלקת עובד Worker

- לכל עובד נשמרות תכונות כמו שם, ת"ז טלפון וכו' (בהתאם לצורך) ובנוסף תכונות מורכבות כמו **תלוש שכר** וכן **רכב חברה**
- השכר משויך ישירות לעובד, ורכב החברה שייך לחברה, אך נמסר בהשאלה לעובד כהטבה
- העובד יכול לגשת לנתוני הרכב כתכונה שלו, אך גם האחראי על צי הרכב יכול לגשת לרכב, עבור טיפולי רכב
- עובד שעוזב את החברה, נתוניו נמחקים, אך הרכב נותר בחברה
- כדי לכתוב את מחלקת עובד, נכתוב תחילה את מחלקת **שכר** וכן את מחלקת **רכב**

```
class Car
{
    private string model;
    private int licenseNumber;
    private bool isOk;

    public Car(string model, int licenseNumber, bool isOk)
    {
        this.model = model;
        this.licenseNumber = licenseNumber;
        this.isOk = isOk;
    }
    // פעולות קובעות מאחזרות ועוד פעולות לפי הצורך
    ...
    public void SetState(bool isOk)
    {
        this.isOk = isOk;
    }
    public override string ToString()
    {
        return " Car: " + this.model +
            "(" + this.licenseNumber + ") is : " + this.isOk;
    }
}
```

דוגמה - עובד

מחלקת Car

```

class Salary
{
    private int month;
    private int year;
    private double payment;

    public Salary(int month, int year, double payment)
    {
        this.month = month;
        this.year = year;
        this.payment = payment;
    }
    public Salary(Salary other)
    {
        this.month = other.month;
        this.year = other.year;
        this.payment = other.payment;
    }
    // פעולות קובעות מאחזרות ועוד פעולות לפי הצורך
    ...
    public override string ToString()
    {
        return this.month + "/" + this.year +
            " payment: " + this.payment;
    }
}

```

דוגמה - עובד

מחלקת Salary

```
class Worker
```

```
{
```

```
    private string name;  
    private Salary salary;  
    private Car car;
```

```
    public Worker(string name, Salary salary, Car car)
```

```
    {
```

```
        this.name = name;  
        this.salary = new Salary(salary);  
        this.car = car;
```

```
    }
```

```
    public void SetSalary(Salary salary)  
    { this.salary = new Salary(salary); }
```

```
    public Salary GetSalary()  
    { return new Salary(this.salary) ; }
```

```
    public void SetCar(Car car)
```

```
    { this.car = car; }  
    public Car GetCar()  
    { return this.car; }
```

```
    public override string ToString()
```

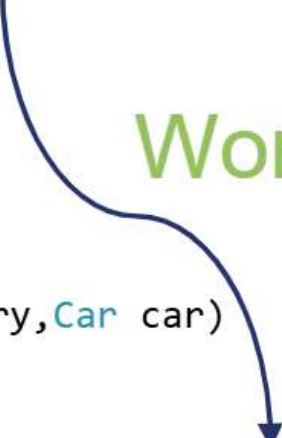
```
    { return this.name+ " has car:" + this.car + " his salary: " + this.salary; }
```

```
}
```

דוגמה – מחלקת עובד Worker

דוגמה – מחלקת עובד Worker

salary



int month: 11
int year: 2022
double payment: 9800.7

string: "name"
Salary: salary
Car: car

```
class Worker
{
    private string name;
    private Salary salary;
    private Car car;

    public Worker(string name, Salary salary, Car car)
    {
        this.name = name;
        this.salary = new Salary(salary);
        this.car = car;
    }

    public void SetSalary(Salary salary)
    { this.salary = new Salary(salary); }
    public Salary GetSalary()
    { return new Salary(this.salary) ; }

    public void SetCar(Car car)
    { this.car = car; }
    public Car GetCar()
    { return this.car; }

    public override string ToString()
    { return this.name+ " has car:" + this.car + " his salary: " + this.salary; }
}
```

```
class Worker
```

```
{
```

```
    private string name;  
    private Salary salary;  
    private Car car;
```

```
    public Worker(string name, Salary salary, Car car)  
    {
```

```
        this.name = name;
```

```
        this.salary = new Salary(salary);
```

```
        this.car = car;
```

```
    }
```

```
    public void SetSalary(Salary salary)  
    { this.salary = new Salary(salary); }
```

```
    public Salary GetSalary()  
    { return new Salary(this.salary) ; }
```

```
    public void SetCar(Car car)  
    { this.car = car; }
```

```
    public Car GetCar()  
    { return this.car; }
```

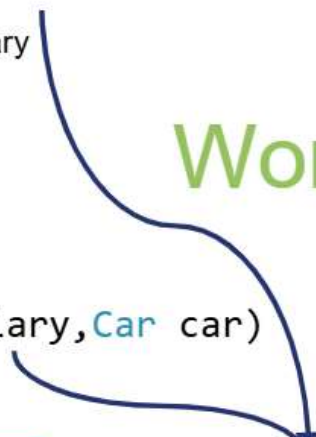
```
    public override string ToString()
```

```
    { return this.name+ " has car:" + this.car + " his salary: " + this.salary; }
```

```
}
```

salary

דוגמה – מחלקת עובד Worker



int month: 11
int year: 2022
double payment: 9800.7

string: "name"
Salary: salary
Car: car

```
class Worker
```

```
{
```

```
    private string name;  
    private Salary salary;  
    private Car car;
```

```
    public Worker(string name, Salary salary, Car car)
```

```
    {
```

```
        this.name = name;
```

```
        this.salary = new Salary(salary);
```

```
        this.car = car;
```

```
    }
```

```
    public void SetSalary(Salary salary)
```

```
    { this.salary = new Salary(salary); }
```

```
    public Salary GetSalary()
```

```
    { return new Salary(this.salary) ; }
```

```
    public void SetCar(Car car)
```

```
    { this.car = car; }
```

```
    public Car GetCar()
```

```
    { return this.car; }
```

```
    public override string ToString()
```

```
    { return this.name+ " has car:" + this.car + " his salary: " + this.salary; }
```

```
}
```

salary

דוגמה – מחלקת עובד Worker

int month: 11
int year: 2022
double payment: 9800.7

int month: 11
int year: 2022
double payment: 9800.7

string: "name"
Salary: salary
Car: car

```
class Worker
```

```
{  
    private string name;  
    private Salary salary;  
    private Car car;
```

דוגמה – מחלקת עובד Worker

```
public Worker(string name, Salary salary, Car car)  
{  
    this.name = name;  
    this.salary = new Salary(salary);  
    this.car = car;  
}
```

```
public void SetSalary(Salary salary)  
{ this.salary = new Salary(salary); }  
public Salary GetSalary()  
{ return new Salary(this.salary) ; }
```

```
public void SetCar(Car car)  
{ this.car = car; }  
public Car GetCar()  
{ return this.car; }
```

```
public override string ToString()  
{ return this.name+ " has car:" + this.car + " his salary: " + this.salary; }
```

```
}
```

string: "name"
Salary: salary
Car: car

int month: 11
int year: 2022
double payment: 9800.7

```
class Worker
```

```
{  
    private string name;  
    private Salary salary;  
    private Car car;
```

```
public Worker(string name, Salary salary, Car car)  
{  
    this.name = name;  
    this.salary = new Salary(salary);  
    this.car = car;  
}
```

```
public void SetSalary(Salary salary)  
{ this.salary = new Salary(salary); }  
public Salary GetSalary()  
{ return new Salary(this.salary) ; }
```

```
public void SetCar(Car car)  
{ this.car = car; }  
public Car GetCar()  
{ return this.car; }
```

```
public override string ToString()  
{ return this.name+ " has car:" + this.car + " his salary: " + this.salary; }
```

```
}
```

דוגמה – מחלקת עובד Worker

המשכורת Salary שייכת לעובד, ואין לה משמעות מחוץ לעצם. לכן, במחלקת Worker נשמור עותק שלה

string: "name"
Salary: salary
Car: car

int month: 11
int year: 2022
double payment: 9800.7

```
class Worker
```

```
{
```

```
    private string name;  
    private Salary salary;  
    private Car car;
```

```
    public Worker(string name, Salary salary, Car car)
```

```
    {
```

```
        this.name = name;  
        this.salary = new Salary(salary);  
        this.car = car;
```

```
    }
```

```
    public void SetSalary(Salary salary)
```

```
    { this.salary = new Salary(salary); }
```

```
    public Salary GetSalary()
```

```
    { return new Salary(this.salary) ; }
```

```
    public void SetCar(Car car)
```

```
    { this.car = car; }
```

```
    public Car GetCar()
```

```
    { return this.car; }
```

```
    public override string ToString()
```

```
    { return this.name+ " has car:" + this.car + " his salary: " + this.salary; }
```

```
}
```

דוגמה – מחלקת עובד Worker

המשכורת Salary שייכת לעובד, ואין לה משמעות מחוץ לעצם. לכן, במחלקת Worker נשמור עותק שלה

string: "name"
Salary: salary
Car: car

model: "ford"
licence: 12345
isOk: true

int month: 11
int year: 2022
double payment: 9800.7

car

```
class Worker
```

```
{  
    private string name;  
    private Salary salary;  
    private Car car;
```

דוגמה – מחלקת עובד Worker

המשכורת Salary שייכת לעובד, ואין לה משמעות מחוץ לעצם. לכן, במחלקת Worker נשמור עותק שלה

```
public Worker(string name, Salary salary, Car car)  
{  
    this.name = name;  
    this.salary = new Salary(salary);  
    this.car = car;  
}
```

```
public void SetSalary(Salary salary)  
{ this.salary = new Salary(salary); }  
public Salary GetSalary()  
{ return new Salary(this.salary) ; }
```

```
public void SetCar(Car car)  
{ this.car = car; }  
public Car GetCar()  
{ return this.car; }
```

```
public override string ToString()  
{ return this.name+ " has car:" + this.car + " his salary: " + this.salary; }
```

```
}
```

string: "name"
Salary: salary
Car: car

model: "ford"
licence: 12345
isOk: true

int month: 11
int year: 2022
double payment: 9800.7

car

```
class Worker
```

```
{
```

```
    private string name;  
    private Salary salary;  
    private Car car;
```

```
    public Worker(string name, Salary salary, Car car)
```

```
    {
```

```
        this.name = name;  
        this.salary = new Salary(salary);  
        this.car = car;
```

```
    }
```

```
    public void SetSalary(Salary salary)
```

```
    { this.salary = new Salary(salary); }
```

```
    public Salary GetSalary()
```

```
    { return new Salary(this.salary) ; }
```

```
    public void SetCar(Car car)
```

```
    { this.car = car; }
```

```
    public Car GetCar()
```

```
    { return this.car; }
```

```
    public override string ToString()
```

```
    { return this.name+ " has car:" + this.car + " his salary: " + this.salary; }
```

```
}
```

דוגמה – מחלקת עובד Worker

המשכורת Salary שייכת לעובד, ואין לה משמעות מחוץ לעצם. לכן, במחלקת Worker נשמור עותק שלה

string: "name"
Salary: salary
Car: car

model: "ford"
licence: 12345
isOk: true

int month: 11
int year: 2022
double payment: 9800.7

car

```
class Worker
```

```
{
```

```
    private string name;  
    private Salary salary;  
    private Car car;
```

```
    public Worker(string name, Salary salary, Car car)
```

```
    {
```

```
        this.name = name;  
        this.salary = new Salary(salary);  
        this.car = car;
```

```
    }
```

```
    public void SetSalary(Salary salary)
```

```
    { this.salary = new Salary(salary); }
```

```
    public Salary GetSalary()
```

```
    { return new Salary(this.salary) ; }
```

```
    public void SetCar(Car car)
```

```
    { this.car = car; }
```

```
    public Car GetCar()
```

```
    { return this.car; }
```

```
    public override string ToString()
```

```
    { return this.name+ " has car:" + this.car + " his salary: " + this.salary; }
```

```
}
```

דוגמה – מחלקת עובד Worker

המשכורת Salary שייכת לעובד, ואין לה משמעות מחוץ לעצם. לכן, במחלקת Worker נשמור עותק שלה

הרכב Car שייך לקצין הרכב של החברה. כלומר, יש לו משמעות מחוץ לעצם. לכן, במחלקת Worker נשמור הפנייה נוספת

int month: 11
int year: 2022
double payment: 9800.7

string: "name"
Salary: salary
Car: car

model: "ford"
licence: 12345
isOk: true

car



דוגמה – מחלקת תלמיד Pupil

- במחלקת החינוך במחוז, תלמיד מאופיין ע"י שמו, כתובתו הפרטית וכתובת בית הספר
- כתובתו הפרטית של התלמיד, משוייכת ישירות לתלמיד
- כתובת בית הספר משוייכת לכלל התלמידים הלומדים באותו בית הספר
- תלמיד שעובר דירה, מעודכנת כתובתו החדשה
- תלמיד שעובר ללמוד בבי"ס אחר, מעודכנת כתובתו של בית הספר החדש
- כתובת מאופיינת בשם הרחוב, מספר הבית והעיר
- למרות שמדובר בעצמים מאותה מחלקה, Address, התפקיד השונה שלהם מחייב התייחסות שונה

```
class Address
{
    private string street;
    private int houseNumber;
    private string city;

    public Address(string street,int houseNumber,string city)
    {
        SetStreet(street);
        SetHouseNumber(houseNumber);
        SetCity(city);
    }
    public Address(Address other)
    {
        SetStreet(other.street);
        SetHouseNumber(other.houseNumber);
        SetCity(other.city);
    }
    // פעולות קובעות מאחזרות ועוד פעולות לפי הצורך
    ...
    public override string ToString()
    {
        return houseNumber + " " +
            this.street + "st. " + this.city;
    }
}
```

דוגמה - תלמיד

מחלקת Address

דוגמה – מחלקת תלמיד Pupil

```
class Pupil
```

```
{
```

```
    private string name;
```

```
    private Address homeAddress;
```

```
    private Address schoolAddress;
```

את הכתובת הפרטית נשמור
כהעתק

את כתובת בית הספר נשמור
כהפניה נוספת

```
    public Pupil(string name, Address homeAddress, Address schoolAddress)
```

```
    {
```

```
        this.name = name;
```

```
        SetHomeAddress(homeAddress);
```

```
        SetSchoolAddress(schoolAddress);
```

```
    }
```

```
    public void SetHomeAddress(Address homeAddress){ this.homeAddress = new Address(homeAddress); }
```

```
    public void SetSchoolAddress(Address schoolAddress){ this.schoolAddress = schoolAddress; }
```

```
    public Address GetHomeAddress(){ return new Address(homeAddress); }
```

```
    public Address GetSchoolAddress(){ return schoolAddress; }
```

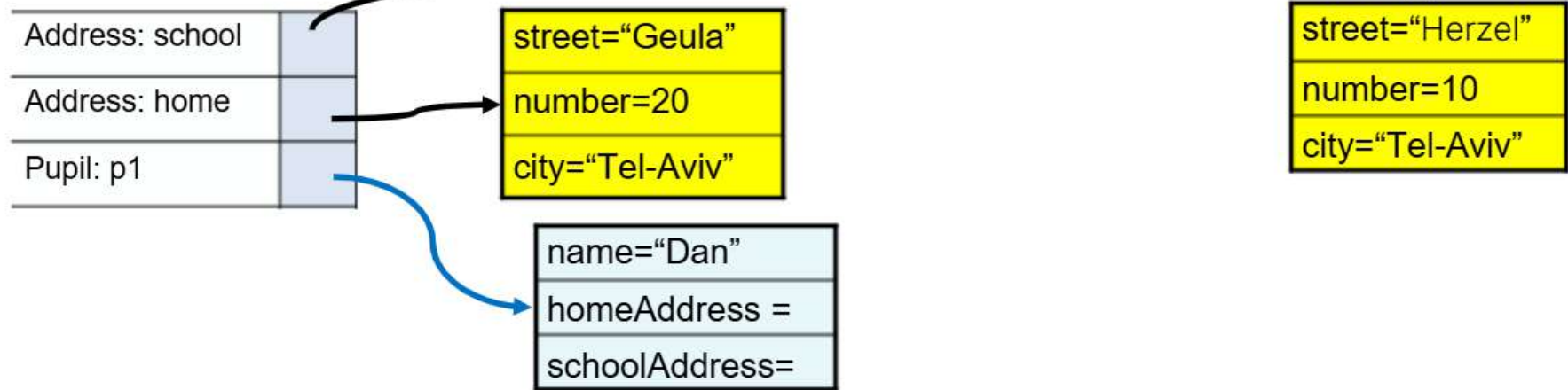
```
    public override string ToString()
```

```
    {
```

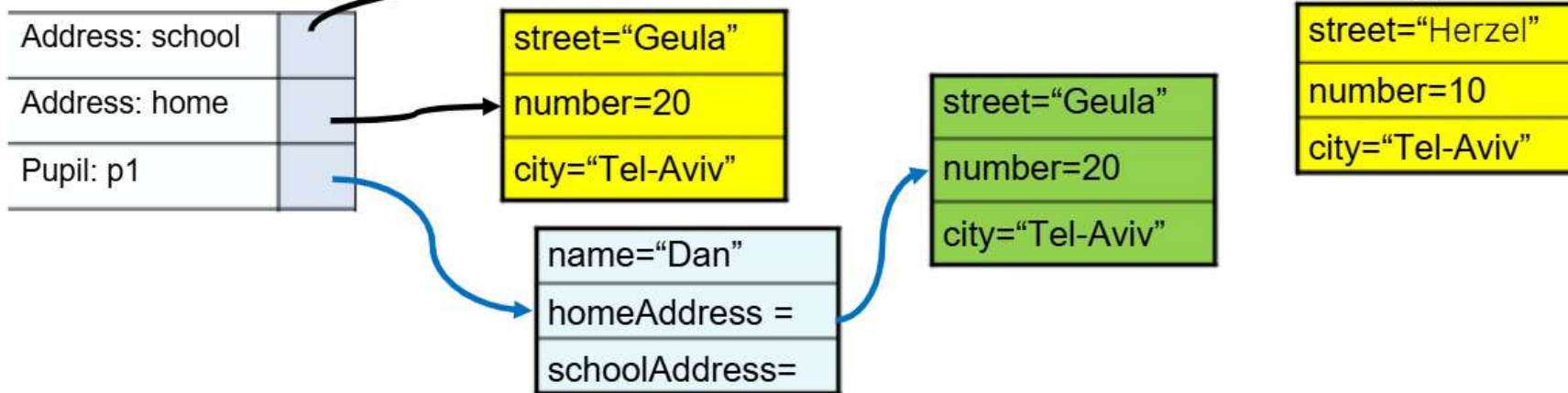
```
        return name + " details:\n" + "Home address: " + homeAddress.ToString() + "\n"
            + "School address: " + schoolAddress.ToString();
```

```
    }
```

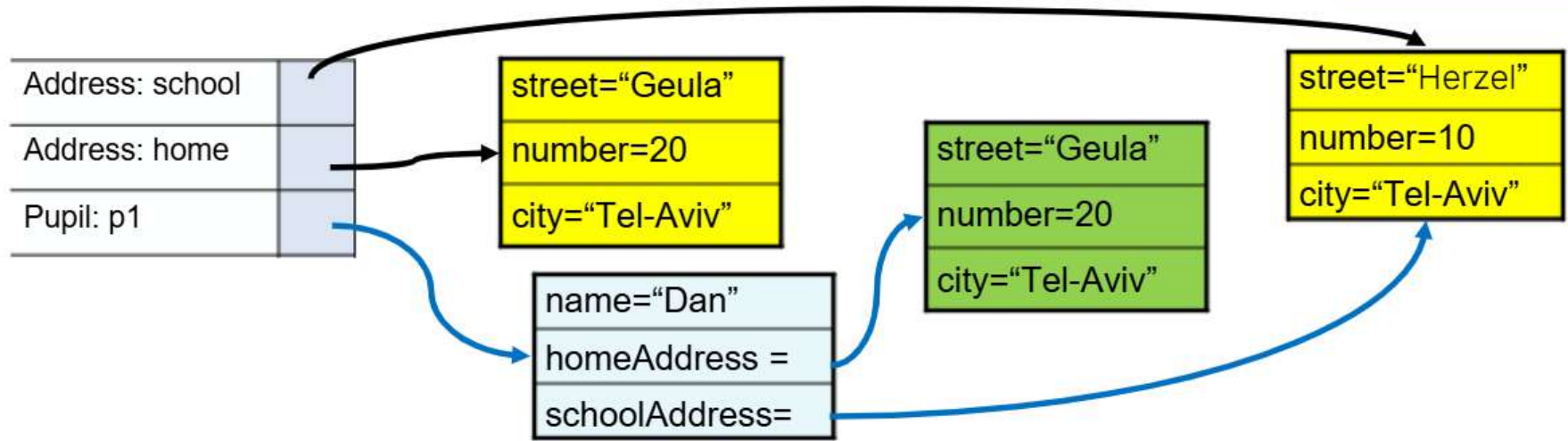
```
}
```



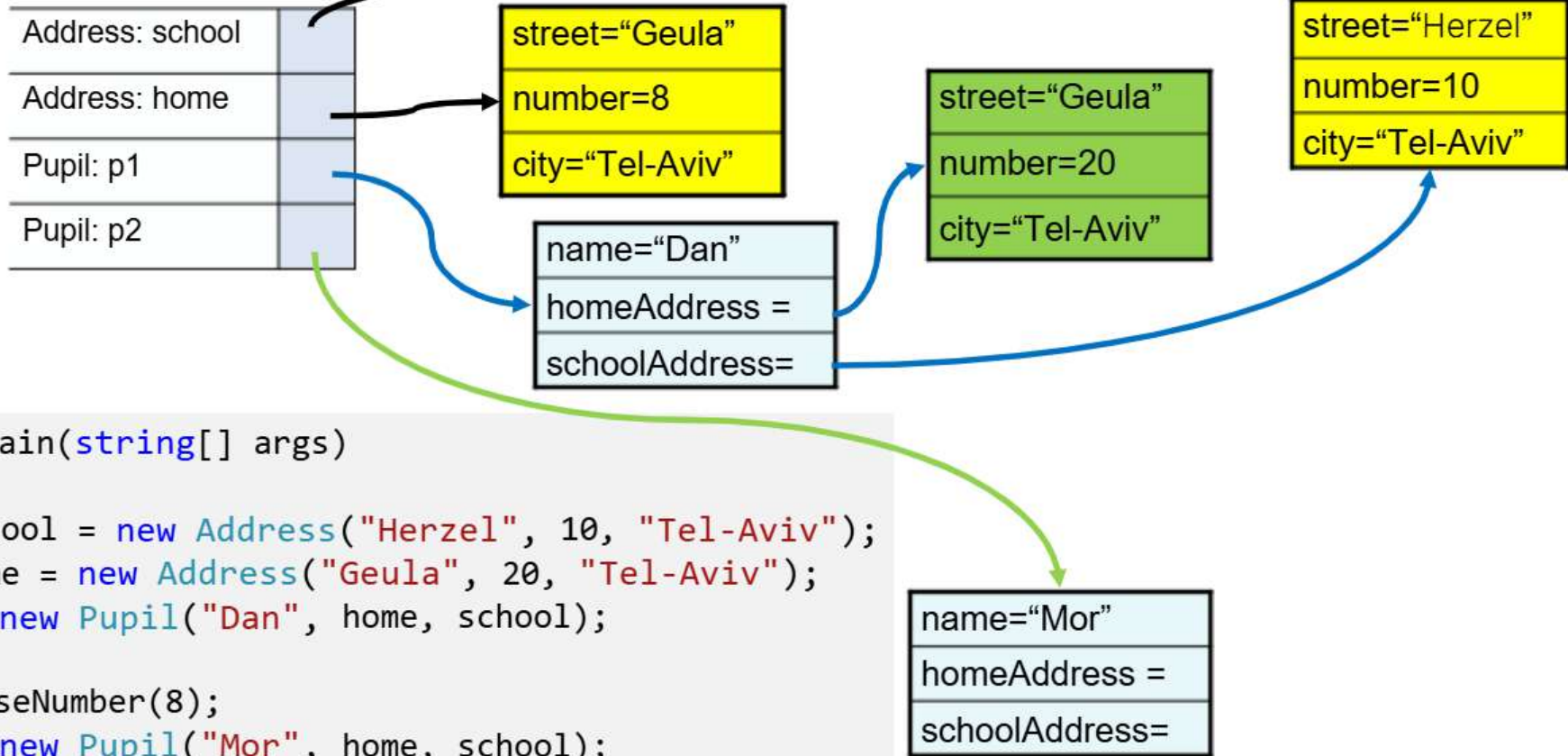
```
static void Main(string[] args)
{
    Address school = new Address("Herzel", 10, "Tel-Aviv");
    Address home = new Address("Geula", 20, "Tel-Aviv");
    Pupil p1 = new Pupil("Dan", home, school);
}
```



```
static void Main(string[] args)
{
    Address school = new Address("Herzel", 10, "Tel-Aviv");
    Address home = new Address("Geula", 20, "Tel-Aviv");
    Pupil p1 = new Pupil("Dan", home, school);
}
```



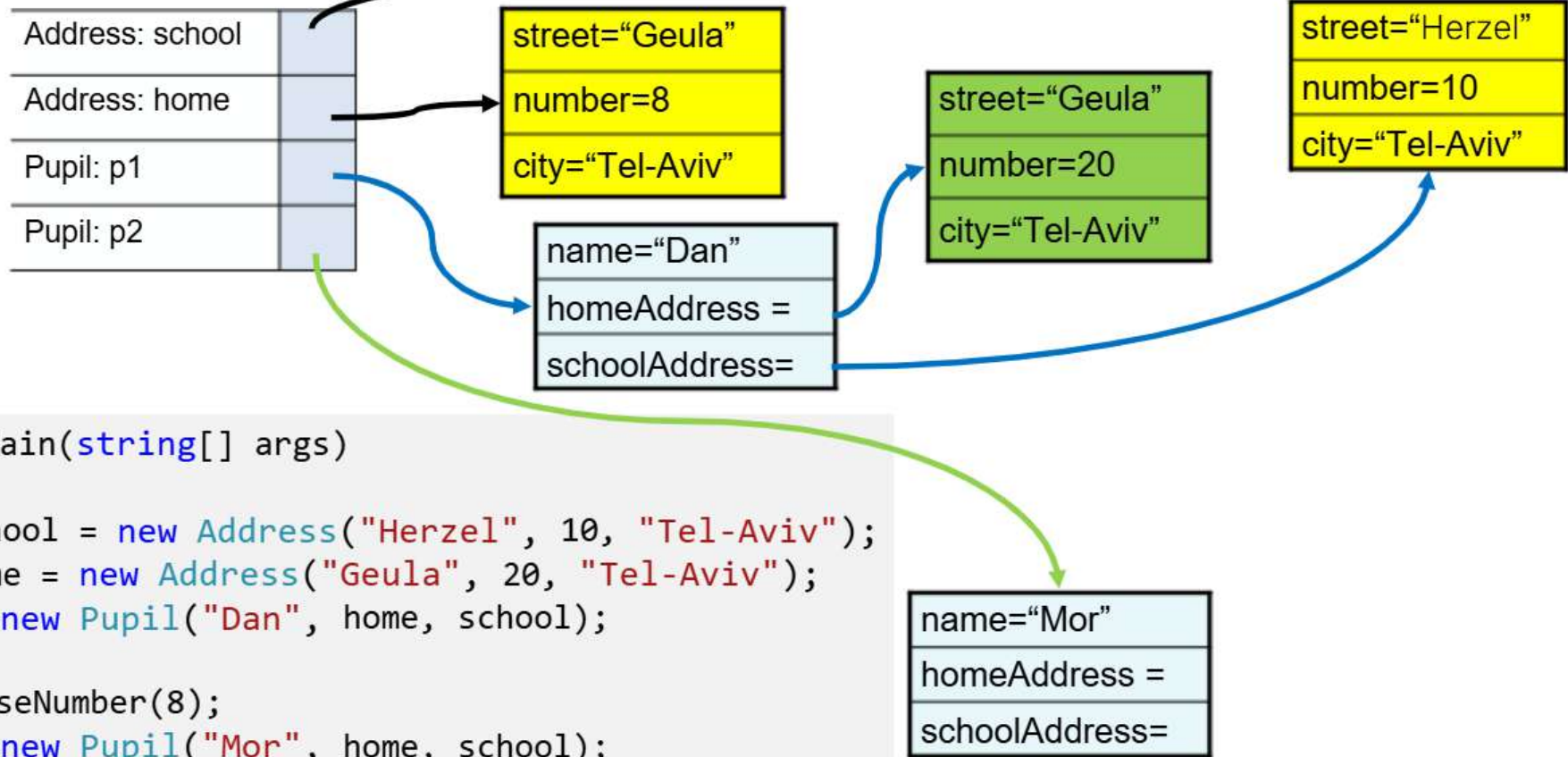
```
static void Main(string[] args)
{
    Address school = new Address("Herzel", 10, "Tel-Aviv");
    Address home = new Address("Geula", 20, "Tel-Aviv");
    Pupil p1 = new Pupil("Dan", home, school);
}
```



```
static void Main(string[] args)
{
    Address school = new Address("Herzel", 10, "Tel-Aviv");
    Address home = new Address("Geula", 20, "Tel-Aviv");
    Pupil p1 = new Pupil("Dan", home, school);

    home.SetHouseNumber(8);
    Pupil p2 = new Pupil("Mor", home, school);

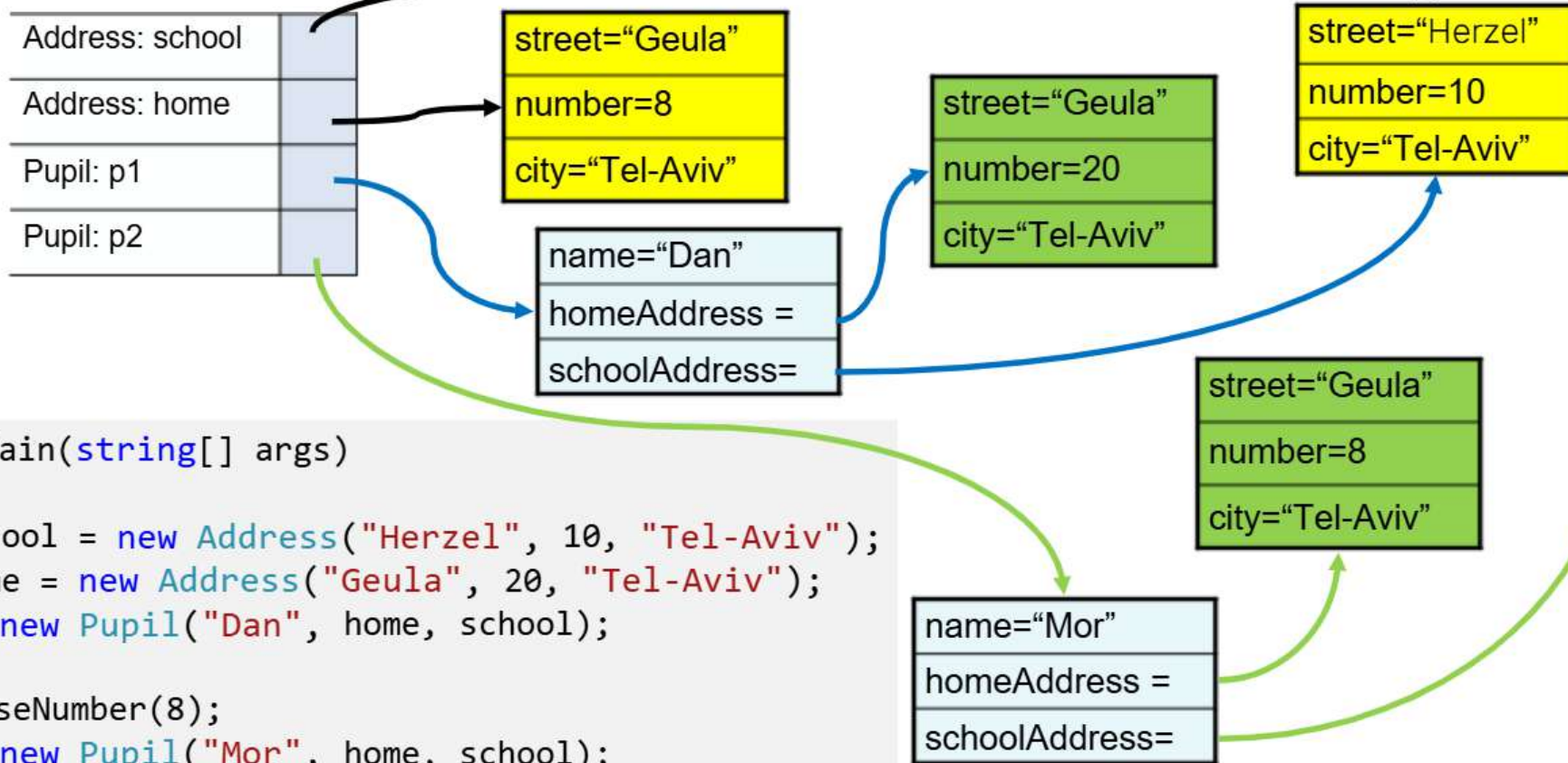
}
```



```
static void Main(string[] args)
{
    Address school = new Address("Herzel", 10, "Tel-Aviv");
    Address home = new Address("Geula", 20, "Tel-Aviv");
    Pupil p1 = new Pupil("Dan", home, school);

    home.SetHouseNumber(8);
    Pupil p2 = new Pupil("Mor", home, school);

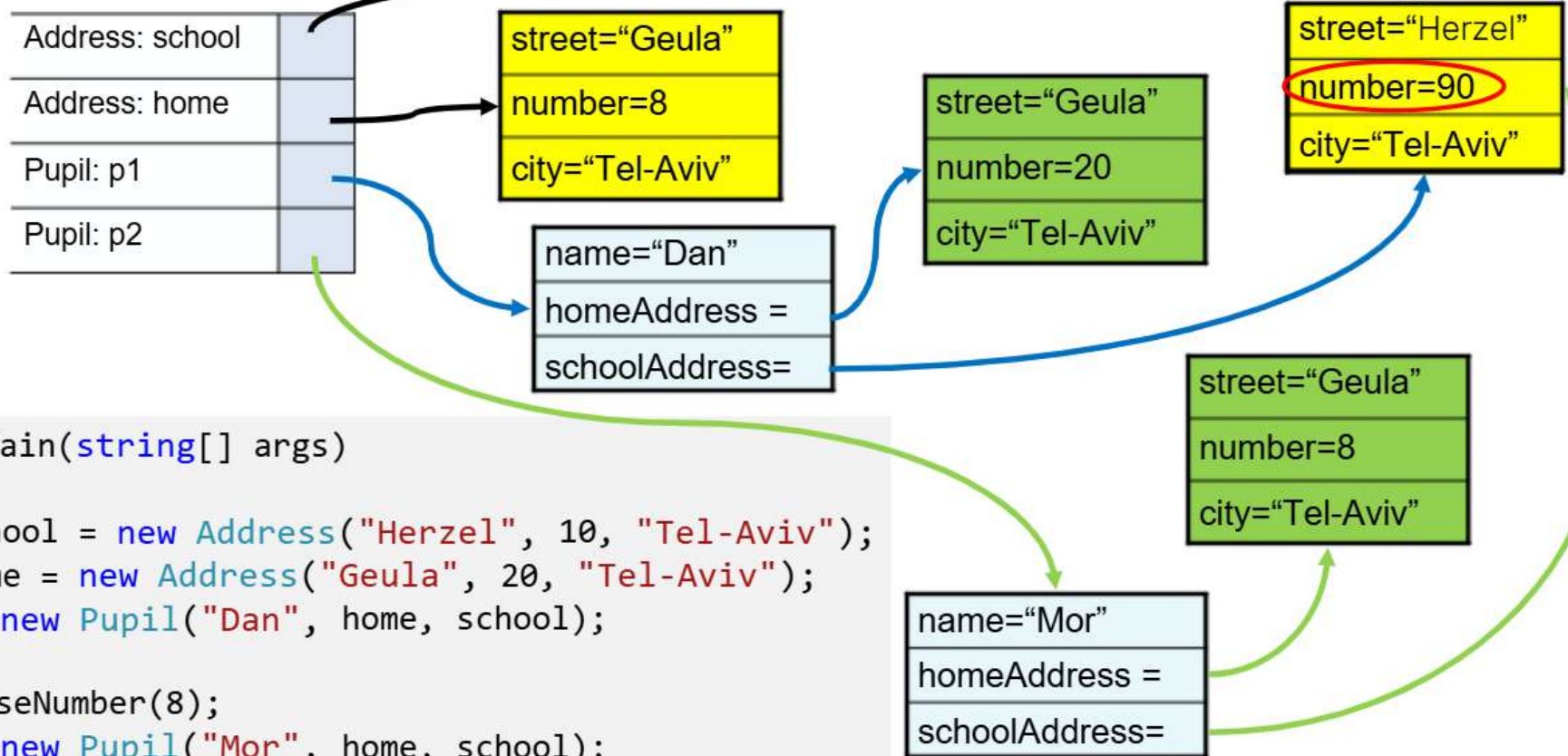
}
```



```
static void Main(string[] args)
{
    Address school = new Address("Herzel", 10, "Tel-Aviv");
    Address home = new Address("Geula", 20, "Tel-Aviv");
    Pupil p1 = new Pupil("Dan", home, school);

    home.SetHouseNumber(8);
    Pupil p2 = new Pupil("Mor", home, school);

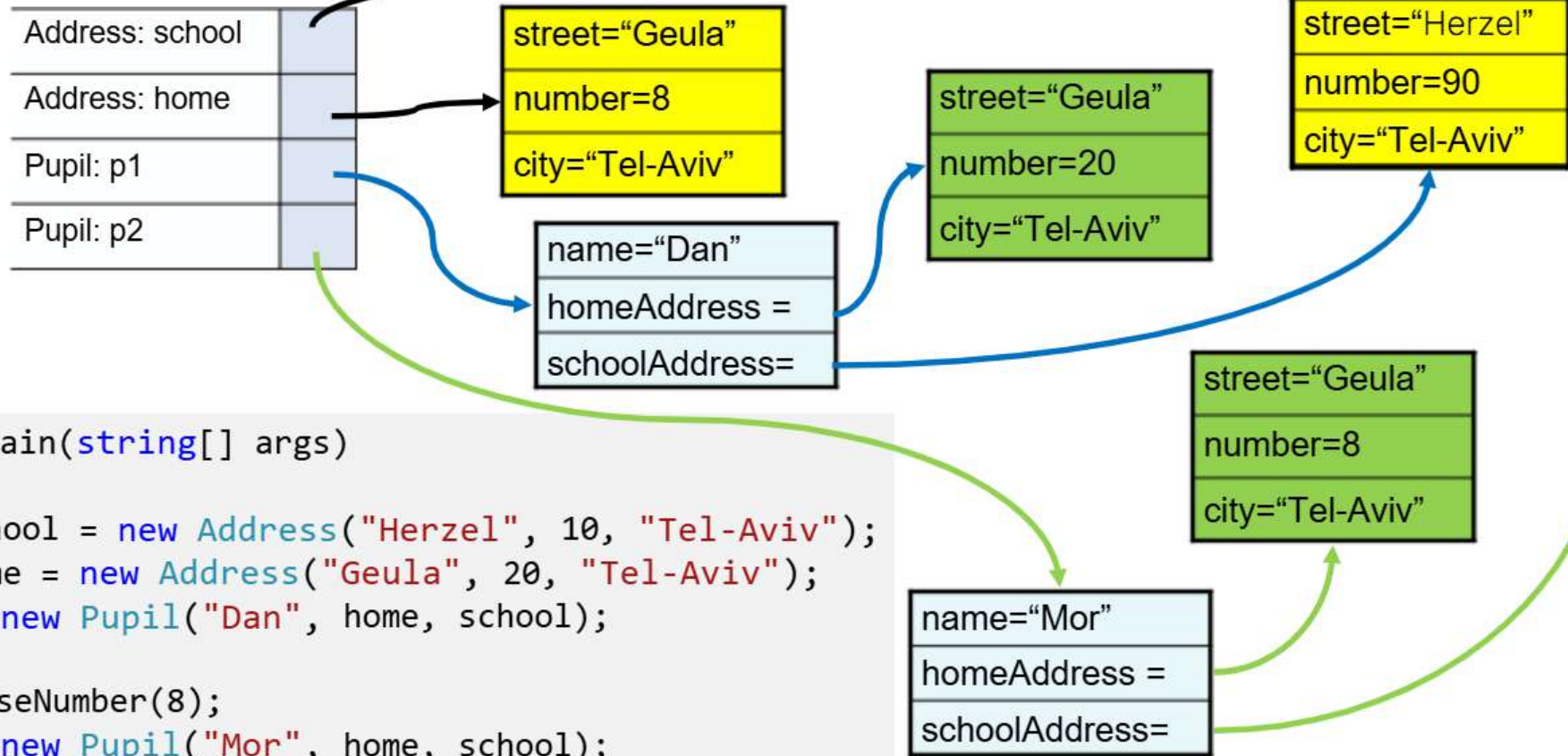
}
```



```
static void Main(string[] args)
{
    Address school = new Address("Herzel", 10, "Tel-Aviv");
    Address home = new Address("Geula", 20, "Tel-Aviv");
    Pupil p1 = new Pupil("Dan", home, school);

    home.SetHouseNumber(8);
    Pupil p2 = new Pupil("Mor", home, school);

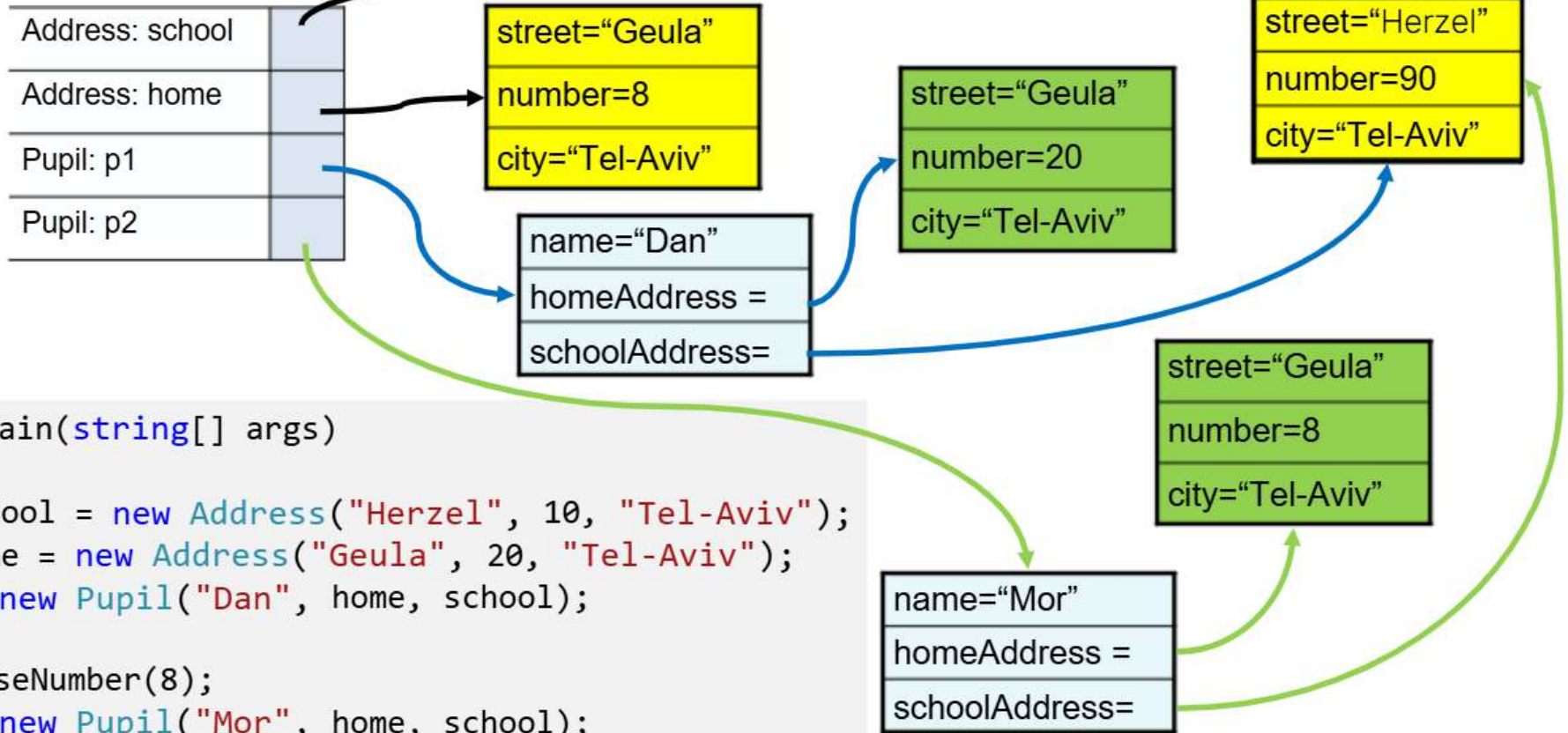
    school.SetHouseNumber(90);
}
```



```
static void Main(string[] args)
{
    Address school = new Address("Herzel", 10, "Tel-Aviv");
    Address home = new Address("Geula", 20, "Tel-Aviv");
    Pupil p1 = new Pupil("Dan", home, school);

    home.SetHouseNumber(8);
    Pupil p2 = new Pupil("Mor", home, school);

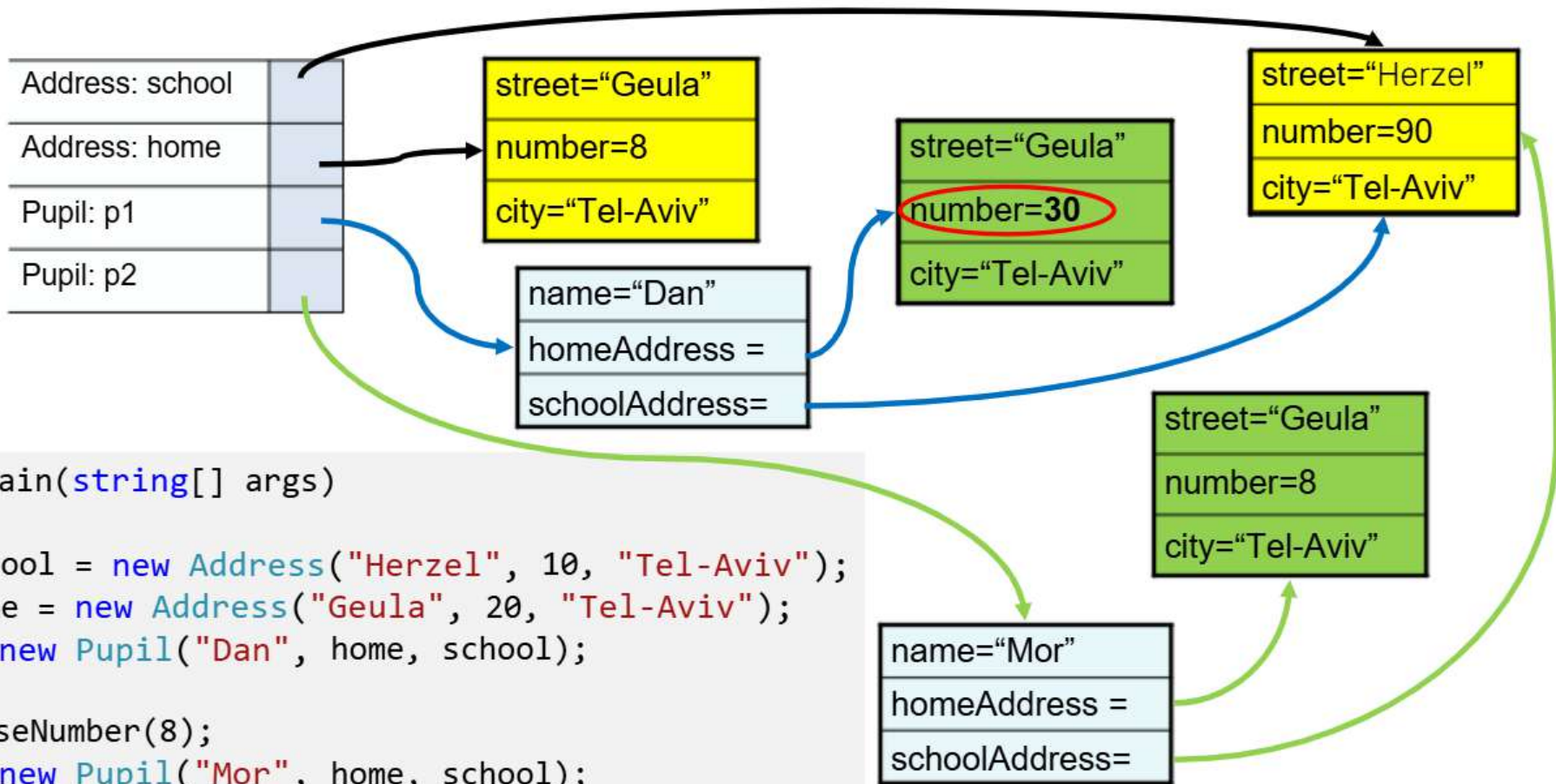
    school.SetHouseNumber(90);
    p1.GetHomeAddress()
}
```



```
static void Main(string[] args)
{
    Address school = new Address("Herzel", 10, "Tel-Aviv");
    Address home = new Address("Geula", 20, "Tel-Aviv");
    Pupil p1 = new Pupil("Dan", home, school);

    home.SetHouseNumber(8);
    Pupil p2 = new Pupil("Mor", home, school);

    school.SetHouseNumber(90);
    p1.GetHomeAddress().SetHouseNumber(30);
}
```



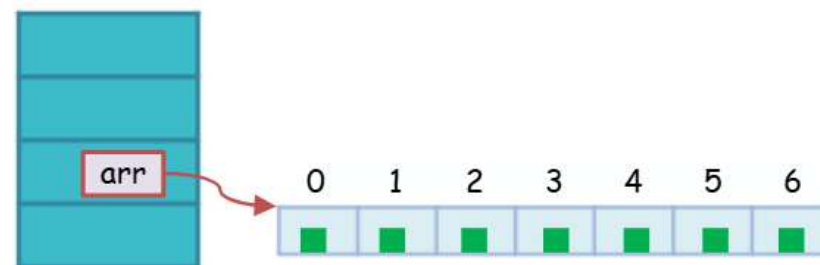
```
static void Main(string[] args)
{
    Address school = new Address("Herzel", 10, "Tel-Aviv");
    Address home = new Address("Geula", 20, "Tel-Aviv");
    Pupil p1 = new Pupil("Dan", home, school);

    home.SetHouseNumber(8);
    Pupil p2 = new Pupil("Mor", home, school);

    school.SetHouseNumber(90);
    p1.GetHomeAddress().SetHouseNumber(30);
}
```

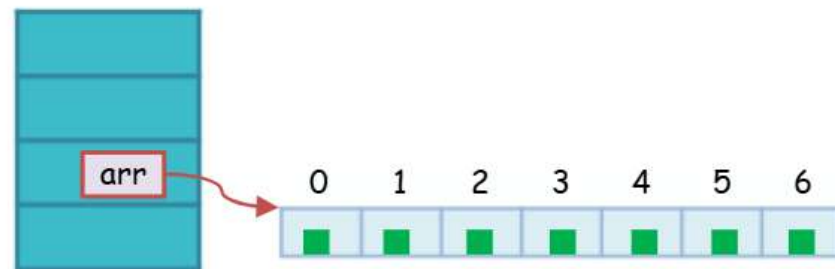
הכלת עצמים

מחלקה המכילה מערך



הכלת עצמים

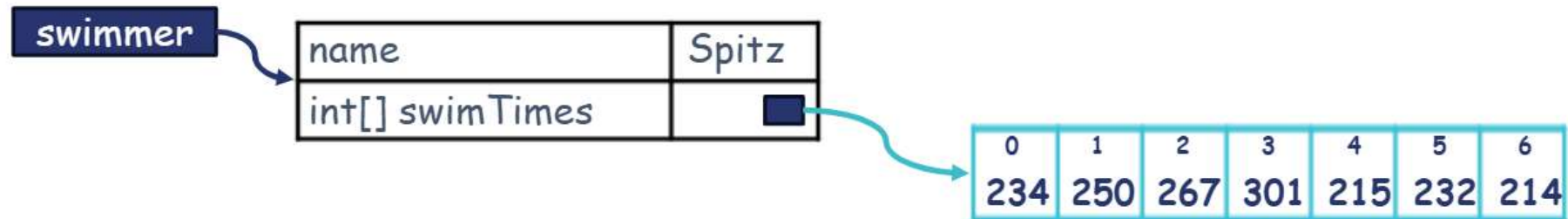
מחלקה המכילה מערך



מערך כתכונה במחלקה

○ מערך הוא עצם מהמחלקה Array

- כאשר אחת מתכונות העצם הוא מערך, העצם מכיל הפניה למערך ולא את המערך עצמו



דוגמה – מחלקת שחיין Swimmer

○ למחלקה שחיין שני מאפיינים – שם השחיין ורשימת תוצאות התחרויות בהן השתתף, בשניות

○ רשימת התוצאות שמורה במערך מטיפוס שלם:

```
class Swimmer
{
    private string name;
    private int[] swimTimes;

    ...
}
```

הפניה למערך מספוס שלם.
ערך ברירת המחדל לאתחול של המשתנה
swimTimes הוא null. אין עדיין מערך.

○ במחלקה נממש את הפעולות - מה התוצאה הטובה ביותר של השחיין, מה התוצאה הממוצעת שלו, וכמה פעמים השתפר במשחה יחסית למשחה הקודם

דוגמה – מחלקת שחיין Swimmer

פעולות במחלקה

```
class Swimmer
{
    private string name;
    private int[] swimTimes;

    ...

    // הפעולה מחזירה את התוצאה הטובה ביותר של השחיין
    public int GetBestTime()
    {
        int bestTime = swimTimes[0];

        foreach (int time in swimTimes)
        {
            if (time < bestTime)
                bestTime = time;
        }
        return bestTime;
    }
}
```

כיוון שהמערך הוא תכונה במחלקה,
אין צורך בפרמטר

הפעולה מניחה שהמערך מלא בזמני
התחרויות

```
class Swimmer
```

```
{
```

```
    private string name;
```

```
    private int[] swimTimes;
```

```
    ...
```

```
    // הפעולה מחזירה את התוצאה הטובה ביותר של השחיין
```

```
    public int GetBestTime()
```

```
    {
```

```
        int bestTime = swimTimes[0];
```

```
        foreach (int time in swimTimes)
```

```
        {
```

```
            if (time < bestTime)
```

```
                bestTime = time;
```

```
        }
```

```
        return bestTime;
```

```
    }
```

```
}
```

דוגמה – מחלקת שחיין Swimmer

פעולות במחלקה

כיוון שהמערך הוא תכונה במחלקה,
אין צורך בפרמטר

הפעולה מניחה שהמערך מלא בזמני
התחרויות

לולאת foreach - המשתנה time מקבל בכל איטרציה
(סיבוב) של הלולאה את הערך של האיבר במערך.
המשתנה הוא מטיפוס איברי המערך

```
class Swimmer
```

```
{
```

```
    private string name;
```

```
    private int[] swimTimes;
```

```
    ...
```

```
    public double GetAverageTime()
```

```
    {
```

```
        double sum = 0;
```

```
        foreach (int time in swimTimes)
```

```
            sum += time;
```

```
        return sum / swimTimes.Length;
```

```
    }
```

```
    public int GetImprovementCount()
```

```
    {
```

```
        int improvementCount = 0;
```

```
        for (int i = 1; i < swimTimes.Length; i++)
```

```
            if (swimTimes[i] < swimTimes[i - 1])
```

```
                improvementCount++;
```

```
        return improvementCount;
```

```
    }
```

```
}
```

דוגמה – מחלקת שחיין Swimmer

פעולות במחלקה

לולאת foreach - המשתנה time מקבל בכל איטרציה (סיבוב) של הלולאה את הערך של האיבר במערך. המשתנה הוא מטיפוס איברי המערך.

הפעולה מחזירה את הזמן הממוצע במערך התוצאות

הפעולה מחזירה את מספר הפעמים שהשחיין שיפר את התוצאה יחסית לתחרות הקודמת

```
class Swimmer
```

```
{
```

```
    private string name;  
    private int[] swimTimes;
```

```
    ...
```

```
    public double GetAverageTime()  
    {
```

```
        double sum = 0;  
        foreach (int time in swimTimes)  
            sum += time;  
        return sum / swimTimes.Length;
```

```
    }
```

```
    public int GetImprovementCount()  
    {
```

```
        int improvementCount = 0;  
        for (int i = 1; i < swimTimes.Length; i++)  
            if (swimTimes[i] < swimTimes[i - 1])  
                improvementCount++;  
        return improvementCount;
```

```
    }
```

```
}
```

דוגמה – מחלקת שחיין Swimmer

פעולות במחלקה

לולאת foreach - המשתנה time מקבל בכל איטרציה (סיבוב) של הלולאה את הערך של האיבר במערך. המשתנה הוא מטיפוס איברי המערך.

הפעולה מחזירה את הזמן הממוצע במערך התוצאות

בלולאה זו, המשתנה i מקבל בכל איטרציה ערך של אינדקס במערך.

הפעולה מחזירה את מספר הפעמים שהשחיין שיפר את התוצאה יחסית לתחרות הקודמת

```
class Swimmer
```

```
{
```

```
    private string name;
```

```
    private int[] swimTimes;
```

```
    ...
```

```
    public double GetAverageTime()
```

```
    {
```

```
        double sum = 0;
```

```
        foreach (int time in swimTimes)
```

```
            sum += time;
```

```
        return sum / swimTimes.Length;
```

```
    }
```

```
    public int GetImprovementCount()
```

```
    {
```

```
        int improvementCount = 0;
```

```
        for (int i = 1; i < swimTimes.Length; i++)
```

```
            if (swimTimes[i] < swimTimes[i - 1])
```

```
                improvementCount++;
```

```
        return improvementCount;
```

```
    }
```

```
}
```

דוגמה – מחלקת שחיין Swimmer

פעולות במחלקה

לולאת foreach - המשתנה time מקבל בכל איטרציה (סיבוב) של הלולאה את הערך של האיבר במערך. המשתנה הוא מטיפוס איברי המערך.

הפעולה מחזירה את הזמן הממוצע במערך התוצאות

בלולאה זו, המשתנה i מקבל בכל איטרציה ערך של אינדקס במערך.

הפעולה מחזירה את מספר הפעמים שהשחיין שיפר את התוצאה יחסית לתחרות הקודמת

גם פעולות אלו מניחות שהמערך מלא בזמני התחרויות

```
class Swimmer
{
    private string name;
    private int[] swimTimes;
    ...
    public Swimmer(string name, int[] swimTimes)
    {
        this.name = name;
        this.swimTimes = swimTimes;
    }
}
```

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

1. מערך המתקבל כפרמטר

○ אם במחלקה לא תוגדר פעולה
בונה – ערכו ההתחלתי של משתנה
ההפניה swimTimes יהיה null

○ אם הפעולה הבונה תקבל את
המערך כפרמטר – תהיה גישה
ישירה למערך מחוץ למחלקה, ובעל
הגישה יוכל לשנות את תוכן המערך
כרצונו

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

1. מערך המתקבל כפרמטר

```
class Swimmer
{
    private string name;
    private int[] swimTimes;
    ...
    public Swimmer(string name, int[] swimTimes)
    {
        this.name = name;
        this.swimTimes = swimTimes;
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        int[] a = {356, 365, 345, 347, 330};
        Swimmer s = new Swimmer("Rona", a);
    }
}
```

Swimmer s	■
int[] a	■

0	1	2	3	4
356	365	345	347	330

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

1. מערך המתקבל כפרמטר

```
class Swimmer
{
    private string name;
    private int[] swimTimes;
    ...
    public Swimmer(string name, int[] swimTimes)
    {
        this.name = name;
        this.swimTimes = swimTimes;
    }
}
```

string name	Rona
int[] swimTimes	■

```
class Program
{
    static void Main(string[] args)
    {
        int[] a = {356, 365, 345, 347, 330};
        Swimmer s = new Swimmer("Rona", a);
    }
}
```

Swimmer s	■
int[] a	■

0	1	2	3	4
356	365	345	347	330

```

class Swimmer
{
    private string name;
    private int[] swimTimes;
    ...
    public Swimmer(string name, int[] swimTimes)
    {
        this.name = name;
        this.swimTimes = swimTimes;
    }
}

```

```

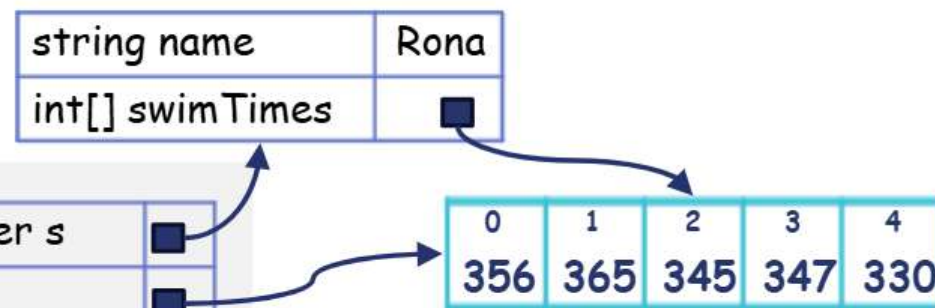
class Program
{
    static void Main(string[] args)
    {
        int[] a = {356, 365, 345, 347, 330};
        Swimmer s = new Swimmer("Rona", a);
    }
}

```

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

1. מערך המתקבל כפרמטר



דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

1. מערך המתקבל כפרמטר

```
class Swimmer
{
    private string name;
    private int[] swimTimes;
    ...
    public Swimmer(string name, int[] swimTimes)
    {
        this.name = name;
        this.swimTimes = swimTimes;
    }
}
```

```
class Program
static void Main(string[] args)
{
    int[] a = {356, 365, 345, 347, 330};
    Swimmer s = new Swimmer("Rona", a);
    a[1] = -90;
}
```

string name	Rona
int[] swimTimes	■

Swimmer s	■
int[] a	■

0	1	2	3	4
356	-90	345	347	330

```
class Swimmer
```

```
{
```

```
    private string name;
```

```
    private int[] swimTimes;
```

```
    ...
```

```
    public Swimmer(string name, int[] swimTimes)
```

```
    {
```

```
        this.name = name;
```

```
        this.swimTimes = swimTimes;
```

```
    }
```

```
}
```

```
class Program
```

```
    static void Main(string[] args)
```

```
    {
```

```
        int[] a = {356, 365, 345, 347, 330};
```

```
        Swimmer s = new Swimmer("Rona", a);
```

```
        a[1] = -90;
```

```
    }
```

```
}
```

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

1. מערך המתקבל כפרמטר

string name	Rona
int[] swimTimes	

Swimmer s	
int[] a	

0	1	2	3	4
356	-90	345	347	330

במקרים רבים לא נרצה כי תהיה גישה למערך מחוץ למחלקה – כאן הוכנס ערך שלילי לתוצאת תחרות

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

1. מערך המתקבל כפרמטר

```
class Swimmer
{
    private string name;
    private int[] swimTimes;
    ...
    public Swimmer(string name, int[] swimTimes)
    {
        this.name = name;
        this.swimTimes = new int[swimTimes.Length];
        for (int i = 1; i < swimTimes.Length; i++)
            this.swimTimes[i] = swimTimes[i];
    }
}
```



0	1	2	3	4
356	365	345	347	330

}

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

1. מערך המתקבל כפרמטר

```
class Swimmer
{
    private string name;
    private int[] swimTimes;
    ...
    public Swimmer(string name, int[] swimTimes)
    {
        this.name = name;
        this.swimTimes = new int[swimTimes.Length];
        for (int i = 1; i < swimTimes.Length; i++)
            this.swimTimes[i] = swimTimes[i];
    }
}
```



0	1	2	3	4
356	365	345	347	330



0	1	2	3	4
356	365	345	347	330

```
class Swimmer
```

```
{
```

```
    private string name;
```

```
    private int[] swimTimes;
```

```
    ...
```

```
    public Swimmer(string name, int[] swimTimes)
```

```
    {
```

```
        this.name = name;
```

```
        this.swimTimes = new int[swimTimes.Length];
```

```
        for (int i = 1; i < swimTimes.Length; i++)
```

```
            this.swimTimes[i] = swimTimes[i];
```

```
    }
```

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

1. מערך המתקבל כפרמטר

0	1	2	3	4
356	365	345	347	330

0	1	2	3	4
356	365	345	347	330

המערך המתקבל כפרמטר מועתק
למערך חדש

```
}
```

```
class Swimmer
```

```
{
```

```
    private string name;
```

```
    private int[] swimTimes;
```

```
    ...
```

```
    public Swimmer(string name, int[] swimTimes)
```

```
    {
```

```
        this.name = name;
```

```
        this.swimTimes = new int[swimTimes.Length];
```

```
        for (int i = 1; i < swimTimes.Length; i++)
```

```
            this.swimTimes[i] = swimTimes[i];
```

```
    }
```

```
public int[] getSwimTimes() { return swimTimes; }
```

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

1. מערך המתקבל כפרמטר



0	1	2	3	4
356	365	345	347	330

המערך המתקבל כפרמטר מועתק
למערך חדש

לא נספק גישה למערך מחוץ
למחלקה

```
}
```

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

1. מערך המתקבל כפרמטר

0	1	2	3	4
356	365	345	347	330

```
class Swimmer
{
    private string name;
    private int[] swimTimes;
    ...
    public Swimmer(string name, int[] swimTimes)
    {
        this.name = name;
        this.swimTimes = new int[swimTimes.Length];
        for (int i = 1; i < swimTimes.Length; i++)
            this.swimTimes[i] = swimTimes[i];
    }

    public int[] getSwimTimes() { return swimTimes; }

    public int getTime(int swim)
    {
        if (swim >= 1 && swim <= swimTimes.Length)
            return swimTimes[swim + 1];
        return -999;
    }
}
```

המערך המתקבל כפרמטר מועתק
למערך חדש

לא נספק גישה למערך מחוץ
למחלקה

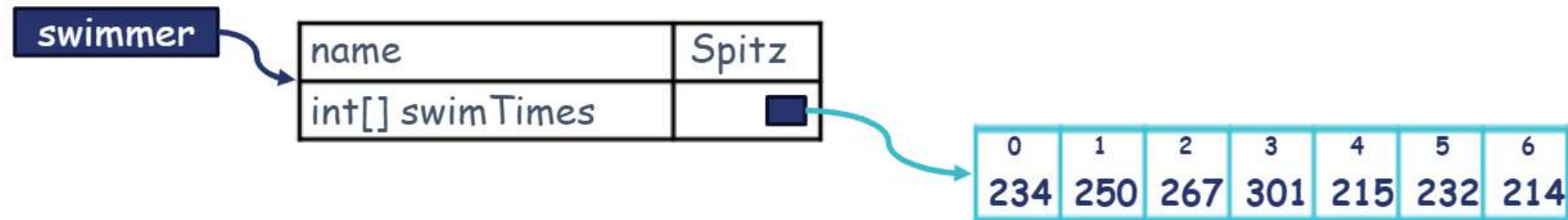
אם צריך, אחזור אחת התוצאות במערך,
על פי מספר התחרות, אם קיימת

פעולה בונה – אפשרות אחת – מערך המתקבל כפרמטר

- בתכונה שהיא מערך, ערך המשתנה הוא הפניה למערך ולא המערך עצמו
- אם איננו רוצים שתהיה גישה למערך מחוץ למחלקה, בפעולה הבונה, המקבלת מערך כפרמטר, במקום לבצע השמה של ההפניה שהתקבלה ישירות לתכונת המחלקה, ניצור עותק של המערך ואותו נשמור כתכונה במחלקה
- המחלקה תנהל, בצורה פנימית, את הגישה והטיפול באיברי המערך
- לא ניצור פעולה מאחזרת המחזירה את (ההפניה ל)מערך, וזאת כדי לא לאפשר את השינוי של המערך מחוץ למחלקה
- ניתן לכתוב פעולה "מאחזרת" שתחזיר העתק של המערך, וכך לא יוכלו לשנותו מחוץ למחלקה

מערך כתכונה במחלקה – אוסף נתונים נצבר

- בתחילת עונת התחרויות, לא ידוע מה יהיו זמני המשחים השונים
- הרבה פעמים, עם בניית העצם לא ידועים נתוני האוסף, ואלו יתווספו במהלך הזמן





```
class Swimmer
{
    private string name;
    private int[] swimTimes;

    ...

    public Swimmer(string name, int num)
    {
        this.name = name;
        this.swimTimes = new int[num];
    }
}
```

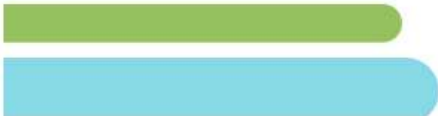
דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

2. אוסף נתונים נצבר

○ המערך יבנה בפעולה הבונה. ערכיו יהיו ערכי ברירת המחדל ע"פ טיפוס המערך – 0,0.0,false,null

○ אם לכל שחיין מספר תחרויות שונה, מספר התחרויות יתקבל כפרמטר



```

class Swimmer
{
    private string name;
    private int[] swimTimes;
    public static const int NUM = 6;

    ...

    public Swimmer(string name)
    {
        this.name = name;
        this.swimTimes = new int[NUM];
    }
}

```

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

2. אוסף נתונים נצבר

○ המערך יבנה בפעולה הבונה. ערכיו יהיו ערכי ברירת המחדל ע"פ טיפוס המערך – 0,0.0,false,null

○ אם מספר התחרויות **קבוע** לכל השחיינים, נוסיף למחלקה קבוע בגודל הנדרש

```
class Swimmer
```

```
{
```

```
    private string name;
```

```
    private int[] swimTimes;
```

```
    ...
```

```
    public bool Add(int time)
```

```
    {
```

```
        for (int i = 0; i < swimTimes.Length; i++)
```

```
            if (swimTimes[i] == 0)
```

```
            {
```

```
                swimTimes[i] = time;
```

```
                return true;
```

```
            }
```

```
        return false;
```

```
    }
```

```
    public int GetBestTime(){...}
```

```
    public double GetAverageTime(){...}
```

```
    public int GetImprovementCount(){...}
```

```
}
```

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

2. אוסף נתונים נצבר

0 בפעולה Add נוסף תוצאה למערך. כיוון שאיננו יודעים כמה תוצאות יש במערך, נחפש תא שיש בו ערך 0.

0 פיתרון זה אינו נוח – בפעולות המחלקה הנחנו כי המערך מלא, כעת נצטרך לבדוק כל פעם מחדש מהם ערכי האמת שיש בו.

```
class Swimmer
```

```
{
```

```
    private string name;
```

```
    private int[] swimTimes;
```

```
    ...
```

```
    public bool Add(int time)
```

```
    {
```

```
        for (int i = 0; i < swimTimes.Length; i++)
```

```
            if (swimTimes[i] == 0)
```

```
            {
```

```
                swimTimes[i] = time;
```

```
                return true;
```

```
            }
```

```
        return false;
```

```
    }
```

```
    public int GetBestTime(){...}
```

```
    public double GetAverageTime(){...}
```

```
    public int GetImprovementCount(){...}
```

```
}
```

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה

2. אוסף נתונים נצבר

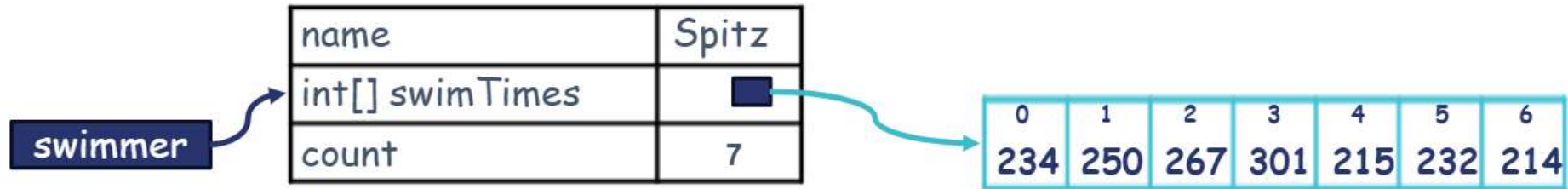
○ בפעולה Add נוסף תוצאה למערך. כיוון שאיננו יודעים כמה תוצאות יש במערך, נחפש תא שיש בו ערך 0.

○ פיתרון זה אינו נוח – בפעולות המחלקה הנחנו כי המערך מלא, כעת נצטרך לבדוק כל פעם מחדש מהם ערכי האמת שיש בו.

○ בעיה גדולה יותר – מה קורה כאשר הערך 0 הוא חוקי? מה קורה כאשר כל ערך שלם הוא חוקי, לרבות מספרים שליליים? במקרה זה, הפיתרון לא רק שאינו נוח – הוא שגוי!

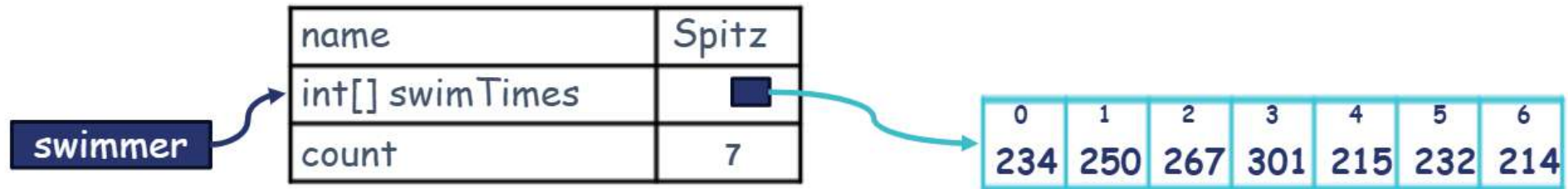
מערך כתכונה במחלקה – אוסף נתונים נצבר

- הפתרון – נחזיק מונה למספר האיברים החוקיים במערך
- שימו לב – אם נקפיד שכל האיברים יהיו בתחילת המערך, המונה הוא גם האינדקס של המקום הבא להוספת איבר:



מערך כתכונה במחלקה – אוסף נתונים נצבר

- הפתרון – נחזיק מונה למספר האיברים החוקיים במערך
- שימו לב – אם נקפיד שכל האיברים יהיו בתחילת המערך, המונה הוא גם האינדקס של המקום הבא להוספת איבר:



- כשהמונה מגיע לגודל המערך, אין מקום להכנסת איברים נוספים



```
class Swimmer
```

```
{  
    private string name;  
    private int[] swimTimes;  
    private int count;  
    ...  
    public Swimmer(string name, int num)  
    {  
        this.name = name;  
        this.swimTimes = new int[num];  
        this.count = 0;  
    }  
    public bool Add (int time)  
    {  
        if (count < swimTimes.Length)  
        {  
            swimTimes[count] = time;  
            count++;  
            return true;  
        }  
        return false;  
    }  
}
```

דוגמה – מחלקת שחיין Swimmer

הפעולה הבונה ופעולת ההוספה

2. אוסף נתונים נצבר

נוסף תכונה - מונה

בפעולה הבונה, נאפס את המונה.
שימו לב – המונה גם מצביע על המקום הפנוי הבא, כרגע 0

ניתן להוסיף איבר למערך, רק אם אינו מלא.
בהוספה, אין צורך בלולאה, אלא נכניס למיקום הבא.
יש לעדכן את המונה



```
class Swimmer
```

```
{
```

```
    private string name;  
    private int[] swimTimes;  
    private int count;
```

```
    ...
```

```
    // הפעולה מחזירה את התוצאה הטובה ביותר של השחיין
```

```
    public int GetBestTime()
```

```
    {
```

```
        int bestTime = swimTimes[0];  
        foreach (int time in swimTimes)  
            if (time < bestTime)  
                bestTime = time;  
        return bestTime;
```

```
    }
```

```
    public int GetBestTime()
```

```
    {
```

```
        int bestTime = swimTimes[0];  
        for (int i = 1; i < count; i++)  
            if (swimTimes[i] < bestTime)  
                bestTime = swimTimes[i];  
        return bestTime;
```

```
    }
```

```
}
```

דוגמה – מחלקת שחיין Swimmer

פעולות במחלקה

2. אוסף נתונים נצבר

לא ניתן להשתמש בלולאת
foreach, כיוון שזו עוברת על כל
איברי המערך, וכאן רק בתחילת
המערך יש ערכים חוקיים

הלולאה – רק על האיברים החוקיים

```
}
```

מערך כתכונה במחלקה - מחלקת שחיין Swimmer

- במחלקת שחיין, היה נכון לשמור את זמני התחרויות במערך, בו האיברים נכנסים לפי סדר התחרויות
- ראינו גם, שבמקרה זה, הוספת מונה כתכונה למחלקה הקלה על הכנסת ערכים וצמצמה את הלולאות למעבר רק על ערכים החוקיים
- במחלקת שחיין, גם לא היה צורך למחוק איברים במערך



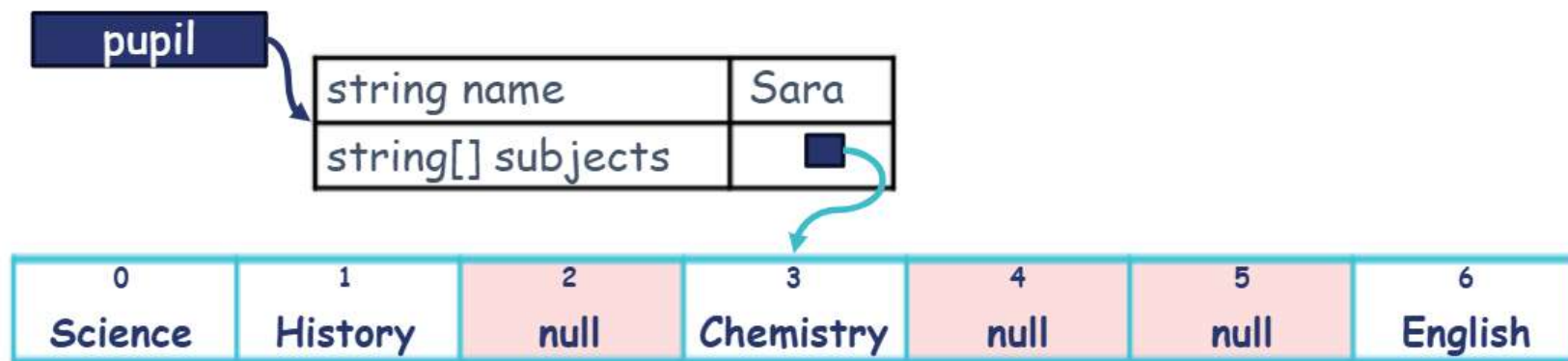
מערך כתכונה במחלקה - מחלקת שחיין Swimmer

- במחלקת שחיין, היה נכון לשמור את זמני התחרויות במערך, בו האיברים נכנסים לפי סדר התחרויות
- ראינו גם, שבמקרה זה, הוספת מונה כתכונה למחלקה הקלה על הכנסת ערכים וצמצמה את הלולאות למעבר רק על ערכים החוקיים
- במחלקת שחיין, גם לא היה צורך למחוק איברים במערך



מערך כתכונה במחלקה - מחלקת תלמיד Pupil

- מחלקת תלמיד מאופיינת על ידי שם התלמיד ורשימת מקצועות הלימוד
- רשימת נושאי הלימוד היא דינמית – בכל סמסטר יכולים להתווסף מקצועות לימוד, ולאחר בחינת הסיום, המקצוע נגרע מרשימת המקצועות
- ההחלטה כיצד לנהל את ההוספה/הוצאה מהמערך, תלויה בצרכי המחלקה





```
class Pupil  
{
```

```
    private string name;  
    private string[] subjects;
```

```
    public Pupil(Sarastring name, 5int maxSubjects)  
    {  
        this.name = name;  
        this.subjects = new string[maxSubjects];  
    }  
  
    ...  
}
```

דוגמה – מחלקת תלמיד Pupil

הפעולה הבונה

1. מערך ללא חשיבות לסדר וללא צורך ברצף

○ אם במחלקה לא תוגדר פעולה בונה – ערכו
ההתחלתי של משתנה ההפניה subjects
יהיה null

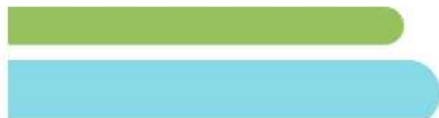
○ בפעולה הבונה נבנה מערך באורך רצוי

○ במערך ערכי ברירת המחדל של string –
null

pupil

string name	Sara
string[] subjects	

0	1	2	3	4
null	null	null	null	null



```

class Pupil
{
    private string name;
    private string[] subjects;
    ...
    public bool RemoveSubject(string subject)
    {
        for (int i = 0; i < subjects.Length; i++)
            if (subjects[i] == subject)
            {
                subjects[i] = null;
                return true;
            }
        return false;
    }
}

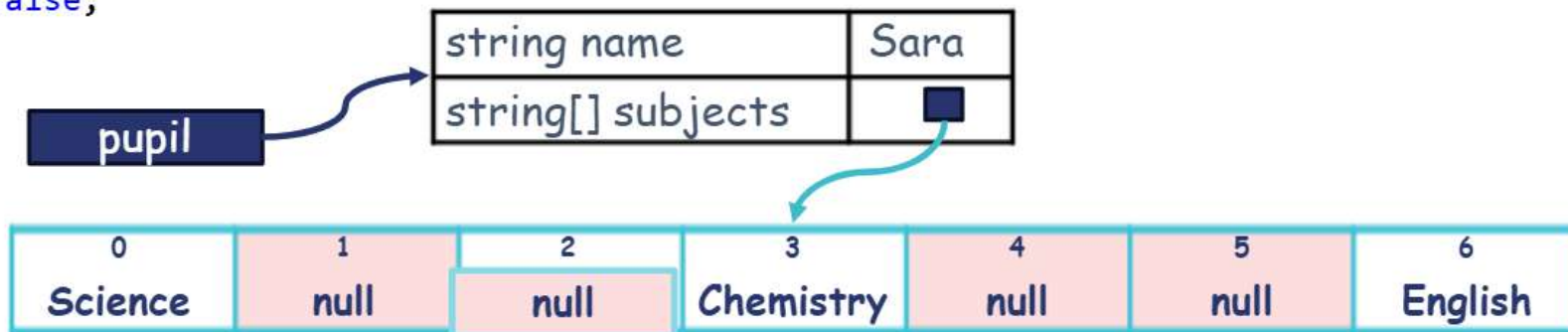
```

דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה

1. מערך ללא חשיבות לסדר וללא צורך ברצף

בהסרת איבר, כיוון שאין צורך שהאיברים יהיו רצופים, ניתן "לרוקן" את התא, בהשמת `null`



```

class Pupil
{
    private string name;
    private string[] subjects;
    ...

    public bool IsSubjectIn1(string subject)
    {
        for (int i = 0; i < subjects.Length; i++)
            if (subjects[i] == subject)
                return true;
        return false;
    }

    public bool IsSubjectIn2(string subject)
    {
        foreach (string sub in subjects)
            if (sub == subject)
                return true;
        return false;
    }
}

```

דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה

1. מערך ללא חשיבות לסדר וללא צורך ברצף

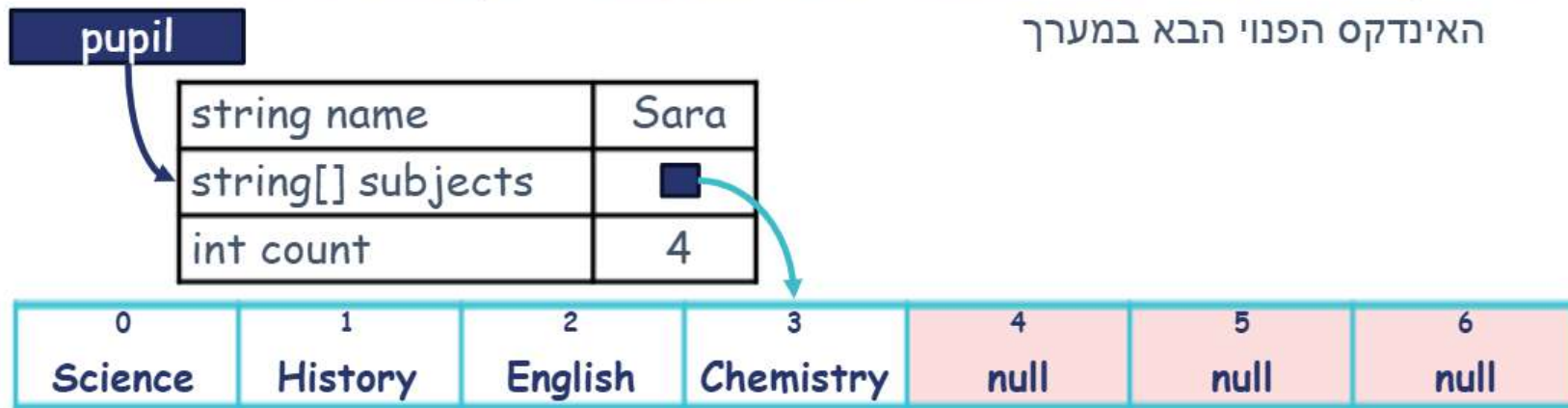
○ לעיתים, אין בעיה עם ערך `null`, כמו בפעולות `IsSubjectIn` המחזירות אמת אם התלמיד לומד את המקצוע שהועבר כפרמטר.

○ כיוון שניתן להשוות מחרוזת ל `null`, אין צורך לבדוק אם בתא המערך יש `null`.

○ כיוון שכך, הפעם נוכל להשתמש בלולאת `foreach`

מעריך כתכונה במחלקה - מחלקת תלמיד Pupil

- לעיתים יש חשיבות לסדר האיברים הכנסת האיברים למעריך, לדוגמה כאשר המקצועות במחלקת תלמיד מסודרים בהתאם למועד בו התחיל התלמיד ללמוד את המקצוע
- במקרה זה, כדאי לשמור משתנה המונה את מספר המקצועות, וערכו הוא גם האינדקס הפנוי הבא במעריך

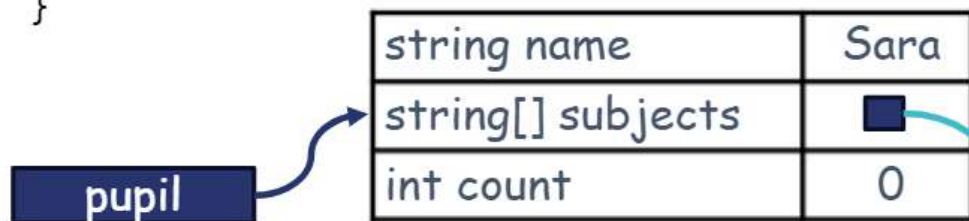


```
class Pupil
{
```

```
    private string name;
    private string[] subjects;
    private int count;
```

```
    public Pupil(Sara, 3
    {
        this.name = name;
        this.subjects = new string[maxSubjects];
        this.count = 0;
    }
    ...
}
```

```
}
```



דוגמה – מחלקת תלמיד Pupil

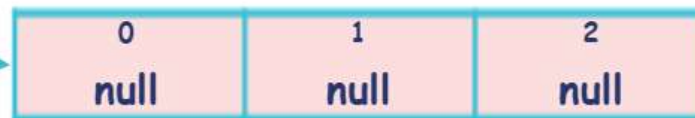
הפעולה הבונה

2. מערך עם חשיבות לסדר

נוסף מונה לתכונות ובפעולה הבונה נקבע את ערכו ל-0. המונה הוא גם המיקום הפנוי הבא

בפעולה הבונה נבנה מערך באורך רצוי

במערך ערכי ברירת המחדל של string - null



```

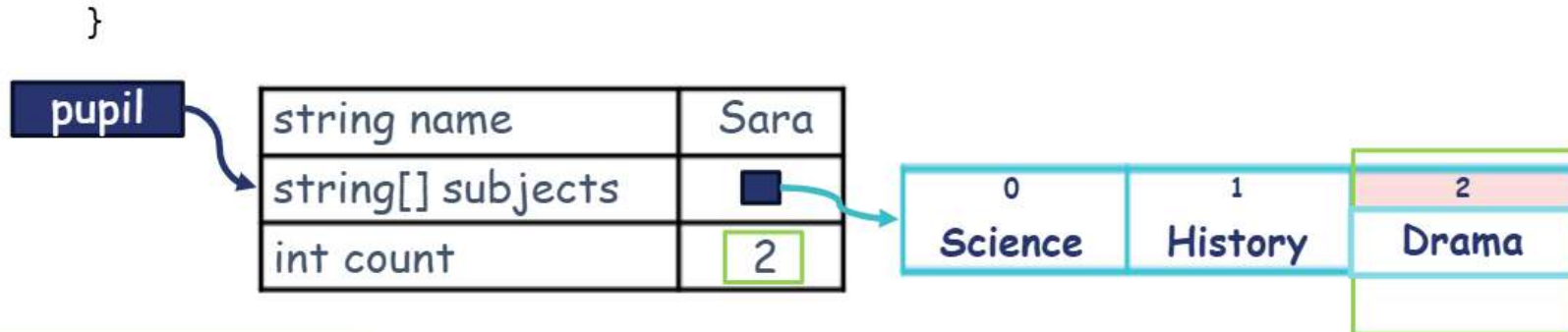
public void AddSubject(string newSubject)
{
    if (count < subjects.Length)
    {
        subjects[count] = newSubject;
        count++;
    }
}

```

דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה – הוספת מקצוע
2. מערך עם חשיבות לסדר

אם המערך אינו מלא במקצועות, נשים את
המקצוע בתא באינדקס count, ונעלה את
ה count ב-1

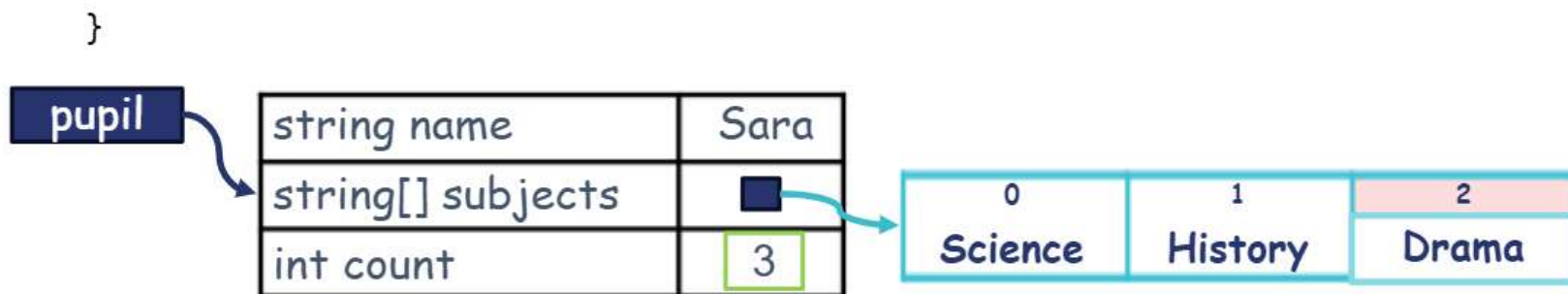


```
public void AddSubject(string newSubject)
{
    if (count < subjects.Length)
    {
        subjects[count] = newSubject;
        count++;
    }
}
```

דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה – הוספת מקצוע
2. מערך עם חשיבות לסדר

אם המערך אינו מלא במקצועות, נשים את המקצוע בתא באינדקס count, ונעלה את ה count ב-1



```

public void AddSubject(string newSubject)
{
    if (count < subjects.Length)
    {
        subjects[count] = newSubject;
        count++;
    }
    else // count = subjects.Length
    {
        string[] arr = new string[count + 2];
        for (int i = 0; i < subjects.Length; i++)
            arr[i] = subjects[i];
        arr[count] = newSubject;
        count++;
        subjects = arr;
    }
}

```

דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה

2. מערך עם חשיבות לסדר

○ כאשר המערך מלא, יש אפשרות לבנות מערך חדש גדול יותר, להעתיק את ערכי המערך המקורי ולעשות השמה לתכונת המערך

0	1	2	3	4
Science	History	Drama	Music	null

0	1	2
Science	History	Drama

string name	Sara
string[] subjects	
int count	4

pupil

```

public bool RemoveSubject(string subject) {
    int i = 0;

    while (i < count && subjects[i] != subject)
        i++;
    if (i == count)
        return false;

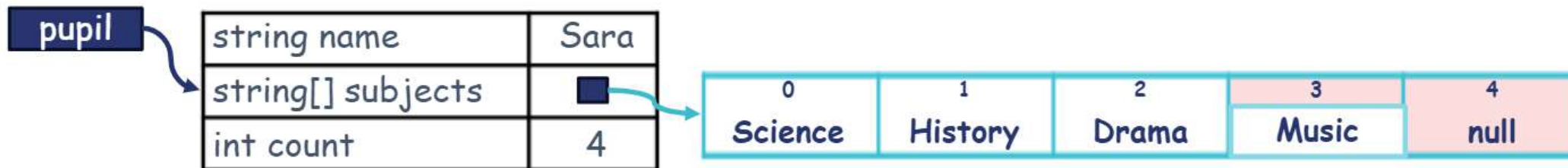
    for (; i < count - 1; i++)
        subjects[i] = subjects[i + 1];

    subjects[count - 1] = null;
    count--;
    return true;
}

```

דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה
2. מערך עם חשיבות לסדר



```

public bool RemoveSubject(string subject) {
    int i = 0;

    while (i < count && subjects[i] != subject)
        i++;
    if (i == count)
        return false;

    for (; i < count - 1; i++)
        subjects[i] = subjects[i + 1];

    subjects[count - 1] = null;
    count--;
    return true;
}

```

דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה

2. מערך עם חשיבות לסדר

0 בהוצאת איבר, עלינו לדאוג לסגירת ה"חור"

במערך, כיוון שכל הפעולות מניחות מערך
שאבריו החוקיים צמודים לשמאל

0 הסגירה – על ידי העתקת ההפניות למיקום
הקודם

0 יש לעדכן את מספר האיברים

0 השמת null במקום האחרון הקודם אינה
מתחייבת, כיוון שהלולאות ממילא לא יגיעו עד
אליו, ובהכנסה הבאה הוא ידרס



דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה

2. מערך עם חשיבות לסדר

○ בהוצאת איבר, עלינו לדאוג לסגירת ה"חור"

במערך, כיוון שכל הפעולות מניחות מערך שאבריו החוקיים צמודים לשמאל

○ הסגירה – על ידי העתקת ההפניות למיקום הקודם

○ יש לעדכן את מספר האיברים

○ השמת null במקום האחרון הקודם אינה מתחייבת, כיוון שהלולאות ממילא לא יגיעו עד אליו, ובהכנסה הבאה הוא ידרס

```
public bool RemoveSubject(string subject) {
    int i = 0;

    while (i < count && subjects[i] != subject)
        i++;
    if (i == count)
        return false;

    for (; i < count - 1; i++)
        subjects[i] = subjects[i + 1];

    subjects[count - 1] = null;
    count--;
    return true;
}
```



3
i

pupil

string name	Sara
string[] subjects	
int count	3

0	1	2	3	4
Science	Drama	Music	null	null

דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה

2. מערך עם חשיבות לסדר

```
public bool RemoveSubject(string subject) {
    int i = 0;

    while (i < count && subjects[i] != subject)
        i++;
    if (i == count)
        return false;

    for (; i < count - 1; i++)
        subjects[i] = subjects[i + 1];

    subjects[count - 1] = null;
    count--;
    return true;
}
```

0 בהוצאת איבר, עלינו לדאוג לסגירת ה"חור"

במערך, כיוון שכל הפעולות מניחות מערך שאבריו החוקיים צמודים לשמאל

0 הסגירה – על ידי העתקת ההפניות למיקום הקודם

0 יש לעדכן את מספר האיברים

0 השמת null במקום האחרון הקודם אינה מתחייבת, כיוון שהלולאות ממילא לא יגיעו עד אליו, ובהכנסה הבאה הוא ידרס

pupil

string name	Sara
string[] subjects	
int count	3

3
i

0	1	2	3	4
Science	Drama	Music	null	null

```

public bool RemoveSubject(string subject) {
    int i = 0;

    while (i < count && subjects[i] != subject)
        i++;
    if (i == count)
        return false;

    subjects[i] = subjects[count - 1];
    subjects[count - 1] = null;
    count--;
    return true;
}

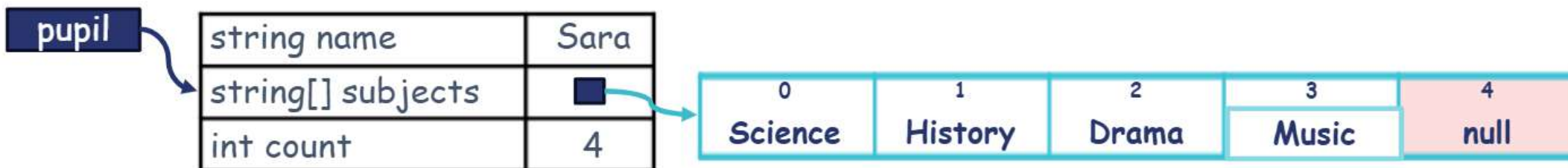
```

דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה

מערך שאין בו חשיבות לסדר

שימו לב – אם אין חשיבות לסדר האיברים במערך, אין צורך להזיז את כל האיברים כדי למלא את "החור", אלא אפשר להעביר את האיבר האחרון למקום שהתפנה ולעדכן את המונה



```

public bool RemoveSubject(string subject) {
    int i = 0;

    while (i < count && subjects[i] != subject)
        i++;
    if (i == count)
        return false;

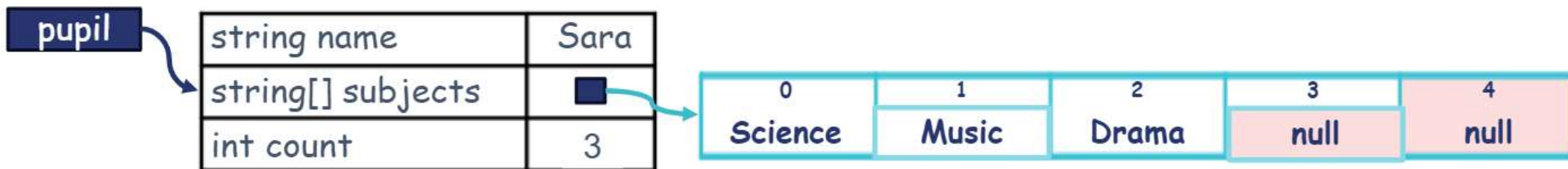
    subjects[i] = subjects[count - 1];
    subjects[count - 1] = null;
    count--;
    return true;
}

```

דוגמה – מחלקת תלמיד Pupil

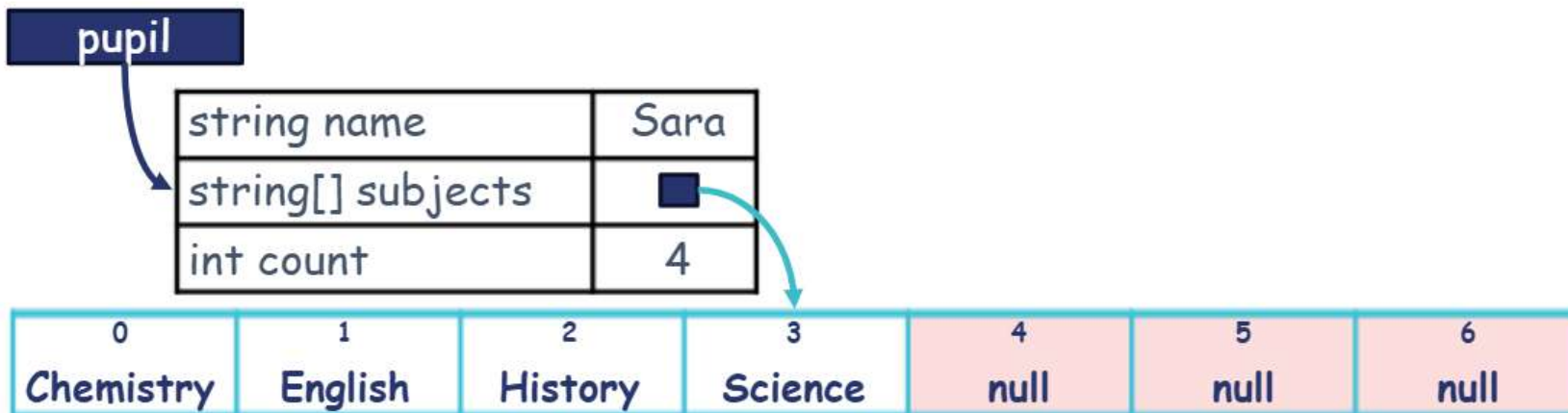
פעולות במחלקה
מערך שאין בו חשיבות לסדר

שימו לב – אם אין חשיבות לסדר האיברים במערך, אין צורך להזיז את כל האיברים כדי למלא את "החור", אלא אפשר להעביר את האיבר האחרון למקום שהתפנה ולעדכן את המונה



מערך כתכונה במחלקה - מחלקת תלמיד Pupil

- לעיתים יש חשיבות לסדר האיברים במערך בהתאם לערכם – והמערך חייב להיות **ממוין**, לדוגמה כאשר המקצועות במחלקת תלמיד **מסודרים לפי סדר הא"ב**
- במקרה זה, כדאי לשמור משתנה המונה את מספר המקצועות, וערכו הוא גם האינדקס הפנוי הבא במערך
- הוצאת איבר מהמערך תטופל כמו קודם, כיוון שבהוצאה נשמר סדר המיון



```

public bool RemoveSubject(string subject) {
    int i = 0;

    while (i < count && subjects[i] != subject)
        i++;
    if (i == count)
        return false;

    for (; i < count - 1; i++)
        subjects[i] = subjects[i + 1];

    subjects[count - 1] = null;
    count--;
    return true;
}

```

pupil →

string name	Sara
string[] subjects	
int count	4

0	1	2	3	4
Science	History	Drama	Music	null

דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה
הכנסה למערך ממזין

○ בהוצאת איבר, עלינו לדאוג לסגירת ה"חור" במערך, כיוון שכל הפעולות מניחות מערך שאבריו החוקיים צמודים לשמאל

○ הסגירה – על ידי העתקת ההפניות למיקום הקודם

○ יש לעדכן את מספר האיברים

○ השמת null במקום האחרון הקודם אינה מתחייבת, כיוון שהלולאות ממילא לא יגיעו עד אליו, ובהכנסה הבאה הוא ידרס

דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה
הכנסה למערך ממזין

```
public bool RemoveSubject(string subject) {
    int i = 0;

    while (i < count && subjects[i] != subject)
        i++;
    if (i == count)
        return false;

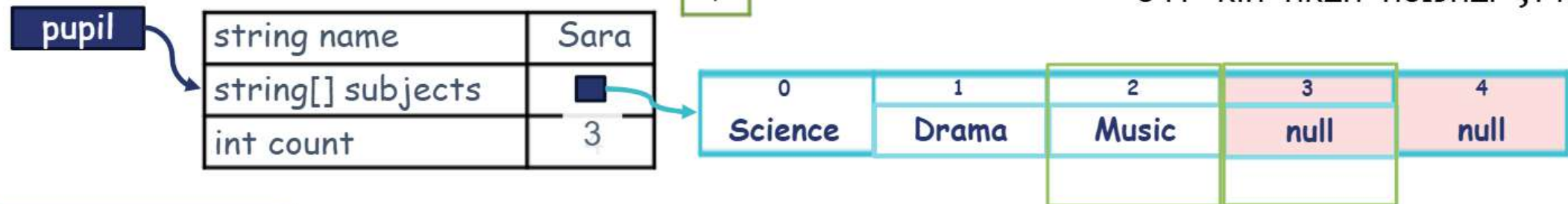
    for (; i < count - 1; i++)
        subjects[i] = subjects[i + 1];

    subjects[count - 1] = null;
    count--;
    return true;
}
```

בהוצאת איבר, עלינו לדאוג לסגירת ה"חור" במערך, כיוון שכל הפעולות מניחות מערך שאבריו החוקיים צמודים לשמאל הסגירה – על ידי העתקת ההפניות למיקום הקודם

יש לעדכן את מספר האיברים

השמת null במקום האחרון הקודם אינה מתחייבת, כיוון שהלולאות ממילא לא יגיעו עד אליו, ובהכנסה הבאה הוא ידרס



דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה
הכנסה למערך ממוין

```
public void AddSubjectSorted(string subject)
{
    int i = 0;

    while (i < count && subjects[i].CompareTo(subject) < 0)
        i++;

    for (int j = count; j > i; j--)
        subjects[j] = subjects[j-1];

    subjects[i] = subject;
    count++;
}
```

נחפש את המקום הנכון – לפני מקצוע שהוא אחרי במילון (סדר אלפביתי)

נפנה מקום – נזיז את כל האיברים ממקום זה ימינה

הכנסה למקום שהתפנה

פעולה מהמחלקה מחרוזת. אם המחרוזת שהועברה כפרמטר היא אחרי המילה עליה מפעילים את הפעולה, הפעולה תחזיר מספר שלילי.

פעולה זו מניחה שיש מקום במערך

אם נרצה לטפל גם במקרה בו אין מקום, נעשה זאת ע"י בניית מערך חדש, כמו קודם

פעולה זו אינה מחזירה ערך, כיוון שאנו מניחים שיש מקום במערך

0	1	2	3	4	5	6
Chemistry	English	History	Science	null	null	null

דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה
הכנסה למערך ממזין

```
public void AddSubjectSorted(string subject)
{
    int i = 0;

    while (i < count && subjects[i].CompareTo(subject) < 0)
        i++;

    for (int j = count; j > i; j--)
        subjects[j] = subjects[j-1];

    subjects[i] = subject;
    count++;
}
```

string name	Sara
int count	4
string[] subjects	

0	1	2	3	4	5	6
Chemistry	English	History	Science	null	null	null

- פעולה זו מניחה שיש מקום במערך
- אם נרצה לטפל גם במקרה בו אין מקום, נעשה זאת ע"י בניית מערך חדש, כמו קודם
- פעולה זו אינה מחזירה ערך, כיוון שאנו מניחים שיש מקום במערך

דוגמה – מחלקת תלמיד Pupil

פעולות במחלקה
הכנסה למערך ממזין

```
public void AddSubjectSorted(string subject)
{
    int i = 0;

    while (i < count && subjects[i].CompareTo(subject) < 0)
        i++;

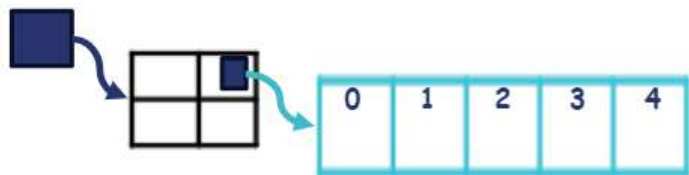
    for (int j = count; j > i; j--)
        subjects[j] = subjects[j-1];

    subjects[i] = subject;
    count++;
}
```

string name	Sara
int count	5
string[] subjects	

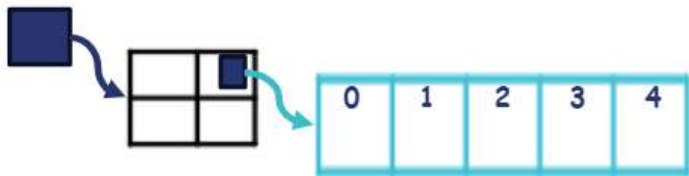
0	1	2	3	4	5	6
Chemistry	English	History	Science	Zoology	null	null

- פעולה זו מניחה שיש מקום במערך
- אם נרצה לטפל גם במקרה בו אין מקום, נעשה זאת ע"י בניית מערך חדש, כמו קודם
- פעולה זו אינה מחזירה ערך, כיוון שאנו מניחים שיש מקום במערך



מערך כתכונה במחלקה - סיכום

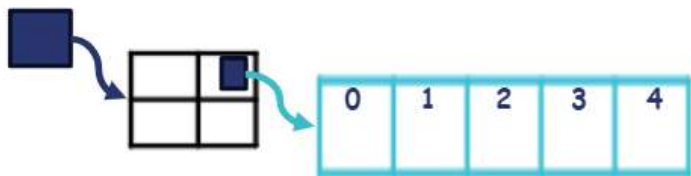
- מערך הוא עצם מהמחלקה Array
- כאשר אחת מתכונות העצם הוא מערך, העצם מכיל **הפניה למערך** ולא את המערך עצמו
- ניהול המידע יהיה בהתאם לצרכי המחלקה:
- מערך המתקבל כפרמטר בפעולה הבונה שכשיש בו ערכים:
 - אם איננו רוצים לאפשר גישה מחוץ למחלקה, הפעולה הבונה ופעולת הקביעה (Get) יצרו וישמרו עותק של המערך, ופעולת האחזור (Set), אם יש בה צורך, תחזיר (הפניה ל) עותק של המערך
 - מערך שמטרתו ניהול נתונים (הכנסה והוצאה)



מערך כתכונה במחלקה - סיכום

○ במערך שמטרתו ניהול נתונים (הכנסה והוצאה):

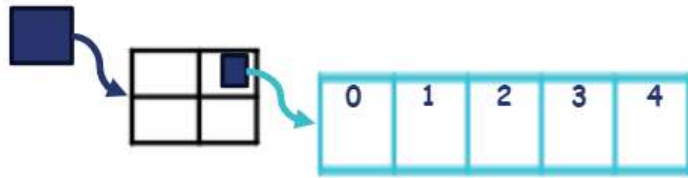
- הפעולה הבונה:
 - הפעולה הבונה תבנה מערך "ריק" מערכים חוקיים
 - המערך אינו באמת ריק, ערכיו הם ערכי ברירת מחדל ע"פ טיפוס המערך (0, 0.0, false, null)
 - גודל המערך:
- אם לכל העצמים מהטיפוס יש גודל אחיד/גודל מקסימלי של הנתונים – ניתן לקבוע תכונה קבועה (const) ובפעולה הבונה נבנה מערך בגודל זה
- הפעולה הבונה יכולה גם לקבל כפרמטר את גודל המערך הרצוי, אם הוא ייחודי לכל עצם מהמחלקה



מערך כתכונה במחלקה - סיכום

ניהול נתונים (הכנסה והוצאה):

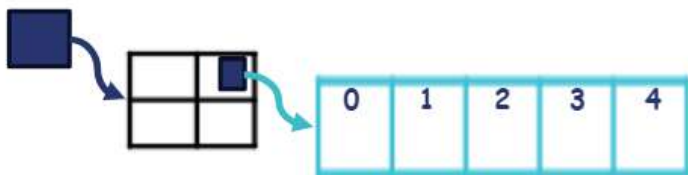
- כאשר אין חשיבות לסדר:
 - ניתן להוסיף בכל מקום "פנוי"
 - הדבר אפשרי רק כאשר קיים ערך ברירת מחדל, שהוא ערך חוקי לטיפוס המערך, אך לא ערך חוקי לאוסף הנתונים. לדוגמה – במערך ציונים, תא שערכו 0, יכול לציין תא "ריק", במערך שמות, תא שערכו null, יכול לציין תא "ריק"
- הכנסת איבר מחייבת מעבר על איברי המערך ומציאת מקום פנוי
- הוצאת האיבר תהיה ע"י השמת ערך ברירת המחדל במקום האיבר
- בפעולות המחלקה, בכל פעולה המבצעת מעבר על המערך, יש לבדוק שבתא יש ערך חוקי ולא ערך ברירת מחדל (תא "ריק")



מעריך כתכונה במחלקה - סיכום

ניהול נתונים (הכנסה והוצאה):

- שימוש במונה:
- ניתן להשתמש במונה המחזיק במספר האיברים במעריך
 - בפעולה הבונה נאתחל את המונה ל - 0
 - המונה הוא גם ערכו של התא ה"פנוי" הבא
 - בהכנסת איבר – אם יש מקום - נכניס לתא במיקום המונה, ונקדם את המונה. אם אין מקום, נוכל, אם נרצה, לבנות מעריך גדול יותר, להעתיק את הנתונים ולשנות את ערך התכונה למעריך החדש
 - בהוצאת איבר, **אם אין חשיבות לסדר**, נוכל לשים את הנתון האחרון (שנמצא במיקום $\text{count}-1$) במקום הנתון אותו יש להוציא, ולעדכן את ה count
 - בפעולות המחלקה, בכל פעולה המבצעת מעבר על המעריך, הלולאה תהיה עד המונה ולא עד סוף המעריך



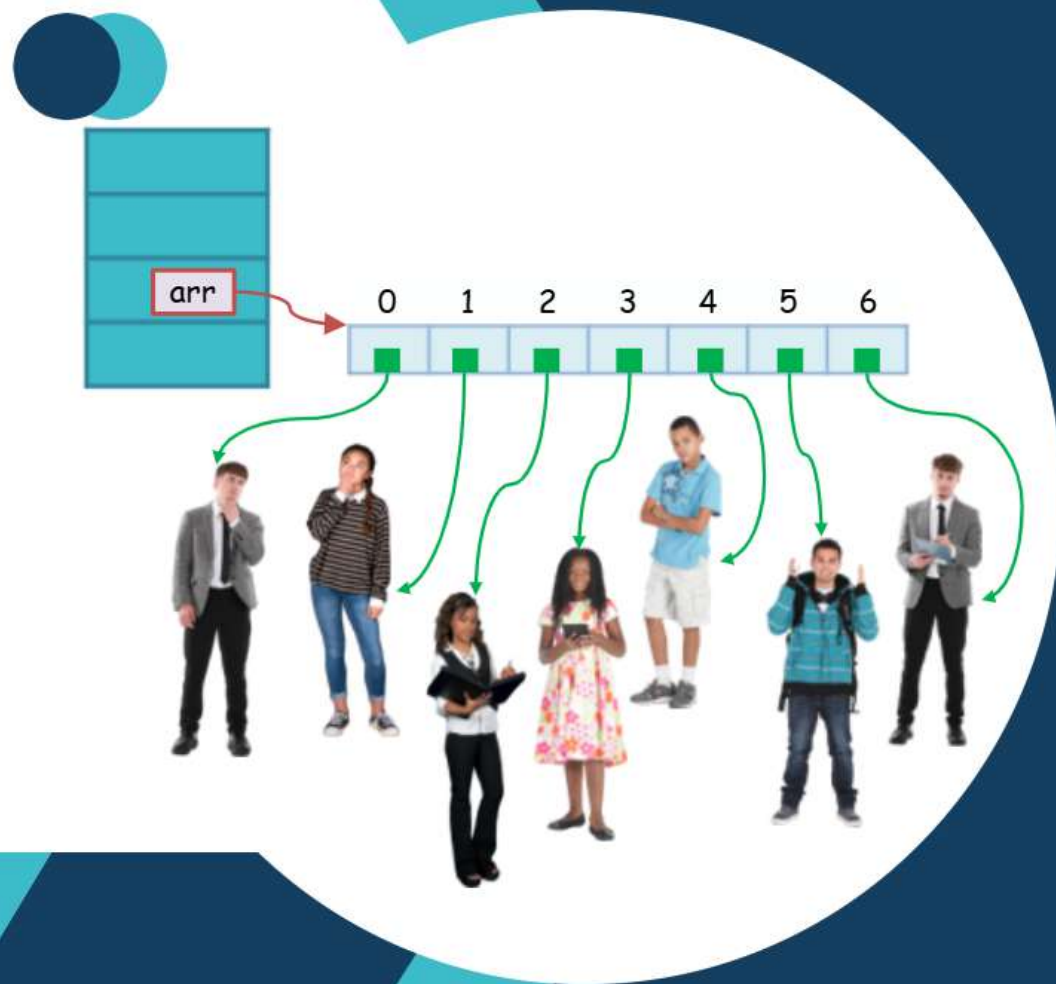
מערך כתכונה במחלקה - סיכום

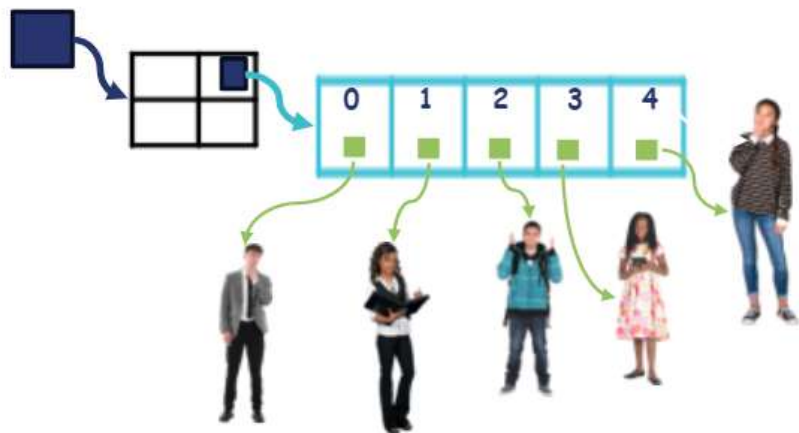
ניהול נתונים (הכנסה והוצאה):

- כאשר יש חשיבות לסדר ההכנסה ומשתמשים במונה איברים:
 - הכנסת האיבר תהיה במקום עליו מצביע המונה
 - הוצאת איבר מחייבת הזזה שמאלה של יתר האיברים לסגירת ה"חור" במערך
- כאשר יש חשיבות לסדר האיברים (מערך ממויין) ומשתמשים במונה איברים:
 - בהכנסת איבר, יש למצוא את המקום הנכון, להזיז את האיברים ימינה כדי לפנות מקום לאיבר החדש
 - הוצאת איבר מחייבת הזזה שמאלה של יתר האיברים לסגירת ה"חור" במערך

הכלת עצמים

מחלקה המכילה מערך עצמים

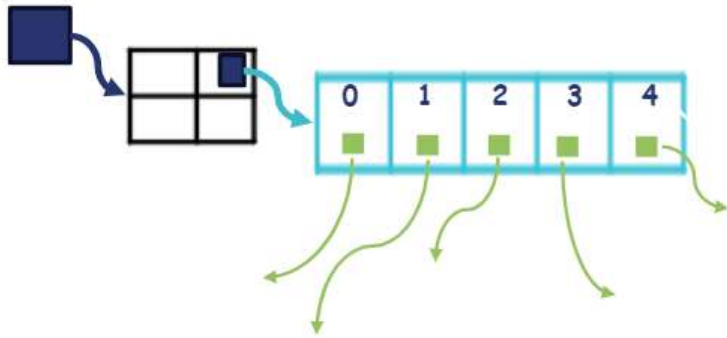




מערך עצמים כתכונה במחלקה

○ מערך הוא עצם מהמחלקה Array

- כאשר אחת מתכונות העצם הוא מערך, העצם מכיל הפניה למערך ולא את המערך עצמו
- כאשר המערך הוא מערך עצמים, בכל תא במערך תהיה הפניה לעצם, ולא העצם עצמו
- כאשר נוצר מערך עצים, יש בתאים null, כערך ברירת מחדל
- האופן בו ינוהל האוסף, תלוי בנתוני השאלה, בדיוק כמו במערך פשוט – האם יש חשיבות לסדר האיברים, האם נדרשות פעולות הכנסה והוצאה וכיוצ"ב
- במערך עצמים, יש לשקול לא רק האם תנתן גישה למערך מחוץ למחלקה, אלא האם תנתן גישה לכל עצם הנמצא במערך



מערך עצמים כתכונה במחלקה

○ סוס מרוץ מאופיין בשם, גיל, שווי, ומהירות מרבית

```
class RacingHorse
{
    private string name;
    private double price;
    private double speed;
    ...
}
```

○ לא ניתן לשנות את שם הסוס, מחיר ומהירות לא יכולים שליליים, לא ניתן לשנות את המהירות למהירות נמוכה הקיימת



```
class RacingHorse
```

```
{
```

```
    private string name;
```

```
    private double price;
```

```
    private double speed;
```

```
    public RacingHorse(string name, double price, double speed)
```

```
    {
```

```
        this.name = name;
```

```
        if (price > 0)
```

```
            this.price = price;
```

```
        if (speed > 0)
```

```
            this.speed = speed;
```

```
    }
```

```
    public RacingHorse(RacingHorse other)
```

```
    {
```

```
        this.name = other.name;
```

```
        this.price = other.price;
```

```
        this.speed = other.speed;
```

```
    }
```

```
    ...
```

```
}
```

RacingHorse מחלקת סוס מרוץ



מחלקת סוס מרוץ RacingHorse

```
class RacingHorse
{
    private string name;
    private double price;
    private double speed;

    public RacingHorse(string name, double price, double speed)
    {
        this.name = name;
        if (price > 0)
            this.price = price;
        if (speed > 0)
            this.speed = speed;
    }
    public RacingHorse(RacingHorse other)
    {
        this.name = other.name;
        this.price = other.price;
        this.speed = other.speed;
    }
    ...
}
```

פעולה בונה מעתיקה

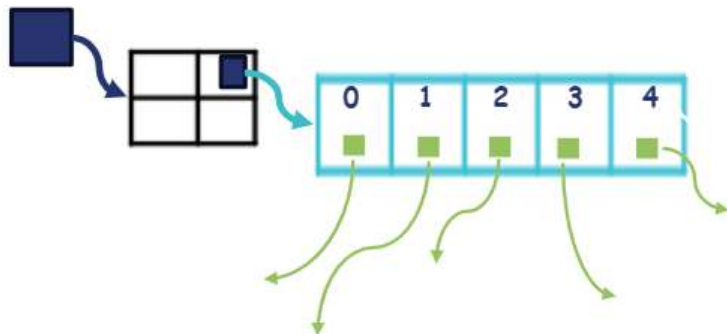


```
class RacingHorse
```

```
{  
    ...  
    public string GetName() { return name; }  
    public double GetSpeed() { return speed; }  
    public double GetPrice() { return price; }  
    public void SetPrice(double newPrice)  
    {  
        if (price > 0)  
            price = newPrice;  
    }  
    public void SetSpeed(double newSpeed)  
    {  
        if (newSpeed > speed)  
            speed = newSpeed;  
    }  
    public override string ToString()  
    {  
        return "Racing Horse: " + name + ", Price: " + pr  
            ", Speed: " + speed + " mph";  
    }  
}
```

RacingHorse מחלקת סוס מרוץ





מערך עצמים כתכונה במחלקה

- פרש מאופיין בשם והשנה בה נרשם לאגודת הפרשים
- פרש יכול להחזיק במספר סוסי מרוץ, ואלו מוחזקים בתאים מגודרים באורווה פרטית, שגודלה נקבע על ידו, והיא לפחות בת 3 תאים
- במידת האפשר, הסוסים ימוקמו התאים שאינם צמודים זה לזה, או על פי רצון הפרש



```
class Rider
{
    private string name;
    private int year;
    private RacingHorse[] horses;
    ...
}
```



```
class Rider
```

```
{  
    private string name;  
    private int year;  
    private RacingHorse[] horses;  
  
    public Rider(string name, int year, int num)  
    {  
        this.name = name;  
        this.year = year;  
        this.horses = new RacingHorse[num];  
    }  
  
    public string GetName() { return name; }  
    public int GetYear() { return year; }  
  
    public RacingHorse GetRacingHorse(int i)  
    {  
        if (horses[i] != null)  
            return new RacingHorse(horses[i]);  
        return null;  
    }  
    ...  
}
```

מחלקת פרש Rider

גודלו של מערך הסוסים מתקבל
כפרמטר. המערך נבנה בפעולה
הבונה וכל תאיו null

הסוס שייך לפרש, ולו בלבד. פעולת
Getn תחזיר העתק של הסוס, על
מנת להגן על נתוניו



```
class Rider
```

```
{  
    public bool AddHorse(RacingHorse horse, int place)  
    {  
        if (place >= 0 && place < horses.Length &&  
            horses[place] == null)  
        {  
            horses[place] = new RacingHorse(horse);  
            return true;  
        }  
        return false;  
    }  
  
    private int FreePlace()  
    {  
        for (int i = 0; i < horses.Length; i++)  
            if (horses[i] == null)  
                return i;  
        return -1;  
    }  
  
    ...  
}
```

מחלקת פרש Rider

פעולת ההוספה

- אין כל חשיבות לסדר הסוסים באורווה
- ניתן להכניס סוס לתא ספציפי באורווה, אם המיקום המבוקש חוקי ואם התא ריק
- הפעולה תחזיר אמת אם פעולת ההוספה הצליחה
- למערך נכניס סוס חדש, על מנת שלא תהיה הפנייה נוספת לסוס במקום אחר, דרכה יוכלו לפגוע בנתוניו

```
class Rider
```

```
{  
    public bool AddHorse(RacingHorse horse, int place)  
    {  
        if (place >= 0 && place < horses.Length &&  
            horses[place] == null)  
        {  
            horses[place] = new RacingHorse(horse);  
            return true;  
        }  
        return false;  
    }  
  
    private int FreePlace()  
    {  
        for (int i = 0; i < horses.Length; i++)  
            if (horses[i] == null)  
                return i;  
        return -1;  
    }  
  
    ...  
}
```

הפעולה מוצאת את המקום הראשון במערך, בו
אין סוס. אם אין מקום פנוי, יוחזר -1.

מחלקת פרש Rider

פעולת ההוספה

- אין כל חשיבות לסדר הסוסים באורווה
- ניתן להכניס סוס לתא ספציפי באורווה, אם המיקום המבוקש חוקי ואם התא ריק
- הפעולה תחזיר אמת אם פעולת ההוספה הצליחה
- למערך נכניס סוס חדש, על מנת שלא תהיה הפנייה נוספת לסוס במקום אחר, דרכה יוכלו לפגוע בנתוניו

מחלקת פרש Rider

פעולת ההוספה

```
class Rider  
{
```

```
    private int Find2SidesFreePlace()  
{
```

```
    if (horses[0] == null && horses[1] == null)  
        return 0;
```

```
    if (horses[horses.Length - 2] == null && horses[horses.Length - 1] == null)  
        return horses.Length - 1;
```

```
    for (int i = 1; i < horses.Length - 1; i++)
```

```
        if (horses[i] == null && horses[i - 1] == null && horses[i + 1] == null)  
            return i;
```

```
    }  
    ...
```

```
}
```

תא ראשון פנוי – אם אין אחריו סוס

תא אחרון פנוי – אם אין לפניו סוס

שימו לב לגבולות הלולאה – מניעת סכנה של חריגה ממערך

ניתן להכניס לאורווה במיקום פנוי כלשהו, אך סדר ההכנסה הוא:

במידת האפשר להכניס סוס לתא בו לא יהיו מצדדיו סוסים

אחרת, נעדיף לשים בתא בו יהיה לסוס החדש שכן אחד בלבד

אחרת, נשים במקום פנוי, אם קיים

נוסיף את הפעולות המוצאות מקום פנוי לפי כללים אלה, אחרת יוחזר -1

```
class Rider
```

```
{
```

```
    private int Find2SidesFreePlace()
```

```
    {
```

```
        if (horses[0] == null && horses[1] == null)
```

```
            return 0;
```

```
        if (horses[horses.Length - 2] == null && horses[horses.Length - 1] == null)
```

```
            return horses.Length - 1;
```

```
        for (int i = 1; i < horses.Length - 1; i++)
```

```
            if (horses[i] == null && horses[i - 1] == null && horses[i + 1] == null)
```

```
                return i;
```

```
    }
```

```
    ...
```

```
}
```

מחלקת פרש Rider

פעולת ההוספה

תא ראשון פנוי – אם אין אחריו סוס

תא אחרון פנוי – אם אין לפניו סוס

שימו לב לגבולות הלולאה – מניעת סכנה של חריגה ממערך

ניתן להכניס לאורווה במיקום פנוי כלשהו, אך סדר ההכנסה הוא:

במידת האפשר להכניס סוס לתא בו לא יהיו מצדדיו סוסים

אחרת, נעדיף לשים בתא בו יהיה לסוס החדש שכן אחד בלבד

אחרת, נשים במקום פנוי, אם קיים

נוסיף את הפעולות המוצאות מקום פנוי לפי כללים אלו, אחרת יוחזר -1

מחלקת פרש Rider

פעולת ההוספה

תא ראשון פנוי – אם אין אחריו סוס

תא אחרון פנוי – אם אין לפניו סוס

שימו לב לגבולות הלולאה – מניעת סכנה של חריגה ממערך

```
class Rider
{
    private int Find2SidesFreePlace()
    {
        if (horses[0] == null && horses[1] == null)
            return 0;
        if (horses[horses.Length - 2] == null && horses[horses.Length - 1] == null)
            return horses.Length - 1;

        for (int i = 1; i < horses.Length - 1; i++)
        {
            if (horses[i] == null && horses[i - 1] == null && horses[i + 1] == null)
                return i;
        }

        return -1;
    }
    ...
}
```

○ ניתן להכניס לאורווה במיקום פנוי כלשהו, אך סדר ההכנסה הוא:

○ במידת האפשר להכניס סוס לתא בו לא יהיו מצדדיו סוסים

○ אחרת, נעדיף לשים בתא בו יהיה לסוס החדש שכן אחד בלבד

○ אחרת, נשים במקום פנוי, אם קיים

○ נוסיף את הפעולות המוצאות מקום פנוי לפי כללים אלה, אחרת יוחזר -1

```
class Rider
```

```
{
```

```
    private int Find1SideFreePlace()
```

```
    {
```

```
        if (horses[0] == null)
```

```
            return 0;
```

```
        if (horses[horses.Length - 1] == null)
```

```
            return horses.Length - 1;
```

```
        for (int i = 1; i < horses.Length - 1; i++)
```

```
            if (horses[i] == null && horses[i - 1] == null || horses[i + 1] == null)
```

```
                return i;
```

```
    }
```

```
    ...
```

```
}
```

מחלקת פרש Rider

פעולת ההוספה

תא ראשון פנוי

תא אחרון פנוי

שימו לב לגבולות הלולאה – מניעת סכנה של חריגה ממערך

ניתן להכניס לאורווה במיקום פנוי כלשהו, אך סדר ההכנסה הוא:

במידת האפשר להכניס סוס לתא בו לא יהיו מצדדיו סוסים

אחרת, נעדיף לשים בתא בו יהיה לסוס החדש שכן אחד בלבד

אחרת, נשים במקום פנוי, אם קיים

נוסיף את הפעולות המוצאות מקום פנוי לפי כללים אלו, אחרת יוחזר -1

```
class Rider
```

```
{
```

```
    private int Find1SideFreePlace()
```

```
    {
```

```
        if (horses[0] == null)
```

```
            return 0;
```

```
        if (horses[horses.Length - 1] == null)
```

```
            return horses.Length - 1;
```

```
        for (int i = 1; i < horses.Length - 1; i++)
```

```
            if (horses[i] == null && horses[i - 1] == null || horses[i + 1] == null)
```

```
                return i;
```

```
        return -1;
```

```
    }
```

```
    ...
```

```
}
```

מחלקת פרש Rider

פעולת ההוספה

תא ראשון פנוי

תא אחרון פנוי

שימו לב לגבולות הלולאה – מניעת סכנה של חריגה ממערך

ניתן להכניס לאורווה במיקום פנוי כלשהו, אך סדר ההכנסה הוא:

במידת האפשר להכניס סוס לתא בו לא יהיו מצדדיו סוסים

אחרת, נעדיף לשים בתא בו יהיה לסוס החדש שכן אחד בלבד

אחרת, נשים במקום פנוי, אם קיים

נוסיף את הפעולות המוצאות מקום פנוי לפי כללים אלה, אחרת יוחזר -1

```
class Rider
```

```
{
```

```
    public bool AddHorse(RacingHorse horse)
```

```
    {
```

```
        int pos = Find2SidesFreePlace();
```

```
        if (pos == -1)
```

```
        {
```

```
            pos = Find1SideFreePlace();
```

```
            if (pos == -1)
```

```
                pos = FreePlace();
```

```
        }
```

```
        if (pos != -1)
```

```
        {
```

```
            horses[pos] = new RacingHorse(horse);
```

```
            return true;
```

```
        }
```

```
        return false;
```

```
    }
```

```
    ...
```

```
}
```

מחלקת פרש Rider

פעולת ההוספה

ניתן להכניס לאורווה במיקום פנוי כלשהו, אך סדר ההכנסה הוא: O

במידת האפשר להכניס סוס לתא בו לא יהיו מצדדיו סוסים O

אחרת, נעדיף לשים בתא בו יהיה לסוס החדש שכן אחד בלבד O

אחרת, נשים במקום פנוי, אם קיים O

```
class Rider
```

```
{
```

```
    public bool AddHorse(RacingHorse horse)
```

```
    {
```

```
        int pos = Find2SidesFreePlace();
```

```
        if (pos == -1)
```

```
        {
```

```
            pos = Find1SideFreePlace();
```

```
            if (pos == -1)
```

```
                pos = FreePlace();
```

```
        }
```

```
        if (pos != -1)
```

```
        {
```

```
            horses[pos] = new RacingHorse(horse);
```

```
            return true;
```

```
        }
```

```
        return false;
```

```
    }
```

```
    ...
```

```
}
```

מחלקת פרש Rider

פעולת ההוספה

נכניס העתק של הסוס, על מנת שלא
תהיה הפניה לסוס ממקום אחר

○ ניתן להכניס לאורווה במיקום פנוי כלשהו, אך סדר ההכנסה הוא:

○ במידת האפשר להכניס סוס לתא בו לא יהיו מצדדיו סוסים

○ אחרת, נעדיף לשים בתא בו יהיה לסוס החדש שכן אחד בלבד

○ אחרת, נשים במקום פנוי, אם קיים

```
public override string ToString()
{
    string str = "Rider: " + name + " , Year: " + year + "\n";
    for (int i = 0; i < horses.Length; i++)
        if (horses[i] != null)
            str += "\t" + i + ")" + horses[i] + "\n";
    return str;
}
```

```
public void Print()
{
    Console.ForegroundColor = ConsoleColor.Red;
    Console.WriteLine("\n Rider: " + name + " , Year: " + year + ":");
    Console.ResetColor();
    for (int i = 0; i < horses.Length; i++)
        if (horses[i] != null)
        {
            Console.ForegroundColor = ConsoleColor.Green;
            Console.Write("      " + i + ")");
            Console.ResetColor();
            Console.WriteLine(" \t" + horses[i]);
        }
    Console.ResetColor();
}
```

מחלקת פרש Rider

פעולת ToString

○ הפעולה מחזירה מחרוזת המתארת את העצם.

○ לכל סוס, היא מחזירה גם את מיקומו וגם את תיאור הסוס.

הוספנו גם פעולת הדפסה, המדפיסה פלט צבעוני. זו אינה נדרשת, והיא לצורך התצוגה בלבד.

```
class Program
```

```
{
```

```
    static void Main(string[] args)
```

```
    {
```

```
        Rider rider1 = new Rider("Isabella", 2015, 5);
```

```
        RacingHorse horse1 =
```

```
            new RacingHorse("Buddy", 33000, 50.45);
```

```
        rider1.AddHorse(horse1);
```

```
Rider rider1
```

```
RacingHorse horse1
```

במערך יכנס עותק (!) של העצם

string name	Isabella
int year	2015
RacingHorse[] horses	

0	1	2	3	4
null	null	null	null	null

string name	Buddy
int price	33000
double speed	50.45

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
    }
}

```

Rider rider1	
RacingHorse horse1	

על פי סדר ההכנסה, אם שני התאים הראשונים ריקים, יוכנס בראשון

string name	Isabella
int year	2015
RacingHorse[] horses	

0	1	2	3	4
null	null	null	null	null

string name	Buddy
int price	33000
double speed	50.45

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
    }
}

```

Rider rider1	
RacingHorse horse1	

string name	Buddy
int price	33000
double speed	50.45

string name	Isabella
int year	2015
RacingHorse[] horses	

0	1	2	3	4
	null	null	null	null

string name	Buddy
int price	33000
double speed	50.45

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
    }
}

```

Rider rider1	
RacingHorse horse1	
RacingHorse horse2	

string name	Buddy
int price	33000
double speed	50.45

string name	Silver
int price	45000
double speed	46.87

string name	Isabella
int year	2015
RacingHorse[] horses	

0	1	2	3	4
	null	null	null	null

string name	Buddy
int price	33000
double speed	50.45

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
    }
}

```

על פי סדר ההכנסה, אם התא
הראשון תפוס, אך שני האחרונים
פנויים, יוכנס באחרון

Rider rider1	
RacingHorse horse1	
RacingHorse horse2	

string name	Buddy
int price	33000
double speed	50.45

string name	Silver
int price	45000
double speed	46.87

string name	Isabella
int year	2015
RacingHorse[] horses	

0	1	2	3	4
	null	null	null	null

string name	Buddy
int price	33000
double speed	50.45

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
    }
}

```

Rider rider1	
RacingHorse horse1	
RacingHorse horse2	

string name	Buddy
int price	33000
double speed	50.45

string name	Silver
int price	45000
double speed	46.87

string name	Isabella
int year	2015
RacingHorse[] horses	

0	1	2	3	4
	null	null	null	

string name	Buddy
int price	33000
double speed	50.45

string name	Silver
int price	45000
double speed	46.87

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
    }
}

```

Rider rider1	
RacingHorse horse1	
RacingHorse horse2	
RacingHorse horse3	

string name	Buddy
int price	33000
double speed	50.45

string name	Silver
int price	45000
double speed	46.87

string name	Isabella
int year	2015
RacingHorse[] horses	

0	1	2	3	4
	null	null	null	

string name	Buddy
int price	33000
double speed	50.45

string name	Silver
int price	45000
double speed	46.87

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
    }
}

```

Rider rider1	
RacingHorse horse1	
RacingHorse horse2	
RacingHorse horse3	

string name	Buddy
int price	33000
double speed	50.45

string name	Silver
int price	45000
double speed	46.87

string name	Shadow
int price	38550
double speed	40.15

string name	Isabella
int year	2015
RacingHorse[] horses	

0	1	2	3	4
	null	null	null	

string name	Buddy
int price	33000
double speed	50.45

string name	Silver
int price	45000
double speed	46.87

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
    }
}

```

אחרת נחפש מקום ריק שצדדיו
ריקים

Rider rider1	
RacingHorse horse1	
RacingHorse horse2	
RacingHorse horse3	

string name	Buddy
int price	33000
double speed	50.45

string name	Silver
int price	45000
double speed	46.87

string name	Shadow
int price	38550
double speed	40.15

string name	Isabella
int year	2015
RacingHorse[] horses	

0	1	2	3	4
	null	null	null	

string name	Buddy
int price	33000
double speed	50.45

string name	Silver
int price	45000
double speed	46.87

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
    }
}

```

Rider rider1	
RacingHorse horse1	
RacingHorse horse2	
RacingHorse horse3	

string name	Buddy
int price	33000
double speed	50.45

string name	Silver
int price	45000
double speed	46.87

string name	Shadow
int price	38550
double speed	40.15

string name	Isabella
int year	2015
RacingHorse[] horses	

0	1	2	3	4
	null		null	

string name	Buddy
int price	33000
double speed	50.45

string name	Shadow
int price	38550
double speed	40.15

string name	Silver
int price	45000
double speed	46.87

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
    }
}

```

Rider rider1	
RacingHorse horse1	
RacingHorse horse2	
RacingHorse horse3	
RacingHorse horse4	

string name	Buddy
int price	33000
double speed	50.45

string name	Silver
int price	45000
double speed	46.87

string name	Shadow
int price	38550
double speed	40.15

string name	Blaze
int price	40000
double speed	48.50

string name	Isabella
int year	2015
RacingHorse[] horses	

0	1	2	3	4
	null		null	

string name	Buddy
int price	33000
double speed	50.45

string name	Shadow
int price	38550
double speed	40.15

string name	Silver
int price	45000
double speed	46.87

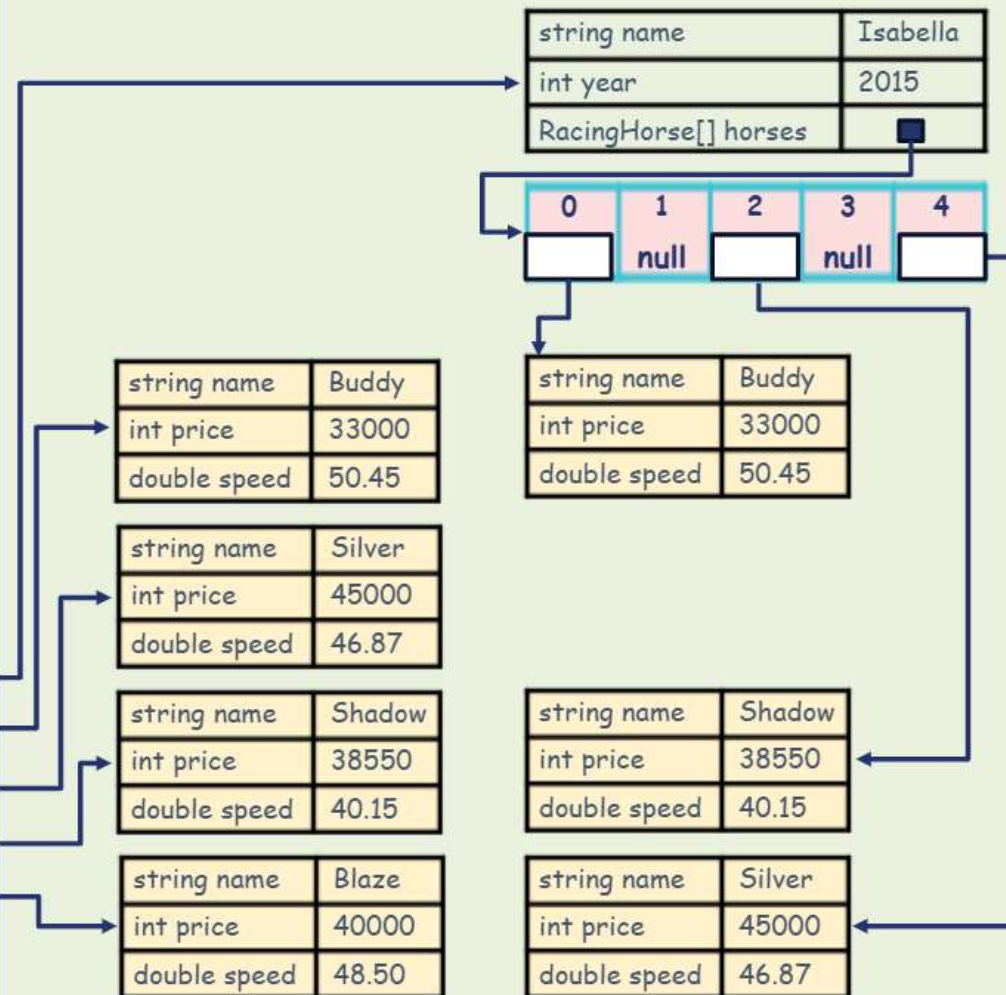
```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
        rider1.AddHorse(horse4);
    }
}

```

Rider rider1	
RacingHorse horse1	
RacingHorse horse2	
RacingHorse horse3	
RacingHorse horse4	

אחרת נחפש תא עם שכן אחד, ואם אין, אז מקום פנוי



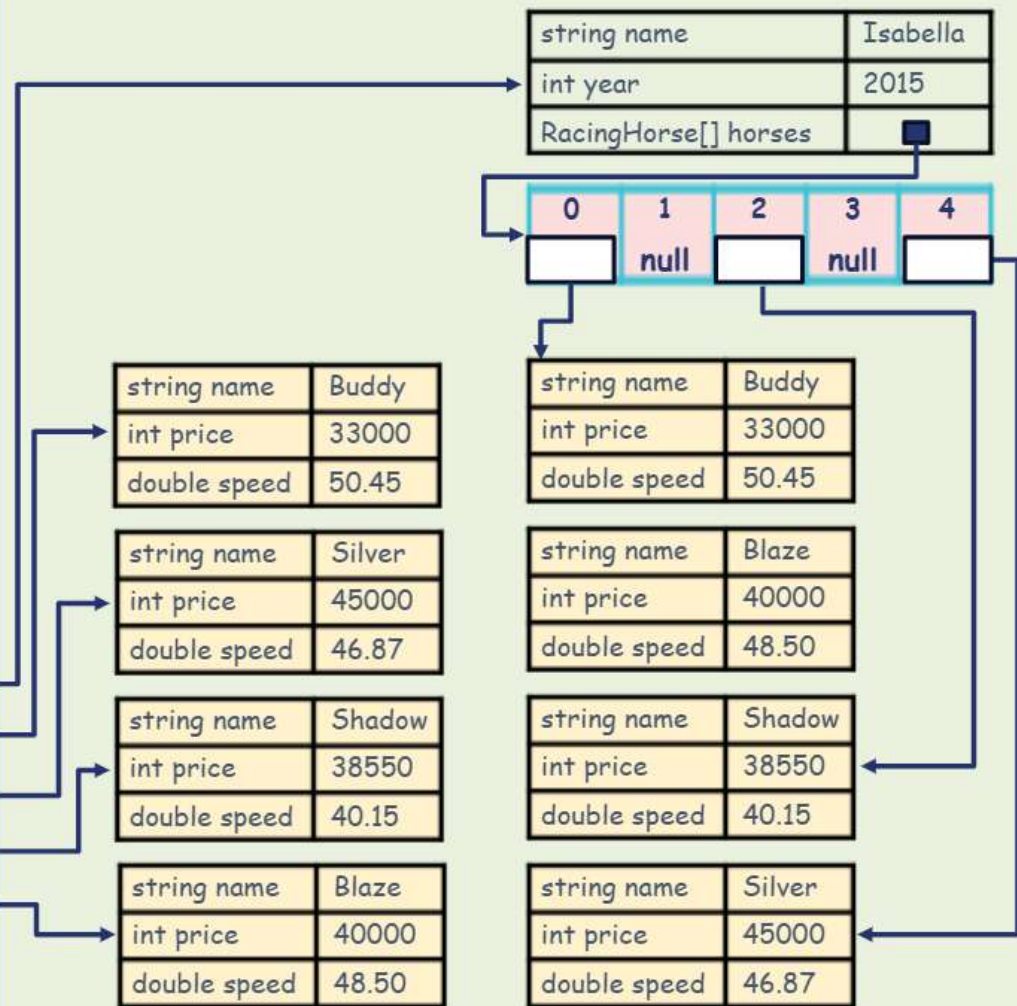
```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
        rider1.AddHorse(horse4);
    }
}

```

Rider rider1	
RacingHorse horse1	
RacingHorse horse2	
RacingHorse horse3	
RacingHorse horse4	

אחרת נחפש תא עם שכן אחד, ואם אין, אז מקום פנוי



```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
        rider1.AddHorse(horse4);
    }
}

```

Rider rider1	
RacingHorse horse1	
RacingHorse horse2	
RacingHorse horse3	
RacingHorse horse4	

string name	Buddy
int price	33000
double speed	50.45

string name	Silver
int price	45000
double speed	46.87

string name	Shadow
int price	38550
double speed	40.15

string name	Blaze
int price	40000
double speed	48.50

string name	Isabella
int year	2015
RacingHorse[] horses	

0	1	2	3	4
			null	

string name	Buddy
int price	33000
double speed	50.45

string name	Blaze
int price	40000
double speed	48.50

string name	Shadow
int price	38550
double speed	40.15

string name	Silver
int price	45000
double speed	46.87

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);

        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        rider1.Print();

        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        rider1.Print();

        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        rider1.Print();

        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
        rider1.AddHorse(horse4);
        rider1.Print();
    }
}

```

שימוש - מחלקת פרש Rider

```

Rider: Isabella , Year: 2015:
0)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

Rider: Isabella , Year: 2015:
0)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
4)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Rider: Isabella , Year: 2015:
0)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
2)      Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
4)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);

        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        rider1.Print();

        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        rider1.Print();

        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        rider1.Print();

        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
        rider1.AddHorse(horse4);
        rider1.Print();
    }
}

```

שימוש - מחלקת פרש Rider

```

Rider: Isabella , Year: 2015:
0)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

Rider: Isabella , Year: 2015:
0)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
4)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Rider: Isabella , Year: 2015:
0)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
2)      Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
4)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Rider: Isabella , Year: 2015:
0)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
1)      Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2)      Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
4)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);

        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        rider1.Print();

        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        rider1.Print();

        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        rider1.Print();

        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
        rider1.AddHorse(horse4);
        rider1.Print();
    }
}

```

שימוש - מחלקת פרש Rider

```

Rider: Isabella , Year: 2015:
0)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

Rider: Isabella , Year: 2015:
0)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
4)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Rider: Isabella , Year: 2015:
0)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
2)      Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
4)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Rider: Isabella , Year: 2015:
0)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
1)      Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2)      Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
4)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

```

בהדפסה הוספנו את האינדקסים של כל
תא. תאים "ריקים" לא הודפסו.

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);

        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        rider1.Print();

        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        rider1.Print();

        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        rider1.Print();

        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
        rider1.AddHorse(horse4);
        rider1.Print();
    }
}

```

שימוש - מחלקת פרש Rider

```

Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

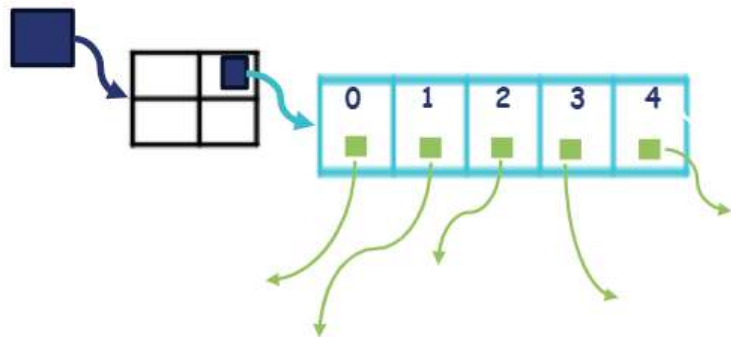
Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
1) Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

```

בהדפסה הוספנו את האינדקסים של כל
תא. תאים "ריקים" לא הודפסו.



מערך עצמים כתכונה במחלקה

- הסוסים המשתתפים בתחרויות רשומים במועדון רכיבה
- כל פרש יכול לרשום סוסים למועדון
- המועדון שומר את הסוסים במערך על פי שווים, מהסוס היקר ביותר לסוס הזול ביותר
- המועדון אחראי על רישום המהירות הטובה ביותר של הסוס, המתעדכנת, במידת הצורך



```
class EquestrianClub
```

```
{
```

```
    private RacingHorse[] horses;
```

```
    private int horseCount;
```

```
    ...
```

```
}
```



מחלקת מועדון רכיבה EquestrianClub

```
class EquestrianClub
{
    private RacingHorse[] horses;
    private int horseCount;

    public EquestrianClub(int maxHorses)
    {
        horses = new RacingHorse[maxHorses];
        horseCount = 0;
    }

    ...

}
```

גודלו של מערך הסוסים מתקבל
כפרמטר. המערך נבנה בפעולה
הבונה וכל תאיו null

מונה הסוסים מאותחל ל-0



מחלקת מועדון רכיבה EquestrianClub

פעולת ההוספה

```
class EquestrianClub
{
    public bool AddHorse(RacingHorse horse)
    {
        if (horseCount < horses.Length)
        {
            int indexToInsert = horseCount;
```

```
        ...
    }
```

- הסוסים מסודרים מהסוס היקר לסוס הזול
- כיוון שהמועדון אחראי לעדכון המהירות המקסימלית של הסוס, תוכנס למערך הפניה נוספת לסוס ולא הפניה להעתק של העצם
- הסוסים שייכים לפרש ולא למועדון

EquestrianClub מועדון רכיבה

פעולת ההוספה

```
class EquestrianClub
{
    public bool AddHorse(RacingHorse horse)
    {
        if (horseCount < horses.Length)
        {
            int indexToInsert = horseCount;
            while (indexToInsert > 0 && horse.GetPrice() > horses[indexToInsert - 1].GetPrice())
            {
                horses[indexToInsert] = horses[indexToInsert - 1];
                indexToInsert--;
            }
            horses[indexToInsert] = horse;
            horseCount++;
            return true;
        }
        return false;
    }
    ...
}
```

המיקום הפנוי הראשון

הלולאה רצה מסוף המערך לתחילתו, ומעבירה כל סוס שמחירו גדול מהסוס שיש להכניס, תא אחד קדימה, כדי לפנות מקום להכנסת הסוס

○ הסוסים מסודרים מהסוס היקר לסוס הזול

○ כיוון שהמועדון אחראי לעדכון המהירות המקסימלית של הסוס, תוכנס למערך הפניה נוספת לסוס ולא הפניה להעתק של העצם

○ הסוסים שייכים לפרש ולא למועדון

```
class Rider
```

```
{
```

```
    public void JoinClub(EquestrianClub club, int i)
```

```
    {
```

```
        if(i >= 0 && i < horses.Length && horses[i] != null)
```

```
            club.AddHorse(horses[i]);
```

```
    }
```

```
    ...
```

```
}
```

מחלקת פרש Rider

פעולת הוספה למועדון

○ הפעולה מאפשרת לפרש לצרף

את אחד מסוסיו למועדון

○ כיוון שזהו הסוס של הפרש,

ונמצא אצל הפרש, הפעולה

שולחת כפרמטר את ההפניה של

הסוס ולא העתק



```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
        rider1.AddHorse(horse4);

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.ListHorses(); //print horses
    }
}

```

Rider: Isabella , Year: 2015:

```

0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
1) Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

```

Racing Horses in the equestrian club:

```

0) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1) Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
        rider1.AddHorse(horse4);

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.ListHorses(); //print horses
    }
}

```

```

Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
1) Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Racing Horses in the equestrian club:
0) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1) Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
        rider1.AddHorse(horse4);

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.ListHorses(); //print horses
    }
}

```

```

Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
1) Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Racing Horses in the equestrian club:
0) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1) Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
        rider1.AddHorse(horse4);

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.ListHorses(); //print horses
    }
}

```

```

Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
1) Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Racing Horses in the equestrian club:
0) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1) Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
        rider1.AddHorse(horse4);

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.ListHorses(); //print horses
    }
}

```

Rider: Isabella , Year: 2015:

```

0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
1) Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

```

Racing Horses in the equestrian club:

```

0) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1) Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        RacingHorse horse3 =
            new RacingHorse("Shadow", 38550, 40.15);
        rider1.AddHorse(horse3);
        RacingHorse horse4 =
            new RacingHorse("Blaze", 40000, 48.50);
        rider1.AddHorse(horse4);

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.ListHorses(); //print horses
    }
}

```

Rider: Isabella , Year: 2015:

```

0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
1) Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

```

Racing Horses in the equestrian club:

```

0) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1) Racing Horse: Blaze, Price: 40000, Speed: 48.5 mph
2) Racing Horse: Shadow, Price: 38550, Speed: 40.15 mph
3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

```

מחלקת מועדון רכיבה EquestrianClub

פעולת שינוי מהירות

```
class EquestrianClub
{
    public void ChangeSpeed(int i, double speed)
    {
        if (horses[i] != null)
            horses[i].SetSpeed(speed);
    }
    ...
}
```

○ המועדון אחראי לשינוי מהירות ברגע שהסוס שבר שיא אישי בתחרות

○ מדובר בהפניה נוספת לסוס של הפרש



```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        rider1.Print();

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.AddHorse(horse1);
        club.AddHorse(horse2);

        club.ListHorses(); //print horses
    }
}

```

Rider: Isabella , Year: 2015:

0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
 4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Racing Horses in the equestrian club:

0) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
 1) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
 2) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
 3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        rider1.Print();

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.AddHorse(horse1);
        club.AddHorse(horse2);

        club.ListHorses(); //print horses

        club.ChangeSpeed(0, 89.9);

    }
}

```

```

Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Racing Horses in the equestrian club:
0) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
2) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        rider1.Print();

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.AddHorse(horse1);
        club.AddHorse(horse2);

        club.ListHorses(); //print horses

        club.ChangeSpeed(0, 89.9);

        club.ListHorses();
    }
}

```

```

Rider: Isabella , Year: 2015:
0)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
4)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Racing Horses in the equestrian club:
0)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
2)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
3)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        rider1.Print();

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.AddHorse(horse1);
        club.AddHorse(horse2);

        club.ListHorses(); //print horses

        club.ChangeSpeed(0, 89.9);

        club.ListHorses();
    }
}

```

```

Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Racing Horses in the equestrian club:
0) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
2) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

Racing Horses in the equestrian club:
0) Racing Horse: Silver, Price: 45000, Speed: 89.9 mph
1) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
2) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        rider1.Print();

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.AddHorse(horse1);
        club.AddHorse(horse2);

        club.ListHorses(); //print horses

        club.ChangeSpeed(0, 89.9);

        club.ListHorses();

        rider1.Print();
    }
}

```

```

Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Racing Horses in the equestrian club:
0) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
2) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

Racing Horses in the equestrian club:
0) Racing Horse: Silver, Price: 45000, Speed: 89.9 mph
1) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
2) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
4) Racing Horse: Silver, Price: 45000, Speed: 89.9 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);
        rider1.Print();

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.AddHorse(horse1);
        club.AddHorse(horse2);

        club.ListHorses(); //print horses

        club.ChangeSpeed(0, 89.9);

        club.ListHorses();

        rider1.Print();
    }
}

```

```

Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Racing Horses in the equestrian club:
0) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
2) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

Racing Horses in the equestrian club:
0) Racing Horse: Silver, Price: 45000, Speed: 89.9 mph
1) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
2) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
3) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
4) Racing Horse: Silver, Price: 45000, Speed: 89.9 mph

```

מחלקת פרש מכניסה העתק של העצם
ולא העתק הפניה!

```
class Rider
{
```

```
    public bool RemoveHorse(String name)
    {
        for (int i = 0; i < horses.Length; i++)
            if (horses[i] != null && horses[i].GetName() == name)
            {
                horses[i] = null;
                return true;
            }
        return false;
    }
    ...
}
```

האם הפעולה מוחקת את הסוס? O
לא עדיין ההפניה לסוס, שקיימת במחלקת מועדון O
הרכיבה, קיימת



מחלקת פרש Rider

פעולת הסרת סוס



```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.ListHorses(); //print horses

        Console.WriteLine("\n remove? " +
            rider1.RemoveHorse("Buddy"));

        rider1.Print();
        club.ListHorses();
    }
}

```

```

Rider: Isabella , Year: 2015:
0)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
4)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Racing Horses in the equestrian club:
0)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

remove? True

Rider: Isabella , Year: 2015:
4)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Racing Horses in the equestrian club:
0)      Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1)      Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        Rider rider1 = new Rider("Isabella", 2015, 5);
        RacingHorse horse1 =
            new RacingHorse("Buddy", 33000, 50.45);
        rider1.AddHorse(horse1);
        RacingHorse horse2 =
            new RacingHorse("Silver", 45000, 46.87);
        rider1.AddHorse(horse2);

        EquestrianClub club = new EquestrianClub(20);
        for(int i = 0; i < 5; i++)
            rider1.JoinClub(club, i);

        club.ListHorses(); //print horses

        Console.WriteLine("\n remove? " +
            rider1.RemoveHorse("Buddy"));

        rider1.Print();
        club.ListHorses();
    }
}

```

```

Rider: Isabella , Year: 2015:
0) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Racing Horses in the equestrian club:
0) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

remove? True

Rider: Isabella , Year: 2015:
4) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph

Racing Horses in the equestrian club:
0) Racing Horse: Silver, Price: 45000, Speed: 46.87 mph
1) Racing Horse: Buddy, Price: 33000, Speed: 50.45 mph

```

חסוס עדיין קיים במועדון!

```
class Rider  
{
```

אם הפרש רוצה לבטל חברות של סוס
במועדון, עליו לפנות אל עצם המועדון
ולהפעיל פעולת הסרה!

הפתרון – יש להוסיף לפרש תכונה
נוספת! הפניה למועדון

```
}
```

מחלקת פרש Rider

בעיה נוספת - פעולת הוצאת סוס מהמועדון



```
class Rider
```

```
{
```

```
    private string name;
```

```
    private int year;
```

```
    private RacingHorse[] horses;
```

```
    EquestrianClub club;
```

```
    public Rider(string name, int year, int num, EquestrianClub club)
```

```
    {
```

```
        this.name = name;
```

```
        this.year = year;
```

```
        this.horses = new RacingHorse[num];
```

```
        this.club = club;
```

```
    }
```

```
    ...
```

```
}
```

תיקון מחלקת פרש Rider

ההפניה למועדון תשמר כתכונה

יש לשמור, כמובן, העתק הפניה –
עלינו לפנות לעצם המועדון האמיתי,
בו נמצאים הסוסים, ולא להעתק



```
class Rider
```

```
{
```

```
    public void RemoveFromClub(int i)
```

```
    {
```

```
        if(i >= 0 && i < horses.Length && horses[i] != null)
```

```
            club.RemoveHorse(horses[i]);
```

```
    }
```

```
    ...
```

```
}
```



תיקון מחלקת פרש Rider

פעולת הסרה מהמועדון

- כיוון שהפניה למועדון נשמרת כתכונה, אין צורך בפרמטר
- יש להוסיף את הפעולה RemoveHorse למועדון

מחלקת מועדון רכיבה EquestrianClub

פעולת ההוצאה

```
class EquestrianClub
```

```
{  
    public bool RemoveHorse(RacingHorse horse)
```

```
{
```

```
    int i = 0;
```

```
    while (i < horseCount &&  
           (horse.GetPrice() < horses[i].GetPrice() || horse != horses[i]))
```

```
        i++;
```

```
    if (horse != horses[i])  
        return false;
```

```
    // horses[i] = null;
```

```
    for (; i < horseCount - 1; i++)  
        horses[i] = horses[i + 1];
```

```
    horses[i] = null;
```

```
    horseCount--;
```

```
    return true;
```

```
}
```

```
...
```

```
}
```

הסוסים ממויינים ולכן כל עוד מחיר הסוס שיש למצוא קטן ממחיר הסוס התורן במערך, או שהוא שונה ממנו, והוא גם שונה ממנו נתקדם

לא מצאנו את הסוס

רק הפרש יכול להוציא סוס מהמועדון, ולכן לא נבדוק שוויון נתונים (Equals), אלא האם מדובר באותו עצם ממש

מיותר, כי ידרס מייד ע"י הסוס הבא, ואם אחרון – ידרס כאחרון

העתקת הפניות אחורה – כל תא יקבל את ההפניה אחריו

○ הסוסים מסודרים מהסוס היקר לסוס הזול

○ פעולת ההוצאה מהמועדון אינה מחיקת הסוס, כיוון שזה נשאר אצל הרוכב, אלא מחיקת ההפניה הנוספת מתוך מערך המועדון

```
class Program
{
    static void Main(string[] args)
    {
        EquestrianClub club = new EquestrianClub(20);
        Rider rider1 = new Rider("Oliver", 2015, 9, club);
        Rider rider2 = new Rider("Harper", 2017, 6, club);
        Rider rider3 = new Rider("Mia", 2020, 7, club);

        // קיים כאן קטע קוד המכניס חלק מהסוסים של כל פרש למועדון

        rider1.Print();
        rider2.Print();
        rider3.Print();

        club.ListHorses();

        rider1.RemoveFromClub(0);
        rider1.Print();
        club.ListHorses();

        rider2.RemoveFromClub(0);
        rider2.Print();
        club.ListHorses();

        rider3.RemoveFromClub(0);
        rider3.Print();
        club.ListHorses();

    }
}
```

```

class Program
{
    static void Main(string[] args)
    {
        EquestrianClub club = new EquestrianClub(20);
        Rider rider1 = new Rider("Oliver", 2015, 9, club);
        Rider rider2 = new Rider("Harper", 2017, 6, club);
        Rider rider3 = new Rider("Mia", 2020, 7, club);

        // קיים כאן קטע קוד המכניס חלק מהסוסים של כל פרש למועדון

        rider1.Print();
        rider2.Print();
        rider3.Print();

        club.ListHorses();

    }
}

```

Rider: Oliver , Year: 2015:

- 0) Racing Horse: Silver, Price: 31124, Speed: 22.39 mph
- 2) Racing Horse: Blaze, Price: 38166, Speed: 12.17 mph
- 4) Racing Horse: Storm, Price: 51747, Speed: 16.32 mph
- 6) Racing Horse: Midnight, Price: 16963, Speed: 10.88 mph
- 8) Racing Horse: Shadow, Price: 17849, Speed: 18.11 mph

Rider: Harper , Year: 2017:

- 0) Racing Horse: Thunder, Price: 52365, Speed: 11.58 mph
- 2) Racing Horse: Shadow, Price: 35111, Speed: 14.85 mph
- 4) Racing Horse: Blaze, Price: 43757, Speed: 23.6 mph
- 7) Racing Horse: Silver, Price: 54554, Speed: 23.03 mph

Rider: Mia , Year: 2020:

- 0) Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
- 2) Racing Horse: Midnight, Price: 26042, Speed: 19.63 mph
- 6) Racing Horse: Storm, Price: 20618, Speed: 11.82 mph

Racing Horses in the equestrian club:

- 0) Racing Horse: Silver, Price: 54554, Speed: 23.03 mph
- 1) Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
- 2) Racing Horse: Thunder, Price: 52365, Speed: 11.58 mph
- 3) Racing Horse: Storm, Price: 51747, Speed: 16.32 mph
- 4) Racing Horse: Blaze, Price: 43757, Speed: 23.6 mph
- 5) Racing Horse: Blaze, Price: 38166, Speed: 12.17 mph
- 6) Racing Horse: Silver, Price: 31124, Speed: 22.39 mph
- 7) Racing Horse: Storm, Price: 20618, Speed: 11.82 mph
- 8) Racing Horse: Shadow, Price: 17849, Speed: 18.11 mph
- 9) Racing Horse: Midnight, Price: 16963, Speed: 10.88 mph

```

class Program
{
    static void Main(string[] args)
    {
        EquestrianClub club = new EquestrianClub(20);
        Rider rider1 = new Rider("Oliver", 2015, 9, club);
        Rider rider2 = new Rider("Harper", 2017, 6, club);
        Rider rider3 = new Rider("Mia", 2020, 7, club);

        // קיים כאן קטע קוד המכניס חלק מהסוסים של כל פרש למועדון

        rider1.Print();
        rider2.Print();
        rider3.Print();

        club.ListHorses();

    }
}

```

Rider: Oliver , Year: 2015:

0)	Racing Horse: Silver, Price: 31124, Speed: 22.39 mph
2)	Racing Horse: Blaze, Price: 38166, Speed: 12.17 mph
4)	Racing Horse: Storm, Price: 51747, Speed: 16.32 mph
6)	Racing Horse: Midnight, Price: 16963, Speed: 10.88 mph
8)	Racing Horse: Shadow, Price: 17849, Speed: 18.11 mph

Rider: Harper , Year: 2017:

0)	Racing Horse: Thunder, Price: 52365, Speed: 11.58 mph
2)	Racing Horse: Shadow, Price: 35111, Speed: 14.85 mph
4)	Racing Horse: Blaze, Price: 43757, Speed: 23.6 mph
7)	Racing Horse: Silver, Price: 54554, Speed: 23.03 mph

Rider: Mia , Year: 2020:

0)	Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
2)	Racing Horse: Midnight, Price: 26042, Speed: 19.63 mph
6)	Racing Horse: Storm, Price: 20618, Speed: 11.82 mph

Racing Horses in the equestrian club:

0)	Racing Horse: Silver, Price: 54554, Speed: 23.03 mph
1)	Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
2)	Racing Horse: Thunder, Price: 52365, Speed: 11.58 mph
3)	Racing Horse: Storm, Price: 51747, Speed: 16.32 mph
4)	Racing Horse: Blaze, Price: 43757, Speed: 23.6 mph
5)	Racing Horse: Blaze, Price: 38166, Speed: 12.17 mph
6)	Racing Horse: Silver, Price: 31124, Speed: 22.39 mph
7)	Racing Horse: Storm, Price: 20618, Speed: 11.82 mph
8)	Racing Horse: Shadow, Price: 17849, Speed: 18.11 mph
9)	Racing Horse: Midnight, Price: 16963, Speed: 10.88 mph

```

class Program
{
    static void Main(string[] args)
    {
        EquestrianClub club = new EquestrianClub(20);
        Rider rider1 = new Rider("Oliver", 2015, 9, club);
        Rider rider2 = new Rider("Harper", 2017, 6, club);
        Rider rider3 = new Rider("Mia", 2020, 7, club);

        // קיים כאן קטע קוד המכניס חלק מהסוסים של כל פרש למועדון

        rider1.Print();
        rider2.Print();
        rider3.Print();

        club.ListHorses();

    }
}

```

Rider: Oliver , Year: 2015:

- 0) Racing Horse: Silver, Price: 31124, Speed: 22.39 mph
- 2) Racing Horse: Blaze, Price: 38166, Speed: 12.17 mph
- 4) Racing Horse: Storm, Price: 51747, Speed: 16.32 mph
- 6) Racing Horse: Midnight, Price: 16963, Speed: 10.88 mph
- 8) Racing Horse: Shadow, Price: 17849, Speed: 18.11 mph

Rider: Harper , Year: 2017:

- 0) Racing Horse: Thunder, Price: 52365, Speed: 11.58 mph
- 2) Racing Horse: Shadow, Price: 35111, Speed: 14.85 mph
- 4) Racing Horse: Blaze, Price: 43757, Speed: 23.6 mph
- 7) Racing Horse: Silver, Price: 54554, Speed: 23.03 mph

Rider: Mia , Year: 2020:

- 0) Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
- 2) Racing Horse: Midnight, Price: 26042, Speed: 19.63 mph
- 6) Racing Horse: Storm, Price: 20618, Speed: 11.82 mph

Racing Horses in the equestrian club:

- 0) Racing Horse: Silver, Price: 54554, Speed: 23.03 mph
- 1) Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
- 2) Racing Horse: Thunder, Price: 52365, Speed: 11.58 mph
- 3) Racing Horse: Storm, Price: 51747, Speed: 16.32 mph
- 4) Racing Horse: Blaze, Price: 43757, Speed: 23.6 mph
- 5) Racing Horse: Blaze, Price: 38166, Speed: 12.17 mph
- 6) Racing Horse: Silver, Price: 31124, Speed: 22.39 mph
- 7) Racing Horse: Storm, Price: 20618, Speed: 11.82 mph
- 8) Racing Horse: Shadow, Price: 17849, Speed: 18.11 mph
- 9) Racing Horse: Midnight, Price: 16963, Speed: 10.88 mph

```

class Program
{
    static void Main(string[] args)
    {
        EquestrianClub club = new EquestrianClub(20);
        Rider rider1 = new Rider("Oliver", 2015, 9, club);
        Rider rider2 = new Rider("Harper", 2017, 6, club);
        Rider rider3 = new Rider("Mia", 2020, 7, club);

        // קיים כאן קטע קוד המכניס חלק מהסוסים של כל פרש למועדון

        rider1.Print();
        rider2.Print();
        rider3.Print();

        club.ListHorses();

    }
}

```

Rider: Oliver , Year: 2015:

- 0) Racing Horse: Silver, Price: 31124, Speed: 22.39 mph
- 2) Racing Horse: Blaze, Price: 38166, Speed: 12.17 mph
- 4) Racing Horse: Storm, Price: 51747, Speed: 16.32 mph
- 6) Racing Horse: Midnight, Price: 16963, Speed: 10.88 mph
- 8) Racing Horse: Shadow, Price: 17849, Speed: 18.11 mph

Rider: Harper , Year: 2017:

- 0) Racing Horse: Thunder, Price: 52365, Speed: 11.58 mph
- 2) Racing Horse: Shadow, Price: 35111, Speed: 14.85 mph
- 4) Racing Horse: Blaze, Price: 43757, Speed: 23.6 mph
- 7) Racing Horse: Silver, Price: 54554, Speed: 23.03 mph

Rider: Mia , Year: 2020:

- 0) Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
- 2) Racing Horse: Midnight, Price: 26042, Speed: 19.63 mph
- 6) Racing Horse: Storm, Price: 20618, Speed: 11.82 mph

Racing Horses in the equestrian club:

- 0) Racing Horse: Silver, Price: 54554, Speed: 23.03 mph
- 1) Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
- 2) Racing Horse: Thunder, Price: 52365, Speed: 11.58 mph
- 3) Racing Horse: Storm, Price: 51747, Speed: 16.32 mph
- 4) Racing Horse: Blaze, Price: 43757, Speed: 23.6 mph
- 5) Racing Horse: Blaze, Price: 38166, Speed: 12.17 mph
- 6) Racing Horse: Silver, Price: 31124, Speed: 22.39 mph
- 7) Racing Horse: Storm, Price: 20618, Speed: 11.82 mph
- 8) Racing Horse: Shadow, Price: 17849, Speed: 18.11 mph
- 9) Racing Horse: Midnight, Price: 16963, Speed: 10.88 mph

```

class Program
{
    static void Main(string[] args)
    {
        EquestrianClub club = new EquestrianClub(20);
        Rider rider1 = new Rider("Oliver", 2015, 9, club);
        Rider rider2 = new Rider("Harper", 2017, 6, club);
        Rider rider3 = new Rider("Mia", 2020, 7, club);

        // קיים כאן קטע קוד המכניס חלק מהסוסים של כל פרש למועדון

        rider1.Print();
        rider2.Print();
        rider3.Print();

        club.ListHorses();

        rider1.RemoveFromClub(0);
        rider1.Print();
        club.ListHorses();

        rider2.RemoveFromClub(0);
        rider2.Print();
        club.ListHorses();

    }
}

```

Rider: Harper , Year: 2017:

```

0) Racing Horse: Thunder, Price: 52365, Speed: 11.58 mph
2) Racing Horse: Shadow, Price: 35111, Speed: 14.85 mph
4) Racing Horse: Blaze, Price: 43757, Speed: 23.6 mph
7) Racing Horse: Silver, Price: 54554, Speed: 23.03 mph

```

Racing Horses in the equestrian club:

```

0) Racing Horse: Silver, Price: 54554, Speed: 23.03 mph
1) Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
2) Racing Horse: Thunder, Price: 52365, Speed: 11.58 mph
3) Racing Horse: Storm, Price: 51747, Speed: 16.32 mph
4) Racing Horse: Blaze, Price: 43757, Speed: 23.6 mph
5) Racing Horse: Blaze, Price: 38166, Speed: 12.17 mph
6) Racing Horse: Storm, Price: 20618, Speed: 11.82 mph
7) Racing Horse: Shadow, Price: 17849, Speed: 18.11 mph
8) Racing Horse: Midnight, Price: 16963, Speed: 10.88 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        EquestrianClub club = new EquestrianClub(20);
        Rider rider1 = new Rider("Oliver", 2015, 9, club);
        Rider rider2 = new Rider("Harper", 2017, 6, club);
        Rider rider3 = new Rider("Mia", 2020, 7, club);

        // קיים כאן קטע קוד המכניס חלק מהסוסים של כל פרש למועדון

        rider1.Print();
        rider2.Print();
        rider3.Print();

        club.ListHorses();

        rider1.RemoveFromClub(0);
        rider1.Print();
        club.ListHorses();

        rider2.RemoveFromClub(0);
        rider2.Print();
        club.ListHorses();

        rider3.RemoveFromClub(0);
        rider3.Print();
        club.ListHorses();

    }
}

```

Rider: Mia , Year: 2020:

```

0) Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
2) Racing Horse: Midnight, Price: 26042, Speed: 19.63 mph
6) Racing Horse: Storm, Price: 20618, Speed: 11.82 mph

```

Racing Horses in the equestrian club:

```

0) Racing Horse: Silver, Price: 54554, Speed: 23.03 mph
1) Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
2) Racing Horse: Storm, Price: 51747, Speed: 16.32 mph
3) Racing Horse: Blaze, Price: 43757, Speed: 23.6 mph
4) Racing Horse: Blaze, Price: 38166, Speed: 12.17 mph
5) Racing Horse: Storm, Price: 20618, Speed: 11.82 mph
6) Racing Horse: Shadow, Price: 17849, Speed: 18.11 mph
7) Racing Horse: Midnight, Price: 16963, Speed: 10.88 mph

```

```

class Program
{
    static void Main(string[] args)
    {
        EquestrianClub club = new EquestrianClub(20);
        Rider rider1 = new Rider("Oliver", 2015, 9, club);
        Rider rider2 = new Rider("Harper", 2017, 6, club);
        Rider rider3 = new Rider("Mia", 2020, 7, club);

        // קיים כאן קטע קוד המכניס חלק מהסוסים של כל פרש למועדון

        rider1.Print();
        rider2.Print();
        rider3.Print();

        club.ListHorses();

        rider1.RemoveFromClub(0);
        rider1.Print();
        club.ListHorses();

        rider2.RemoveFromClub(0);
        rider2.Print();
        club.ListHorses();

        rider3.RemoveFromClub(0);
        rider3.Print();
        club.ListHorses();

    }
}

```

Rider: Mia , Year: 2020:

- 0) Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
- 2) Racing Horse: Midnight, Price: 26042, Speed: 19.63 mph
- 6) Racing Horse: Storm, Price: 20618, Speed: 11.82 mph

Racing Horses in the equestrian club:

- 0) Racing Horse: Silver, Price: 54554, Speed: 23.03 mph
- 1) Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
- 2) Racing Horse: Storm, Price: 51747, Speed: 16.32 mph
- 3) Racing Horse: Blaze, Price: 43757, Speed: 23.6 mph
- 4) Racing Horse: Blaze, Price: 38166, Speed: 12.17 mph
- 5) Racing Horse: Storm, Price: 20618, Speed: 11.82 mph
- 6) Racing Horse: Shadow, Price: 17849, Speed: 18.11 mph
- 7) Racing Horse: Midnight, Price: 16963, Speed: 10.88 mph

Rider: Mia , Year: 2020:

- 0) Racing Horse: Blaze, Price: 53145, Speed: 10.43 mph
- 2) Racing Horse: Midnight, Price: 26042, Speed: 19.63 mph
- 6) Racing Horse: Storm, Price: 20618, Speed: 11.82 mph

Racing Horses in the equestrian club:

- 0) Racing Horse: Silver, Price: 54554, Speed: 23.03 mph
- 1) Racing Horse: Storm, Price: 51747, Speed: 16.32 mph
- 2) Racing Horse: Blaze, Price: 43757, Speed: 23.6 mph
- 3) Racing Horse: Blaze, Price: 38166, Speed: 12.17 mph
- 4) Racing Horse: Storm, Price: 20618, Speed: 11.82 mph
- 5) Racing Horse: Shadow, Price: 17849, Speed: 18.11 mph
- 6) Racing Horse: Midnight, Price: 16963, Speed: 10.88 mph

```
class Rider
```

```
{
```

```
    public bool RemoveHorse(String name)
```

```
    {
```

```
        for (int i = 0; i < horses.Length; i++)
```

```
            if (horses[i] != null && horses[i].GetName() == name)
```

```
            {
```

```
                horses[i] = null;
```

```
                return true;
```

```
            }
```

```
        return false;
```

```
    }
```

```
    ...
```

```
}
```

תיקון מחלקת פרש Rider

תיקון פעולת מחיקת סוס

ראינו שמחיקת הסוס ממחלקת *O*

פרש אינה מוחקת את הסוס

מהמועדון

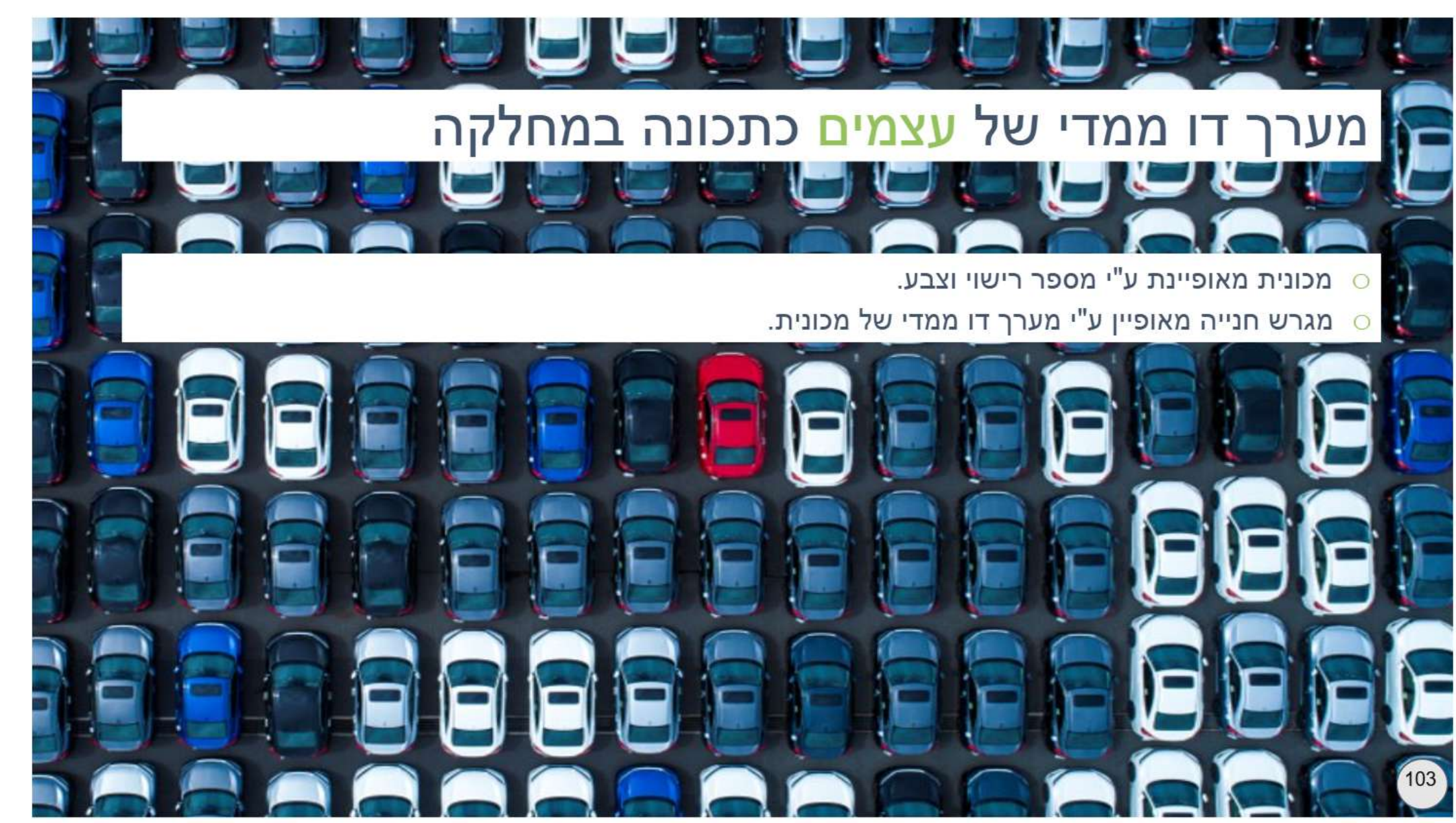
נוסיף הוראה המוחקת את הסוס *O*

גם מהמועדון

הכלת עצמים

מחלקה המכילה מערך דו ממדי של עצמים





מערך דו ממדי של עצמים כתכונה במחלקה

○ מכונית מאופיינת ע"י מספר רישוי וצבע.

○ מגרש חנייה מאופיין ע"י מערך דו ממדי של מכונית.

מחלקת מכונית Car

```
class Car
{
    private int licenseNumber;
    private string color;

    public Car(int licenseNumber, string color)
    {
        this.licenseNumber = licenseNumber;
        this.color = color;
    }

    public int GetLicenseNumber()
    {
        return this.licenseNumber;
    }

    public override string ToString()
    {
        return this.licenseNumber + "(" + this.color + ")";
    }
}
```

מחלקת מגרש חניה ParkingLot

```
class ParkingLot
{
    private Car[,] allSpots;

    public ParkingLot(int rows, int cols)
    {
        this.allSpots = new Car[rows, cols];
    }

    public bool ParkCar(Car theCar, int row, int col) { ... }
    public bool RemoveCar(Car theCar) { ... }
    public override string ToString() { ... }
}
```

○ הפעולה הבונה מקבלת מספר שורות רצוי ומספר עמודות רצוי, ובונה מערך דו ממדי מטיפוס Car.

○ בשלב זה, תאי המערך הם null, ערך ברירת המחדל להפניות

```

static void Main(string[] args)
{
    ParkingLot pl = new ParkingLot(3, 5);

    Car c1 = new Car(111, "orange");
    pl.ParkCar(c1, 0, 0);
    Car c2 = new Car(222, "blue");
    pl.ParkCar(c2, 1, 2);
    Car c3 = new Car(333, "yellow");
    pl.ParkCar(c3, 2, 3);
    → { Car c4 = new Car(444, "red");
        pl.ParkCar(c4, 2, 0);

    Console.WriteLine(pl.ToString());

    pl.RemoveCar(c2);
    Console.WriteLine(pl.ToString());
}

```

שימוש במחלקה ParkingLot

- נרצה שתהיה לנו האפשרות להכניס מכונית למגרש החנייה במיקום מסויים – שורה ועמודה.
- לשם כך עלינו לבנות עצם מטיפוס מגרש חניה, ולכתוב פעולה המקבלת מכונית ומיקום, שתפעל על העצם.

```
static void Main(string[] args)
{
    ParkingLot pl = new ParkingLot(3, 5);

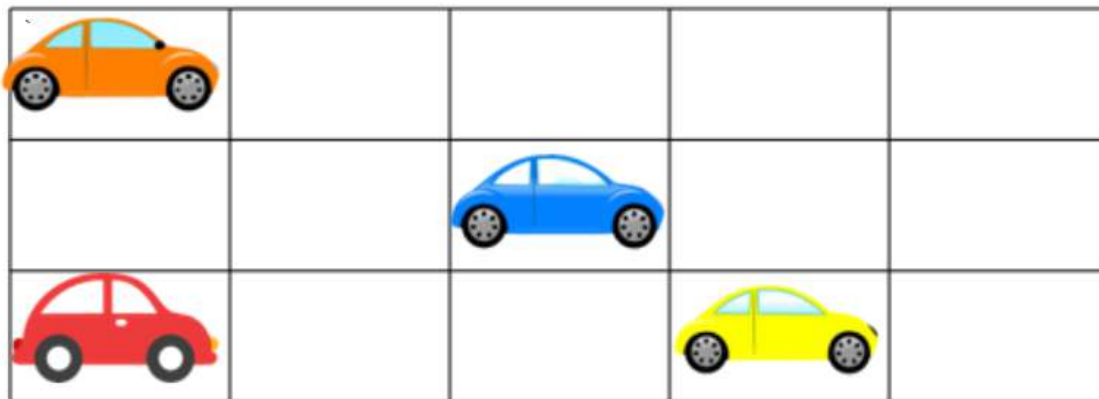
    Car c1 = new Car(111, "orange");
    pl.ParkCar(c1, 0, 0);
    Car c2 = new Car(222, "blue");
    pl.ParkCar(c2, 1, 2);
    Car c3 = new Car(333, "yellow");
    pl.ParkCar(c3, 2, 3);
    Car c4 = new Car(444, "red");
    pl.ParkCar(c4, 2, 0);

    → Console.WriteLine(pl.ToString());

    pl.RemoveCar(c2);
    Console.WriteLine(pl.ToString());
}
```

שימוש במחלקה ParkingLot

- נרצה שתהיה לנו האפשרות להכניס מכונית למגרש החנייה במיקום מסויים – שורה ועמודה.
- לשם כך עלינו לבנות עצם מטיפוס מגרש חניה, ולכתוב פעולה המקבלת מכונית ומיקום, שתפעל על העצם.



111(orange)									

				222(blue)					

444(red)						333(yellow)			

```
static void Main(string[] args)
{
    ParkingLot pl = new ParkingLot(3, 5);

    Car c1 = new Car(111, "orange");
    pl.ParkCar(c1, 0, 0);
    Car c2 = new Car(222, "blue");
    pl.ParkCar(c2, 1, 2);
    Car c3 = new Car(333, "yellow");
    pl.ParkCar(c3, 2, 3);
    Car c4 = new Car(444, "red");
    pl.ParkCar(c4, 2, 0);

    Console.WriteLine(pl.ToString());

    → pl.RemoveCar(c2);
    Console.WriteLine(pl.ToString());
}
```

שימוש במחלקה ParkingLot

נרצה שתהיה לנו האפשרות להכניס מכונית למגרש החנייה במיקום מסויים – שורה ועמודה.

לשם כך עלינו לבנות עצם מטיפוס מגרש חניה, ולכתוב פעולה המקבלת מכונית ומיקום, שתפעל על העצם.

111(orange)									
				222(blue)					
444(red)						333(yellow)			

```
static void Main(string[] args)
{
    ParkingLot pl = new ParkingLot(3, 5);

    Car c1 = new Car(111, "orange");
    pl.ParkCar(c1, 0, 0);
    Car c2 = new Car(222, "blue");
    pl.ParkCar(c2, 1, 2);
    Car c3 = new Car(333, "yellow");
    pl.ParkCar(c3, 2, 3);
    Car c4 = new Car(444, "red");
    pl.ParkCar(c4, 2, 0);

    Console.WriteLine(pl.ToString());

    pl.RemoveCar(c2);
    → Console.WriteLine(pl.ToString());
}
```

שימוש במחלקה ParkingLot

נרצה שתהיה לנו האפשרות להכניס מכונית למגרש החנייה במיקום מסויים – שורה ועמודה.

לשם כך עלינו לבנות עצם מטיפוס מגרש חניה, ולכתוב פעולה המקבלת מכונית ומיקום, שתפעל על העצם.

111(orange)									
444(red)						333(yellow)			

מחלקת מגרש חניה ParkingLot

פעולת הוספת מכונית

הפעולה מקבלת מכונית ומיקום, ואם המיקום פנוי, היא מכניסה את המכונית.

```
class ParkingLot
{
    private Car[,] allSpots;

    public ParkingLot(int rows, int cols) { ... }

    public bool ParkCar(Car theCar, int row, int col)
    {
        bool ok = (this.allSpots[row, col] == null);
        if (ok)
        {
            this.allSpots[row, col] = theCar;
        }
        return ok;
    }

    public bool RemoveCar(Car theCar) { ... }
    public override string ToString() { ... }
}
```

מחלקת מגרש חניה ParkingLot

```
class ParkingLot
{
    private Car[,] allSpots;

    public ParkingLot(int rows, int cols) { ... }

    public bool ParkCar(Car theCar, int row, int col)
    {
        bool ok = (this.allSpots[row, col] == null);
        if (ok)
        {
            this.allSpots[row, col] = theCar;
        }
        return ok;
    }

    public bool RemoveCar(Car theCar) { ... }
    public override string ToString() { ... }
}
```

פעולת הוספת מכונית

- הפעולה מקבלת מכונית ומיקום, ואם המיקום פנוי, היא מכניסה את המכונית.
- הפעולה מקבלת הפניה למכונית יש הפניה כאן ובמקום בו זמנה הפעולה.
- כיוון שמדובר באותה מכונית ממש, אין מקום ליצור כאן העתק – אם מצבה המכונית התרוקן. גם בעל המכונית אמור לדעת מכך.
- הפעולה מניחה שהמיקום תקין ובתוך המערך, נוכל בקלות לוודא זאת ע"י בדיקת טווח האינדקסים. כאן, אם יתקבל ערך לא נכון נקבל שגיאה – חריגה ממערך.

```
class ParkingLot
```

```
{  
    private Car[,] allSpots;
```

```
    public bool RemoveCar(Car theCar)
```

```
    {  
        for (int r = 0; r < allSpots.GetLength(0); r++)
```

לולאה על כל השורות

```
        {  
            for (int c=0; c < allSpots.GetLength(1); c++)
```

בכל שורה, לולאה על כל העמודות

```
            {  
                if (this.allSpots[r, c] != null &&
```

```
                    this.allSpots[r, c].GetLicenseNumber() == theCar.GetLicenseNumber())  
                {
```

```
                    this.allSpots[r, c] = null;  
                    return true;  
                }
```

```
            }
```

```
        }
```

```
    }  
    return false;
```

```
}
```

```
}
```

```
...
```

```
}
```

מחלקת מגרש חניה ParkingLot

פעולת הוצאת מכונית

```
class ParkingLot
```

```
{
```

```
    private Car[,] allSpots;
```

```
    public bool RemoveCar(Car theCar)
```

```
    {
```

```
        for (int r = 0; r < allSpots.GetLength(0); r++)
```

```
        {
```

```
            for (int c=0; c < allSpots.GetLength(1); c++)
```

```
            {
```

```
                if (this.allSpots[r, c] != null &&
```

```
                    this.allSpots[r, c].GetLicenseNumber() == theCar.GetLicenseNumber())
```

```
                {
```

```
                    this.allSpots[r, c] = null;
```

```
                    return true;
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
    return false;
```

```
}
```

```
}
```

```
...
```

```
}
```

מחלקת מגרש חניה ParkingLot

פעולת הוצאת מכונית

לולאה על כל השורות

בכל שורה, לולאה על כל העמודות

כיוון שמזמנים פעולה על עצם, חובה לבדוק שההפניה אינה null.
זימון פעולה על null יוביא לשגיאת זמן ריצה מסוג פנייה ל null

```
class ParkingLot
```

```
{  
    private Car[,] allSpots;
```

```
    public bool RemoveCar(Car theCar)
```

```
{  
    for (int r = 0; r < allSpots.GetLength(0); r++)
```

```
{  
        for (int c=0; c < allSpots.GetLength(1); c++)
```

```
{  
            if (this.allSpots[r, c] != null &&
```

```
                this.allSpots[r, c].GetLicenseNumber() == theCar.GetLicenseNumber())
```

```
{  
                    this.allSpots[r, c] = null;  
                    return true;
```

```
}
```

```
}
```

```
}
```

```
    }  
    return false;
```

```
}
```

```
}
```

```
...
```

```
}
```

מחלקת מגרש חניה ParkingLot

פעולת הוצאת מכונית

לולאה על כל השורות

בכל שורה, לולאה על כל העמודות

כיוון שמזמנים פעולה על עצם, חובה לבדוק שההפניה אינה null.
זימון פעולה על null יוביא לשגיאת זמן ריצה מסוג פנייה ל null

הפעולה בודקת שוויון במספר הרישוי – כלומר שיתכן ונקבל
כפרמטר מכונית שהיא העתק המכונית, ולא הפניה נוספת
למכונית עצמה.

על מנת "לרוקן" את המקום, נשים בו null

```
class ParkingLot
```

```
{
```

```
    private Car[,] allSpots;
```

```
    public bool RemoveCar(Car theCar)
```

```
    {
```

```
        for (int r = 0; r < allSpots.GetLength(0); r++)
```

```
        {
```

```
            for (int c=0; c < allSpots.GetLength(1); c++)
```

```
            {
```

```
                if (this.allSpots[r, c] != null &&
```

```
                    this.allSpots[r, c].GetLicenseNumber() == theCar.GetLicenseNumber())
```

```
                {
```

```
                    this.allSpots[r, c] = null;
```

```
                    return true;
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
    return false;
```

```
}
```

```
...
```

```
...
```

```
}
```

מחלקת מגרש חניה ParkingLot

פעולת הוצאת מכונית

לולאה על כל השורות

בכל שורה, לולאה על כל העמודות

כיוון שמזמנים פעולה על עצם, חובה לבדוק שההפניה אינה null.
זימון פעולה על null יוביא לשגיאת זמן ריצה מסוג פנייה ל null

הפעולה בודקת שוויון במספר הרישוי – כלומר שיתכן ונקבל
כפרמטר מכונית שהיא העתק המכונית, ולא הפניה נוספת
למכונית עצמה.

על מנת "לרוקן" את המקום, נשים בו null

```
class ParkingLot
```

```
{  
    private Car[,] allSpots;  
  
    public override string ToString()  
    {  
        string res = "";  
        for (int r = 0; r < this.allSpots.GetLength(0); r++)  
        {  
            for (int c = 0; c < this.allSpots.GetLength(1); c++)  
            {  
                if (this.allSpots[r, c] != null)  
                    res = res + "   " + this.allSpots[r, c].ToString() + "\t|";  
                else  
                    res = res + "\t\t|";  
            }  
            res = res + "\n-----\n";  
        }  
        return res;  
    }  
}  
  
...
```

ParkingLot מחלקת מגרש חניה

פעולת ToString

מחלקה המכילה עצם ממחלקה אחרת

- תכונה יכולה להיות עצם.
- התכונה אינה העצם עצמו, אלא היא הפניה לעצם, ככל משתנה הפניה.
- בהתאם לבעיה, יש להחליט אם רק במחלקה תהיה הפניה לעצם, או גם מחוץ לה:
 - בפעולה הבונה ניצור עותק של האובייקט א נעתיק את ההפניה.
 - פעולות אחזור Get של אובייקט יחזירו עותק א הפניה.
 - פעולות קובעות Set של אובייקט ישמרו כתכונה עותק א הפניה.
- במקרים של תכונה מטיפוס מערך:
 - ניהול המערך (הכנסה והוצאה) ייעשה בתוך המחלקה, על בסיס מקום פנוי או תוך שימוש בתכונה נוספת של כמות האיברים בפועל.
 - במקום פעולות אחזור Get וקביעה Set, נשתמש בפעולות הוספה, מחיקה ועדכון.
 - אם נרצה להחזיר מערך, נעדיף ליצור העתק שלו.